天链机械臂 SDK 使用说明文档

C++ 开发接口手册

支持平台：Linux(x86-64、ARM)、Windows

（Ver.2.6.0） 天链机器人有限责任公司 销售联系（Tel）：18090080872

版本号	修改内容	修改日期	修改人
V1.0	完成说明文档初步全部撰写	2026.5.28	刘鑫
V2.0	文档内容修改完善	2026.8.07	曹芩滔
V2.5.1	按 V2.5.1 接口面裁剪：移除 V3 专有接口章节、SDK 扩展（实验性）接口章节	2026.9.16	ShiHongchao
V2.5.2	接口面未变更，版本号随 SDK V2.5.2 同步	2026.9.21	ShiHongchao
V2.5.3	分发包结构补充 Windows 调试版 C++ 库说明；接口面未变更，版本号随 SDK V2.5.3 同步	2026.10.8	ShiHongchao
V2.6.0	新增周期同步速度/扭矩（CSV/CST）与拖动轨迹管理接口说明；`DragParam` 字段随协议更新；版本号随 SDK V2.6.0 同步	2026.10.23	ShiHongchao

目录

- 一．概述
- 二．支持的操作系统与软件版本
- 三．SDK结构
- 四．编译命令示例
- 五．运行时动态库搜索路径设置命令示例
- 六．SDK接口说明
- 6.1 接口通用规则
- 6.2 连接管理接口
    - 6.2.1 connect_robot - 创建机械臂双端口连接控制句柄
    - 6.2.2 disconnect_robot - 关闭机械臂通信句柄
    - 6.2.3 get_connection_status - 获取控制器连接状态
    - 6.2.4 get_controller_network_config - 获取控制器网络配置
    - 6.2.5 set_controller_network_config - 设置控制器网络配置

- 6.3 日志管理接口
  - 6.3.1 log_download_by_quantity - 保存日志到文件
  - 6.3.2 set_log_level - 设置日志级别
- 6.4 基础状态查询接口
  - 6.4.1 get_current_position - 获取机械臂当前坐标信息
  - 6.4.2 get_robot_running_state - 获取机器人运行状态
  - 6.4.3 get_servo_state - 获取伺服状态
  - 6.4.4 get_current_mode - 获取机器人当前模式
  - 6.4.5 get_robot_dh_param - 获取机械臂DH参数
  - 6.4.6 get_library_version - 读取机械臂软件信息
  - 6.4.7 get_robot_joint_param - 获取指定关节配置参数
  - 6.4.8 query_joint_software_version - 查询指定关节（轴）的软件版本号
  - 6.4.9 get_nexmotion_lib_version - 获取底层 nexmotion 库版本信息
  - 6.4.10 get_robot_state - 获取机器人整体状态
  - 6.4.11 get_robot_type - 获取机器人类型
  - 6.4.12 get_current_line_speed_and_joint_speed - 获取当前线速度与关节速度
  - 6.4.13 get_pos_reachable - 查询点位可达性
  - 6.4.14 robot_state_callback - 注册机器人状态回调
  - 6.4.15 get_speed - 获取速度
  - 6.4.16 get_cartesian_params - 获取笛卡尔参数
- 6.5 伺服状态与系统参数设置接口
  - 6.5.1 clear_error - 清除系统错误
  - 6.5.2 set_servo_state - 设置伺服就绪状态
  - 6.5.3 set_servo_poweron - 设置伺服上电状态
  - 6.5.4 set_servo_poweroff - 设置伺服下电状态
  - 6.5.5 set_speed - 设置速度
  - 6.5.6 restore_default_param_DH - 恢复机械臂默认DH参数
  - 6.5.7 set_cartesian_params - 设置机器人笛卡尔参数
  - 6.5.8 restore_network_factory_settings - 恢复网络出厂设置
  - 6.5.9 set_robot_joint_param - 设置当前机器人关节参数
  - 6.5.10 set_controller_ip - 配置控制器有线网口IP
  - 6.5.11 set_robot_dh_param - 设置机器人 DH 参数
  - 6.5.12 set_zero_pos_deviation - 设置零位偏差
  - 6.5.13 set_axis_zero_position - 指定关节当前位置为零位/标定当前状态下的零位
  - 6.5.14 set_receive_error_or_warnning_message_callback - 设置错误警告回调
- 6.6 六维力传感器查询接口
  - 6.6.1 set_six_dimensional_force_communication_params - 设置六维力传感器通讯参数
  - 6.6.2 get_six_dimensional_force_communication_params - 获取六维力传感器通讯参数
  - 6.6.3 get_sensor_base_param - 获取六维力传感器的基础参数
  - 6.6.4 mark_base_sensor - 执行六维力传感器标定

    - 6.6.5 get_sensor_6d_data - 获取六维力传感器数据
    - 6.6.6 get_payload_param_by_sensor - 获取六维力传感器负载参数
- 6.7 碰撞检测接口
    - 6.7.1 set_collision - 设置碰撞防护参数
    - 6.7.2 get_collision - 获取碰撞防护参数
- 6.8 电机与关节监控接口
    - 6.8.1 get_current_motor_speed - 获取各关节电机当前转速
    - 6.8.2 get_current_motor_torque - 获取各关节电机当前扭矩
    - 6.8.3 get_current_motor_current - 获取各关节电机当前电流
    - 6.8.4 get_current_motor_payload - 获取各关节电机当前负载
    - 6.8.5 get_joint_voltage - 获取各关节电压
    - 6.8.6 get_joint_temperature - 获取各关节电机温度
- 6.9 作业运动控制接口
    - 6.9.1 job_create - 创建作业文件名
    - 6.9.2 job_open - 打开指定作业文件
    - 6.9.3 job_insert_moveJ - 作业里添加关节空间运动
    - 6.9.4 job_insert_moveL - 作业里添加笛卡尔空间直线运动
    - 6.9.5 job_insert_moveC - 作业里添加笛卡尔空间圆弧运动
    - 6.9.6 job_run - 启动指定作业文件
    - 6.9.7 job_get_all_jobfile_name - 获取全部作业文件名
- 6.10 实时运动控制接口
    - 6.10.1 robot_movej - 关节空间运动到目标位姿
    - 6.10.2 robot_movel - 笛卡尔空间直线运动到目标位姿
    - 6.10.3 robot_go_home - 机器人回原点（home）
    - 6.10.4 robot_go_to_reset_position - 机器人回到复位位置
- 6.11 坐标系与用户坐标管理接口
    - 6.11.1 set_user_coordinate_data - 设置用户坐标
    - 6.11.2 set_user_coord_number - 设置用户坐标编号
    - 6.11.3 get_user_coord_para - 获取当前使用的用户坐标的参数
    - 6.11.4 get_user_coord_number - 获取当前使用的用户坐标编号
    - 6.11.5 get_current_coord - 获取当前工作坐标系
    - 6.11.6 set_current_coord - 切换示教运动坐标系
    - 6.11.7 calibration_oxy - 三点法自动设置用户坐标系
    - 6.11.8 calculate_user_coordinate - 计算用户坐标
    - 6.11.9 get_target_coord_pos_value - 获取目标坐标位置值
    - 6.11.10 get_local_pos_p_value - 获取本地位置变量 P 值
    - 6.11.11 set_default_cartesian_params - 设置笛卡尔参数为默认值
- 6.12 工具手相关接口
    - 6.12.1 set_tool_hand_number - 设置工具手编号
    - 6.12.2 get_tool_hand_number - 获取当前工具手编号

    - 6.12.3 set_tool_hand_param - 设置工具手参数
    - 6.12.4 get_tool_hand_param - 获取当前工具手参数
    - 6.12.5 set_tool_coordinate_range - 设置工具包络球参数（工具坐标范围参数）
    - 6.12.6 get_tool_coordinate_range - 获取工具包络球参数（工具坐标范围参数）
- 6.13 示教操作接口
    - 6.13.1 robot_start_jogging - 开始点动
    - 6.13.2 robot_stop_jogging - 停止点动
- 6.14 全局路点管理接口
    - 6.14.1 set_global_position - 新增/设置全局路点
    - 6.14.2 get_global_position - 查询全局 GP 点位
    - 6.14.3 set_global_string - 设置全局字符串变量
    - 6.14.4 get_global_string - 获取全局字符串变量
    - 6.14.5 get_global_sync_position - 获取全局同步位置
    - 6.14.6 get_global_variant - 获取全局变量
- 6.15 模式与 IO 控制接口
    - 6.15.1 set_current_mode - 设置机器人模式（切换示教模式和运动模式）
    - 6.15.2 set_digital_output - 设置数字 IO 输出
    - 6.15.3 get_digital_output - 查询数字 IO 输出
    - 6.15.4 get_digital_input - 获取所有 IO 输入状态
    - 6.15.5 set_analog_output - 设置模拟量输出
    - 6.15.6 get_analog_input - 获取模拟量输入
    - 6.15.7 get_analog_output - 获取模拟量输出
    - 6.15.8 get_io_type - 获取 IO 类型
    - 6.15.9 get_IO_reset_function - 获取 IO 复位功能
- 6.16 算法接口
    - 6.16.1 get_origin_coord_to_target_coord - 坐标转换算法
    - 6.16.2 get_quat2rpy - 四元数 转 欧拉角
    - 6.16.3 get_rpy2quat - 欧拉角 转 四元数
    - 6.16.4 get_rpy2r - 欧拉角 转 旋转矩阵
    - 6.16.5 get_tr2r - 位姿 转 旋转矩阵
    - 6.16.6 get_r2tr - 旋转矩阵 转 位姿矩阵
- 6.17 modbus通信接口
    - 6.17.1 modbus_set_master_parameter - 设置 Modbus RTU 主站参数
    - 6.17.2 modbus_open_master - 打开 Modbus 主站
    - 6.17.3 modbus_write_multiple_holding_registers - 写多个保持寄存器
    - 6.17.4 modbus_read_holding_registers - Modbus RTU 读取多个保持寄存器
    - 6.17.5 modbus_write_signal_coil_status - Modbus RTU协议写线圈
    - 6.17.6 modbus_write_signal_holding_registers - Modbus RTU协议写保持寄存器
    - 6.17.7 modbus_read_coil_status - Modbus读线圈状态
    - 6.17.8 modbus_read_input_status - Modbus读取输入状态

    - 6.17.9 modbus_read_input_registers - Modbus 读输入寄存器
- 6.18 队列运动接口
    - 6.18.1 queue_motion_set_status - 打开、关闭队列运动模式
    - 6.18.2 queue_motion_push_back_moveJ - 队列运动模式的本地队列最后插入一条move J 运动
    - 6.18.3 queue_motion_push_back_moveL - 队列运动模式的本地队列最后插入一条move L 运动
    - 6.18.4 queue_motion_push_back_moveC - 队列运动本地队列末尾插入一条 MoveC 运动
    - 6.18.5 queue_motion_push_back_moveS - 队列运动本地队列末尾插入一条 MoveS 运动
    - 6.18.6 queue_motion_send_to_controller - 将本地队列的前size个数据发送到控制器
    - 6.18.7 queue_motion_get_status - 查询队列模式状态
    - 6.18.8 queue_motion_size - 查询当前运动队列的长度
    - 6.18.9 queue_motion_get_queuelen - 查询当前运动队列剩余的指令数量
    - 6.18.10 queue_motion_suspend - 暂停连续运动（轨迹暂停）
    - 6.18.11 queue_motion_restart - 暂停后再次运行运动队列（继续当前轨迹运动）
    - 6.18.12 queue_motion_stop - 停止队列运动
    - 6.18.13 queue_motion_stop_not_power_off - 停止连续运动保持上电状态（轨迹缓停）
    - 6.18.14 queue_motion_clear_Data - 清空本地队列数据
- 6.19 拖拽示教与轨迹记录回放
    - 6.19.1 set_darg_mode - 设置拖拽示教的拖拽方式
    - 6.19.2 set_teach_type - 设置示教模式类型
    - 6.19.3 get_teach_type - 查询示教模式类型
    - 6.19.4 set_current_teach_sensitivity - 设置电流环拖动示教灵敏度
    - 6.19.5 get_current_teach_sensitivity - 获取电流环拖动示教灵敏度
    - 6.19.6 get_drag_thread_is_end - 查询拖拽是否结束
    - 6.19.7 track_record_start - 轨迹记录开始
    - 6.19.8 get_track_record_status - 轨迹记录开启状态查询
    - 6.19.9 track_record_stop - 轨迹记录关闭
    - 6.19.10 track_record_save - 保存拖动示教轨迹
    - 6.19.11 track_record_playback - 轨迹回放
    - 6.19.12 track_record_delete - 轨迹清除
    - 6.19.13 drag_trajectory_export - 导出拖动轨迹
    - 6.19.14 drag_trajectory_import - 导入拖动轨迹
    - 6.19.15 drag_trajectory_playback - 回放指定拖动轨迹
    - 6.19.16 drag_trajectory_delete - 删除指定拖动轨迹
- 七．7000端口接口
- 7.1 以下功能在连接7000端口后使用
    - 7.1.1 open_servoJ - 打开关节跟踪模式
    - 7.1.2 close_servoJ - 关闭关节跟踪模式
    - 7.1.3 set_servoJ_pos - 设置实时跟踪关节位置

    - 7.1.4 open_servo_csv_mode - 打开周期同步速度模式
    - 7.1.5 close_servo_csv_mode - 关闭周期同步速度模式
    - 7.1.6 set_servo_csv_velocity - 下发周期同步速度
    - 7.1.7 open_servo_cst_mode - 打开周期同步扭矩模式
    - 7.1.8 close_servo_cst_mode - 关闭周期同步扭矩模式
    - 7.1.9 set_servo_cst_torque - 下发周期同步扭矩
- 八． 使用示例
- 8.1 Modbus主站通信综合示例
- 8.2 关节实时跟踪servoJ使用示例
- 8.3 关节运动到目标点robot_movej与直线运动到目标位置robot_movel使用示例
- 8.4 点动使用示例
- 8.5 作业相关接口使用示例
- 8.6 队列运动使用示例
- 8.7 坐标转换算法示例
- 8.8 模式与 IO 控制接口示例
- 8.9 拖拽示教与轨迹记录/回放示例
- 附录（一）

一．概述

本说明文档为四川天链机器人股份有限公司研发生产的超轻量协作机器人TCB系列C++开发包SDK使用说明文档，通过该文档，用户能够更好使用SDK控制TCB机械臂执行相应任务，更加便捷地进行二次开发。

二．支持的操作系统与软件版本

1. 操作系统：目前支持 Linux（x86 和 ARM）、Windows（x64）。

2. 编译环境：GCC 5.4+ ，支持 C++17 标准。

3. 依赖库：libpthread，动态链接库：libtl_host.so

三．SDK结构

SDK结构如下：

```
| LICENSE
| README.md
|
├─docs                # 说明文档
|
```

```
├─examples          # 示例程序
│ ├─cpp
│ └─py
│
├─include           # 对外头文件（namespace tl）
│ ├─tl_api.h
│ ├─tl_interface.h
│ ├─tl_types.h
│ └─...             # tl_force / tl_servo_ext 等
│
├─lib               # 平台动态库 + Python 绑定
│ ├─linux
│ │ ├─x86           # libtl_host.so + python (_tl_host + tl_interface.py)
│ │ └─arm64         # libtl_host.so + python (_tl_host + tl_interface.py)
│ ├─windows         # tl_host.dll + tl_host.lib + python (tl_host.pyd)
│ │ └─debug         # 调试版 C++ 库：tl_host.dll + tl_host.lib + tl_host.pdb
│
└─src
  ├─tl_host         # tl_host 合并库源码
  └─python_bind     # nanobind Python 绑定
```

四．编译命令示例

注意：每一次修改代码后，需要进行编译，才能进行使用。

五．运行时动态库搜索路径设置命令示例

```
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
# 将当前目录加入动态库搜索路径
```

注意：第一次运行时需要运行这段命令

六．sdk接口说明

除特殊说明外，否则默认所有接口在不同模式下通用。

6.1 接口通用规则

1. **头文件包含**：使用C++接口需包含 include/tl_interface.h 等对应头文件。
2. **句柄要求**：除标注为本地计算、无需连接控制器的接口外，接口的第一个参数为 connect_robot 返回的有效 SOCKETFD（即 int 类型）。

3. **返回值约定**：
    - 0（SUCCESS）：执行成功
    - 非 0：执行失败，具体错误码见附录。
4. **数据结构**：
    - 向量类型：std::vector< double >或 std::vector< int >。
    - 结构体类型：ModbusMasterParameter、MoveCmd、RobotDHParam、ToolParam等。
5. **坐标系约定**：
    - 0：关节坐标系（Joint）
    - 1：直角坐标系（Cartesian）
    - 2：工具坐标系（Tool）
    - 3：用户坐标系（User）
6. **工具函数说明**：为保证示例代码输出格式统一、结果清晰规整，本文档所有C++示例统一封装了调试打印工具函数，用于标准化接口返回状态、向量、矩阵、分隔标题的输出展示。
    - print_result：统一打印接口调用结果，区分成功 /失败并输出错误码；
    - print_separator：打印章节分隔标题，用于区分多组测试案例；
    - print_vector：格式化打印一维浮点向量，固定 4位小数，适用于欧拉角、四元数、关节角度等数据；
    - print_matrix：按行主序格式化打印多维矩阵，适配 3×3 旋转矩阵、4×4 位姿矩阵输出。

```
// ============================================================
// 工具函数
// ============================================================

void print_result(const std::string& msg, int ret)
{
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  [成功] " << msg << std::endl;
    } else {
        std::cout << "  [失败] " << msg << ", 错误码: " << ret << std::endl;
    }
}

void print_separator(const std::string& title)
{
    std::cout << "\n========== " << title << " ==========" << std::endl;
}
```

```
/// @brief 打印double 向量
void print_vector(const std::string& name, const std::vector<double>& v)
{
    std::cout << "  " << name << " [";
    for (size_t i = 0; i < v.size(); ++i) {
        std::cout << v[i] << (i + 1 < v.size() ? ", " : "");
    }
    std::cout << "]" << std::endl;
}

/// @brief 打印矩阵（行主序）
void print_matrix(const std::string& name, const std::vector<double>& m, int rows, int c
ols)
{
    std::cout << "  " << name << " [" << m.size() << "]:" << std::endl;
    for (int r = 0; r < rows; ++r) {
        std::cout << "    ";
        for (int c = 0; c < cols; ++c) {
            int idx = r * cols + c;
            if (idx < (int)m.size()) {
                std::cout << std::fixed << std::setprecision(4) << std::setw(10) << m[id
x] << " ";
            }
        }
        std::cout << std::endl;
    }
}
```

6.2 连接管理接口

头文件：include/tl_interface.h

相关示例代码：examples\cpp\ex_connect.cpp

6.2.1 connect_robot - 创建机械臂双端口连接控制句柄

方法： SOCKETFD connect_robot(const std::string& ip, const std::string& port);

功能描述：建立与机械臂控制器的网络通信通道。需要分别连接指令端口（6001）和实时端口（7000）以获得两个独立的句柄。

参数：

- ip：机械臂控制器 IP 地址，例如 "192.168.1.13"。
- port：端口号字符串，指令通道 "6001"，实时跟踪通道 "7000"。

返回值：

- '> 0：成功，返回套接字句柄。
- <=0：失败，返回负错误码。

使用示例：

```
// 连接控制器
std::cout << "[INFO] 连接控制器..." << std::endl;
auto socket_fd = tl::connect_robot(ip, port);
if (socket_fd < 0) {
    std::cerr << "[ERROR] 连接失败，程序退出!"<< std::endl;
    return -1;
}
std::cout << "[INFO] 连接成功 (socket=" << socket_fd << ")" << std::endl;

std::cout << "[INFO] 延时2s" << std::endl;
std::this_thread::sleep_for(std::chrono::milliseconds(2000));
```

6.2.2 disconnect_robot - 关闭机械臂通信句柄

方法： Result disconnect_robot(SOCKETFD socketFd);

功能描述： 安全关闭指定的网络连接，释放资源。

参数：

- socketFd：要关闭的句柄。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
// 断开控制器连接
std::cout << "[INFO] 断开控制器连接..." << std::endl;
auto ret = tl::disconnect_robot(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 断开连接失败，程序退出!"<< std::endl;
    return -1;
}
std::cout << "[INFO] 断开连接成功" << std::endl;

return 0;
```

6.2.3 get_connection_status - 获取控制器连接状态

方法：Result get_connection_status(SOCKETFD socketFd);

功能描述：获得控制器连接状态。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
// 获取控制器连接状态
std::cout << "[INFO] 获取控制器连接状态..." << std::endl;
auto conn_ret = tl::get_connection_status(socket_fd);
if (conn_ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取连接状态失败，错误码: " << conn_ret << std::endl;
} else {
    std::cout << "[INFO] 连接状态正常 (status=" << conn_ret << ")" << std::endl;
}
```

6.2.4 get_controller_network_config - 获取控制器网络配置

方法：Result get_controller_network_config(SOCKETFD socketFd, std::string& name, std::string& address,std::string& gateway, std::string& dns);

功能描述：获取控制器当前的网络配置信息。

参数：

- socketFd：已连接的句柄。
- name：网络接口名称。
- address：IP地址
- gateway：网关地址
- dns：DNS地址

返回值：

- 0：成功。

- 非 0：失败。

使用示例：

```
    // 获取控制器网络配置
    std::cout << "[INFO] 获取控制器网络配置..." << std::endl;
    std::string net_name, net_address, net_gateway, net_dns;
    auto net_ret = tl::get_controller_network_config(socket_fd, net_name, net_address, n
et_gateway, net_dns);
    if (net_ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取控制器网络配置失败，错误码: " << net_ret << std::endl;
    } else {
        std::cout << "[INFO] 网络接口: " << net_name
                  << ", IP: " << net_address
                  << ", 网关: " << net_gateway
                  << ", DNS: " << net_dns << std::endl;
    }
```

6.2.5 set_controller_network_config - 设置控制器网络配置

方法：Result set_controller_network_config(SOCKETFD socketFd, const std::string& name, const std::string& ip, const std::string& gateway, const std::string& dns);

功能描述：设置控制器的网络配置信息。

参数：

- socketFd：已连接的句柄。
- name：要修改的网络接口名称
- ip：新的 IP 地址
- gateway：网关地址，传空字符串表示不配置网关
- dns：DNS 服务器地址，传空字符串表示不配置 DNS

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
    // 设置控制器网络配置（危险操作，默认注释）
    // ⚠️危险：错误配置会导致控制器网络断连，且可能无法重新连接
    // 取消下面代码的注释前，请确认 name/IP/网关/DNS 参数正确，
    // 并将 IP 改为你实际想要配置的地址（网关/DNS 传空字符串表示不配置）
```

```
    //
    // std::cout << "[INFO] 设置控制器网络配置..." << std::endl;
    // auto set_ret = tl::set_controller_network_config(socket_fd, "eth0",
"192.168.1.14", "", "");
    // if (set_ret != tl::Result::SUCCESS) {
    //     std::cerr << "[ERROR] 设置控制器网络配置失败，错误码: " << set_ret << std::endl;
    // } else {
    //     std::cout << "[INFO] 设置控制器网络配置成功，新IP: 192.168.1.14" << std::endl;
    // }
```

6.3 日志管理接口

头文件：include/tl_interface.h

相关示例代码：examples\cpp\ex_log_download.cpp

6.3.1 log_download_by_quantity - 保存日志到文件

方法：Result log_download_by_quantity(SOCKETFD socketFd, int counts,const std::string& directoryPath);

功能描述：从控制器下载指定数量的日志条目并保存到本地文件。

参数：

- socketFd：已连接的句柄。
- counts：文件数量
- directoryPath：目录的完整路径，字符串格式

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
    // 下载日志

    std::cout << "[INFO] 开始下载日志..." << std::endl;
    auto ret = log_download_by_quantity(socket_fd, count, log_path);
    if (ret != Result::SUCCESS) {
        std::cerr << "[ERROR] 日志下载失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 日志下载完成" << std::endl;
```

6.3.2 set_log_level - 设置日志级别

方法：void set_log_level(LogLevel level);

功能描述：设置 SDK 本地库的全局日志输出级别。该接口为本地配置，**无需连接控制器**（不依赖 socket_fd）；设置后仅输出级别 ≥ 当前级别的日志，默认级别为 `INFO`（一般）。

参数：

- level：日志级别，取值见下表 LogLevel 枚举。

LogLevel 枚举说明：

枚举值	数值	说明
DEBUG	0	调试：输出全部日志（最详细），用于问题排查
INFO	1	一般：输出常规信息（系统默认级别）
WARNING	2	警告：仅输出警告及以上级别的日志
ERROR	3	错误：仅输出错误级别的日志（最精简）

返回值：

- 无（void，不返回任何值）。

使用示例：

```
// 仅输出警告和错误信息（减少日志量）
tl::set_log_level(tl::WARNING);

// 输出全部日志（最详细，便于调试）
tl::set_log_level(tl::DEBUG);
```

6.4 基础状态查询接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_info_query.cpp

6.4.1 get_current_position - 获取机械臂当前坐标信息

方法：Result get_current_position(SOCKETFD socketFd, Coord coord, std::vector< double >& pos);

功能描述：获取机械臂当前在指定坐标系下的位置数据。

参数：

- socketFD：已连接的句柄。

- coord 坐标系 Coord::JOINT / Coord::BASE / Coord::TOOL / Coord::USER（分别对应关节，直角，工具，用户）。
- pos：输出参数，存储位置数据（长度自动适配）；需要说明，当坐标系为直角坐标系时，返回[X,Y,Z,RX,RY,RZ]（mm, rad），姿态角统一为弧度，与运动指令的弧度契约一致。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
// ----获取机械臂当前坐标----
std::cout << "[INFO] **********获取机械臂当前坐标**********" << std::endl;
tl::Coord coord = tl::Coord::JOINT;
std::vector<double> current_position;
auto ret = tl::get_current_position(socket_fd, coord, current_position);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取机械臂当前坐标失败，程序退出!" << std::endl;
    return -1;
}
std::cout << "[INFO] 机械臂当前坐标: ";
for (const auto& pos : current_position) {
    std::cout << pos << " ";
}
std::cout << std::endl;
std::cout << std::endl;
```

6.4.2 get_robot_running_state - 获取机器人运行状态

方法：Result get_robot_running_state(SOCKETFD socketFd, RunState& status);

功能描述：获取当前作业文件的运行状态，执行前需要延时0.1秒。

参数：

- socketFd：已连接的句柄。
- status 输出：RunState::STOP（停止）/ RunState::PAUSE（暂停）/ RunState::RUNNING（运行）

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----获取机械臂运行状态----
    std::cout << "[INFO] **********获取机械臂运行状态**********" << std::endl;
    tl::RunState status;
    ret = tl::get_robot_running_state(socket_fd, status);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取机械臂运行状态失败，程序退出!" << std::endl;
        return -1;
    }
    switch (status)
    {
        case tl::RunState::STOP:
            std::cout << "[INFO] 机械臂处于停止状态 (status = " <<
static_cast<int>(status) << ")" << std::endl;
            break;
        case tl::RunState::PAUSE:
            std::cout << "[INFO] 机械臂处于暂停状态 (status = " <<
static_cast<int>(status) << ")" << std::endl;
            break;
        case tl::RunState::RUNNING:
            std::cout << "[INFO] 机械臂处于运行状态 (status = " <<
static_cast<int>(status) << ")" << std::endl;
            break;
        default:
            break;
    }
    std::cout << std::endl;
```

6.4.3 get_servo_state - 获取伺服状态

方法：Result get_servo_state(SOCKETFD socketFd, ServoState& status);

功能描述：获取机械臂当前伺服状态信息。

参数：

- socketFd：已连接的句柄。
- status 输出：ServoState::STOP / READY / ALARM / RUNNINGstatus (分别对应停止状态, 就绪状态, 报警状态, 运行状态)

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
  // 2. 检查伺服状态
  tl::ServoState state;
```

```cpp
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state)
            << " (STOP=停止 READY=就绪 ALARM=报警 RUNNING=运行)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
  }

  // 3. 上电流程
  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }
```

6.4.4 get_current_mode - 获取机器人当前模式

方法：Result get_current_mode(SOCKETFD socketFd, RobotMode& mode);

功能描述：获取机械臂当前模式。

参数：

- socketFd：已连接的句柄。
- mode 输出：RobotMode::TEACH（示教）/ RobotMode::REMOTE（远程）/ RobotMode::RUN（运行）

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
        // 先获取当前模式
        tl::RobotMode current_mode;
        tl::Result ret = tl::get_current_mode(sock, current_mode);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取当前模式失败: " << ret << std::endl;
            tl::disconnect_robot(sock);
            return -1;
        }
        std::cout << "  [信息] 当前模式: " << static_cast<int>(current_mode) << " (RobotMo
de::TEACH/REMOTE/RUN)" << std::endl;
```

6.4.5 get_robot_dh_param - 获取机械臂DH参数

方法：Result get_robot_dh_param(SOCKETFD socketFd, RobotDHParam& param);

功能描述：获取机械臂当前的 DH 几何参数（运动学标定数据）。

参数：

- socketFd：已连接的句柄。
- param：输出参数，RobotDHParam 结构体，主要成员：
  - alpha[6]：1-6 轴 DH 参数 alpha，单位 deg
  - a[6]：1-6 轴 DH 参数 a，单位 mm
  - theta[6]：1-6 轴 DH 参数 theta，单位 deg
  - d[6]：1-6 轴 DH 参数 d，单位 mm
  - eulerAngle：欧拉角模式
  - mountingAngle：机器人安装角度，单位 deg

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取机械臂DH参数
    std::cout << "[INFO] **********获取机械臂DH参数**********" << std::endl;
    tl::RobotDHParam dh_params;
    auto ret = tl::get_robot_dh_param(socket_fd, dh_params);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取机械臂DH参数失败，程序退出!" << std::endl;
        return -1;
```

```
    }
    std::cout << "[INFO] 机械臂DH参数: ";
    std::cout << "alpha=[";
    for (int i = 0; i < 6; ++i) std::cout << (i ? "," : "") << dh_params.alpha[i];
    std::cout << "], a=[";
    for (int i = 0; i < 6; ++i) std::cout << (i ? "," : "") << dh_params.a[i];
    std::cout << "], theta=[";
    for (int i = 0; i < 6; ++i) std::cout << (i ? "," : "") << dh_params.theta[i];
    std::cout << "], d=[";
    for (int i = 0; i < 6; ++i) std::cout << (i ? "," : "") << dh_params.d[i];
    std::cout << "], eulerAngle=" << dh_params.eulerAngle
              << ", mountingAngle=" << dh_params.mountingAngle << std::endl;
    std::cout << std::endl;
```

6.4.6 get_library_version - 读取机械臂软件信息

方法：std::string get_library_version();

功能描述：获取 SDK 库版本信息。

参数：

- 无

返回值：

- 非空字符串：成功，返回版本信息字符串，格式 "(Base:)"，如 "V2.5.1-0-gabc1234-20260916(Base:2.0.4-20260914-6cf4a3b)"。
- 空字符串：失败（库未正确加载）。

使用示例：

```
    // 获取 SDK 版本
    std::string version = tl::get_library_version();
    std::cout << "[INFO] SDK 版本: " << version << std::endl;
```

6.4.7 get_robot_joint_param - 获取指定关节配置参数

方法：Result get_robot_joint_param(SOCKETFD socketFd, int id, RobotJointParam& param);

功能描述：获取指定关节的运动与硬件参数（减速比、编码器分辨率、限位、转速等）。

参数：

- socketFd：已连接的句柄。
- id：关节编号，取值范围 [1,6]（6 轴，对应 J1~J6）；7 轴为 [1,7]（对应 J1~J7）。

- param：输出参数，RobotJointParam 结构体，主要成员：
    - reduceRatio：减速比
    - encoderResolution：编码器分辨率数值
    - maxAcc：最大加速度
    - maxDec：最大减速度
    - maxPos：正向软限位
    - minPos：负向软限位
    - maxRotSpeed：最大转速
    - ratedRotSpeed：电机额定转速
    - motorDirection：电机方向
    - axisDirection：关节轴正方向

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取指定关节参数
    std::cout << "[INFO] **********获取指定关节参数**********" << std::endl;
    int joint_id = 1;  // J1 轴
    tl::RobotJointParam joint_param;
    ret = tl::get_robot_joint_param(socket_fd, joint_id, joint_param);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取关节" << joint_id << "参数失败，错误码: " << ret << std::e
ndl;
        return -1;
    }
    std::cout << "[INFO] 关节" << joint_id << "参数:" << std::endl;
    std::cout << "  减速比=" << joint_param.reduceRatio
              << ", 额定转速=" << joint_param.ratedRotSpeed
              << ", 最大加速度=" << joint_param.maxAcc
              << ", 电机方向=" << joint_param.motorDirection
              << ", 正向软限位=" << joint_param.maxPos
              << ", 负向软限位=" << joint_param.minPos << std::endl;
    std::cout << std::endl;
```

6.4.8 query_joint_software_version - 查询指定关节（轴）的软件版本号

方法：Result query_joint_software_version(SOCKETFD socketFd, int axisNum, std::string& version_string);

功能描述：查询指定关节（轴）的软件版本号（SDO 0x100A）。

参数：

- socketFd：已连接的句柄。
- axisNum：轴编号。
- version_string：输出参数，用于接收查询到的软件版本号字符串。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询关节软件版本号
    std::cout << "[INFO] **********查询关节软件版本号**********" << std::endl;
    {
        int query_axis = 1;  // 查询 J1 轴
        std::string joint_version;
        auto ret = tl::query_joint_software_version(socket_fd, query_axis,
joint_version);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 查询关节" << query_axis << "软件版本号失败，错误码: " << re
t << std::endl;
        } else {
            std::cout << "[INFO] 关节" << query_axis << "软件版本号: " << joint_version
<< std::endl;
        }
    }
    std::cout << std::endl;
```

6.4.9 get_nexmotion_lib_version - 获取底层 nexmotion 库版本信息

方法：Result get_nexmotion_lib_version(SOCKETFD socketFd, std::string& version);

功能描述：获取底层算法库（nexmotion）版本号。

参数：

- socketFd：已连接的句柄。

- version：输出参数，算法库版本号字符串。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// ----获取底层 nexmotion 库版本----
std::cout << "[INFO] **********获取底层 nexmotion 库版本**********" << std::endl;
{
    std::string nexmotion_version;
    auto ret = tl::get_nexmotion_lib_version(socket_fd, nexmotion_version);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取 nexmotion 库版本失败，错误码: " << ret << std::endl;
    } else {
        std::cout << "[INFO] nexmotion 库版本: " << nexmotion_version << std::endl;
    }
}
std::cout << std::endl;
```

6.4.10 get_robot_state - 获取机器人整体状态

方法：Result get_robot_state(SOCKETFD socketFd, RobotState param);

功能描述：通过 7000 端口查询机器人整体状态。需要连接 7000 端口。

参数：

- socketFd：已连接的句柄（7000 端口）。
- param：RobotState 结构体（输入配置），主要成员：
    - channel：查询的通道，最多 9 个通道
    - stop：为 true 时停止持续发送
    - mode：0-查询只回复一次，1-查询持续回复
    - interval：仅 mode=1 时有效，回复时间范围 [10,60000] ms
    - ioState：查询 IO
    - position：0-关节坐标，1-基坐标
    - dataildmotionpos：机械臂的运动点位
    - programRunStatus：运行状态

- servoStatus：伺服状态
- operationMode：操作模式
- globalSpeed：全局速度
- syncPosition：外部轴坐标

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 回调函数：收到控制器状态时被调用（在独立线程中触发，不能做耗时操作）
    void on_robot_state(const char* message)
    {
        if (message != nullptr) {
            std::cout << "[回调] 收到状态: " << message << std::endl;
        }
    }

    // ----查询机器人状态与注册状态回调（7000 端口，需先连接）----
    std::cout << "[INFO] **********查询机器人状态（7000 端口）**********" << std::endl;
    {
        std::string port_7000 = "7000";
        auto sock_7000 = tl::connect_robot(ip, port_7000);
        if (sock_7000 < 0) {
            std::cerr << "[ERROR] 连接 7000 端口失败，错误码: " << sock_7000 << std::endl;
        } else {
            std::cout << "[INFO] 7000 端口连接成功 (socket=" << sock_7000 << ")" << std::e
ndl;

            // 注册状态回调（回调在独立线程触发，不能做耗时操作）
            ret = tl::robot_state_callback(sock_7000, on_robot_state_message);
            if (ret != tl::Result::SUCCESS) {
                std::cerr << "[ERROR] 注册状态回调失败，错误码: " << ret << std::endl;
            } else {
                std::cout << "[INFO] 状态回调注册成功" << std::endl;
            }

            // 查询机器人状态（单次回复）
            tl::RobotState robot_state;
            robot_state.channel = 1;
            robot_state.stop = false;
            robot_state.mode = 0;              // 0-查询只回复一次 1-持续回复
            robot_state.interval = 1000;
            robot_state.position = 0;          // 0-关节坐标 1-直角坐标
            robot_state.servoStatus = true;
            robot_state.programRunStatus = true;
```

```
            ret = tl::get_robot_state(sock_7000, robot_state);
            if (ret != tl::Result::SUCCESS) {
                std::cerr << "[ERROR] 查询机器人状态失败，错误码: " << ret << std::endl;
            } else {
                std::cout << "[INFO] 状态查询请求已发送，等待回调..." << std::endl;
                std::this_thread::sleep_for(std::chrono::seconds(1));
            }

            // 停止持续推送并断开 7000
            tl::RobotState stop_param;
            stop_param.channel = 1;
            stop_param.stop = true;
            tl::get_robot_state(sock_7000, stop_param);
            tl::disconnect_robot(sock_7000);
            std::cout << "[INFO] 7000 端口已断开" << std::endl;
        }
    }
    std::cout << std::endl;
```

6.4.11 get_robot_type - 获取机器人类型

方法：Result get_robot_type(SOCKETFD socketFd, RobotType& type);

功能描述：获取机器人类型（枚举重载）。返回 RobotType 枚举，标识机器人机械结构类型。

参数：

- socketFd：已连接的句柄。
- type：输出参数，RobotType 枚举，完整取值如下表：

RobotType 枚举说明：

枚举值	数值	说明
SIX_AXIS_SERIAL	1	六轴串联多关节
FOUR_AXIS_SCARA	2	四轴 SCARA
FOUR_AXIS_PALLETIZING	3	四轴码垛
FOUR_AXIS_SERIAL	4	四轴串联多关节
SINGLE_AXIS	5	单轴
FIVE_AXIS_SERIAL	6	五轴串联多关节
SIX_AXIS_COLLABORATIVE	7	六轴协作
TWO_AXIS_SCARA	8	二轴 SCARA
THREE_AXIS_SCARA	9	三轴 SCARA
THREE_AXIS_CARTESIAN	10	三轴直角

枚举值	数值	说明
THREE_AXIS_SPECIAL	11	三轴异形
SEVEN_AXIS_SERIAL	12	七轴串联多关节
SCARA_SPECIAL	13	SCARA 异形一
FOUR_AXIS_PALLETIZING_LEAD	14	四轴码垛丝杆

> 上表与 `include/tl_types.h` 的 `tl::RobotType` 一致，数值即 `get_robot_type` 的 int 重载返回值。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询机器人类型----
    std::cout << "[INFO] **********查询机器人类型**********" << std::endl;
    {
        tl::RobotType robot_type;
        ret = tl::get_robot_type(socket_fd, robot_type);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取机器人类型失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 机器人类型编号: " << static_cast<int>(robot_type) << std:
:endl;
        }
    }
    std::cout << std::endl;
```

6.4.12 get_current_line_speed_and_joint_speed - 获取当前线速度与关节速度

方法：Result get_current_line_speed_and_joint_speed(SOCKETFD socketFd, double& line_speed, std::vector< double >& joint_speed, std::vector< double >& joint_speed_sync);

功能描述：获取当前末端线速度和关节速度。

参数：

- socketFd：已连接的句柄。
- line_speed：输出参数，末端线速度，单位 [mm/s]。
- joint_speed：输出参数，关节速度，长度 7，单位 [度/s]。

- joint_speed_sync：输出参数，外部轴关节速度，长度 5，单位 [度/s]。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询当前末端线速度和关节速度----
    std::cout << "[INFO] **********查询当前末端线速度和关节速度**********" << std::endl;
    {
        double line_speed = 0;
        std::vector<double> joint_speed;
        std::vector<double> joint_speed_sync;
        ret = tl::get_current_line_speed_and_joint_speed(socket_fd, line_speed, joint_sp
eed, joint_speed_sync);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取当前末端线速度和关节速度失败，错误码: " << ret << std::en
dl;
        } else {
            std::cout << "[INFO] 末端线速度: " << line_speed << " mm/s" << std::endl;
            std::cout << "[INFO] 关节速度: ";
            for (const auto& val : joint_speed) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
            std::cout << "[INFO] 外部轴关节速度: ";
            for (const auto& val : joint_speed_sync) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.4.13 get_pos_reachable - 查询点位可达性

方法：Result get_pos_reachable(SOCKETFD socketFd, std::vector< double > pos, std::string movetype, bool& result);

功能描述：检测目标点位是否可达。

参数：

- socketFd：已连接的句柄。

- pos：目标点位坐标数据，vector 长度 14：
  - [0] 坐标系 0:关节 1:直角 2:工具 3:用户
  - [1] 单位 0:角度 1:弧度
  - [2] 形态
  - [3] 工具手坐标序号
  - [4] 用户坐标序号
  - [5][6] 备用
  - [7-13] 点位信息
- movetype：移动方式，"MOVJ" 或 "MOVL"。
- result：输出参数，点位是否可达（true=可达）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询点位是否可达（示例点位，可自行修改）----
    std::cout << "[INFO] **********查询点位是否可达**********" << std::endl;
    {
        // 目标点位 14 位: [0]坐标系 0关节 1直角 2工具 3用户 [1]0角度制 1弧度制 [2]形态 [3]工具号
[4]用户坐标号 [5][6]备用 [7-13]点位值
        std::vector<double> pos(14, 0);
        pos[0] = 0;  // 关节坐标系
        pos[1] = 0;  // 角度制
        pos[7] = 0.0;   // J1
        pos[8] = 0.0;   // J2
        pos[9] = 0.0;   // J3
        pos[10] = 0.0;  // J4
        pos[11] = 0.0;  // J5
        pos[12] = 0.0;  // J6
        std::string movetype = "MOVJ";
        bool reachable = false;
        ret = tl::get_pos_reachable(socket_fd, pos, movetype, reachable);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 查询点位是否可达失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 点位 " << movetype << " 是否可达: " << (reachable ? "可
达" : "不可达") << std::endl;
        }
```

```
}
std::cout << std::endl;
```

6.4.14 robot_state_callback - 注册机器人状态回调

方法：Result robot_state_callback(SOCKETFD socketFd, void (function)(const char state));

功能描述：注册 7000 端口状态返回的回调函数。需要连接 7000 端口。

参数：

- socketFd：已连接的句柄（7000 端口）。
- function：回调函数指针，签名 void (const char* state)，收到控制器状态时在独立线程中触发。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 回调函数：收到控制器状态时被调用
void on_robot_state(const char* message)
{
    if (message != nullptr) {
        std::cout << "[回调] 收到状态: " << message << std::endl;
    }
}

// 注册回调（7000 端口）
auto ret = tl::robot_state_callback(sock_7000, on_robot_state);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 注册回调失败，错误码: " << ret << std::endl;
} else {
    std::cout << "[INFO] 回调注册成功" << std::endl;
}
```

6.4.15 get_speed - 获取速度

方法：Result get_speed(SOCKETFD socketFd, int& speed);

功能描述：获取当前全局速度倍率。

参数：

- socketFd：已连接的句柄。

- speed：输出参数，当前速度百分比。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 查询当前速度
int speed = -1;
auto ret = tl::get_speed(sock, speed);
if (ret == tl::Result::SUCCESS) {
    std::cout << "  [信息] 当前全局速度: " << speed << "%" << std::endl;
} else {
    std::cerr << "[ERROR] 获取速度失败，错误码: " << ret << std::endl;
}
```

6.4.16 get_cartesian_params - 获取笛卡尔参数

方法：Result get_cartesian_params(SOCKETFD socketFd, CartesianParam& param);

功能描述：获取笛卡尔空间运动参数。

参数：

- socketFd：已连接的句柄。
- param：输出参数，CartesianParam 结构体，主要成员：
    - maxSpeed{1000}：最大线速度 (mm/s)
    - maxAcc{3}：最大线加速度 (mm/s^2)
    - maxDec{-3}：最大线减速度 (mm/s^2)
    - maxAttitudeVel{500}：最大姿态角速度 (deg/s)
    - speedLimitMode{0}：速度限制方式 0:位姿 1:位置
    - posResolution{0.01}：绝对位置分辨率，范围 [0.0001, 0.1]
    - maxJerk{10000}：最大线加加速度 (mm/s^3)
    - interpType{0}：插补方式 0:S型 1:梯形
    - minAccTime{100}：最小加速时间 (ms)
    - minDecTime{100}：最小减速时间 (ms)

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询笛卡尔空间参数----
    std::cout << "[INFO] **********查询笛卡尔空间参数**********" << std::endl;
    {
        tl::CartesianParam cartesian_param;
        ret = tl::get_cartesian_params(socket_fd, cartesian_param);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取笛卡尔空间参数失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 笛卡尔空间参数:" << std::endl;
            std::cout << "  最大线速度: " << cartesian_param.maxSpeed << " mm/s" << std::e
ndl;
            std::cout << "  最大线加速度: " << cartesian_param.maxAcc << " mm/s^2" << std:
:endl;
            std::cout << "  最大线减速度: " << cartesian_param.maxDec << " mm/s^2" << std:
:endl;
            std::cout << "  最大姿态角速度: " << cartesian_param.maxAttitudeVel << " deg/
s" << std::endl;
            std::cout << "  速度限制方式: " << cartesian_param.speedLimitMode << " (0-位姿
1-位置)" << std::endl;
            std::cout << "  绝对位置分辨率: " << cartesian_param.posResolution << std::end
l;
            std::cout << "  最大线加加速度: " << cartesian_param.maxJerk << " mm/s^3" << s
td::endl;
            std::cout << "  插补方式: " << cartesian_param.interpType << " (0-S型 1-梯
形)" << std::endl;
            std::cout << "  最小加速时间: " << cartesian_param.minAccTime << " ms" << std:
:endl;
            std::cout << "  最小减速时间: " << cartesian_param.minDecTime << " ms" << std:
:endl;
        }
    }
    std::cout << std::endl;
```

6.5 伺服状态与系统参数设置接口

头文件：include/tl_interface.h

6.5.1 clear_error - 清除系统错误

方法：Result clear_error(SOCKETFD socketFd);

功能描述：复位控制器的当前报警状态（清除红色错误）。在机械臂发生碰撞、超限位等错误后，排除物理故障后调用此接口清除报警。清错后需要手动进行下电操作，释放控制器的占用状态才可以继续上电（清错后不能直接上电，先下电再上电）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败（如错误仍未清除需手动排除故障）。

使用示例：

```
print_separator("清错与伺服复位");
ret = tl::clear_error(sock_6001);
print_result("clear_error", ret);
std::this_thread::sleep_for(std::chrono::milliseconds(300));
```

6.5.2 set_servo_state - 设置伺服就绪状态

方法： Result set_servo_state(SOCKETFD socketFd, ServoState state);

功能描述：控制电机抱闸与使能。state=READY=使能（解锁抱闸），state=STOP=去使能（抱闸锁死）。

参数：

- socketFd：已连接的句柄。
- state ServoState::STOP（停止）/ ServoState::READY（就绪）。

返回值：

- 0：成功。
- -4：操作不允许（如未清除报警）。
- 其他非0：失败。

使用示例：

```
std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
  std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
  return;
```

```
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
```

6.5.3 set_servo_poweron - 设置伺服上电状态

方法：Result set_servo_poweron(SOCKETFD socketFd);

功能描述：闭合强电接触器，为电机供电。需先调用 set_servo_state(1)。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- -4：操作不允许（如未清除报警）。
- 其他非0：失败。

使用示例：

```
void enable_servo(const int socket_fd)
{
  std::cout << "--- 检查伺服状态 ---" << std::endl;

  // 1. 先清除控制器错误和报警，再下电复位确保状态同步
  tl::clear_error(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));
  tl::set_servo_poweroff(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));

  // 2. 检查伺服状态
  tl::ServoState state;
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state)
            << " (STOP=停止 READY=就绪 ALARM=报警 RUNNING=运行)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
```

```
  }

  // 3. 上电流程
  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }

  std::cout << "  [信息] 上电使能..." << std::endl;
  ret = tl::set_servo_poweron(socket_fd);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
    return;
  }
  std::this_thread::sleep_for(std::chrono::seconds(2));

  // 4. 确认结果
  tl::get_servo_state(socket_fd, state);
  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂使能上电成功（servo_state = " << static_cast<int>(state) <<
") " << std::endl;
  } else {
    std::cerr << "[ERROR] 机械臂使能上电失败（servo_state = " << static_cast<int>(state)
<< ") " << std::endl;
  }
}
```

6.5.4 set_servo_poweroff - 设置伺服下电状态

方法：Result set_servo_poweroff(SOCKETFD socketFd);

功能描述：切断强电，电机抱闸锁死。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- -4：操作不允许（如未清除报警）。
- 其他非0：失败。

使用示例：

```
void disable_servo(const int socket_fd)
{
  tl::ServoState state;
  tl::get_servo_state(socket_fd, state);
  switch (state)
  {
    case tl::ServoState::READY:
        std::cout << "[INFO] 机械臂已使能下电" << std::endl;
        break;
    case tl::ServoState::RUNNING:
        tl::set_servo_poweroff(socket_fd);
        tl::get_servo_state(socket_fd, state);
        std::cout << "[INFO] 机械臂使能下电成功（servo_state = " <<
static_cast<int>(state) << "）" << std::endl;
        return;
    default:
        break;
  }
  std::cout << "[INFO] 机械臂使能下电失败（servo_state = " << static_cast<int>(state) << "）
" << std::endl;
}
```

6.5.5 set_speed - 设置速度

方法： Result set_speed(SOCKETFD socketFd, int speed);

功能描述：设置全局自动运行速度百分比（1~100）。

参数:

- socketFd：已连接的句柄。
- speed：速度百分比。

返回值:

- 0：成功。
- 非0：失败。

使用示例:

```
    // 设置全局速度
    ret = tl::set_speed(socket_fd, 30);
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
```

6.5.6 restore_default_param_DH - 恢复机械臂默认DH参数

方法：Result restore_default_param_DH(SOCKETFD socketFd, int robotNum);

功能描述：恢复机械臂默认DH参数。

参数：

- socketFd：已连接的句柄。
- robotNum：机器人编号(1-4)

返回值：

- Result：
- 0：成功。
- 非0：失败
- DHparam DH：参数。

使用示例：

```
// // ⚠️以下功能不建议测试，请在厂家指导下使用
// // ============================================================
// // 6. 恢复机械臂默认 DH 参数
// // ============================================================
// print_separator("恢复机械臂默认 DH 参数");

// ret = restore_default_param_DH(sock, 1);
// print_result("restore_default_param_DH (robot=1)", ret);
// std::cout << std::endl;
```

6.5.7 set_cartesian_params - 设置机器人笛卡尔参数

方法：Result set_cartesian_params(SOCKETFD socketFd, const CartesianParam& param);

功能描述：配置笛卡尔空间运动动力学约束参数，限定末端执行器最大直线速度、直线加速度、旋转角速度与角加速度，实现轨迹平滑度与运行安全管控。

参数：

- socketFd：已连接的句柄。
- param 包含笛卡尔空间运动参数的结构体

返回值：

- Result：
- 0：成功。

- 非0：失败

使用示例：

```
    // // ⚠️以下功能不建议测试，请在厂家指导下使用
    // // ============================================================
    // // 8. 设置笛卡尔参数
    // // ============================================================
    // print_separator("设置笛卡尔参数");

    // {
    //     CartesianParam cart_param;
    //     cart_param.maxSpeed = 500.0;         // 最大线速度 (mm/s)
    //     cart_param.maxAcc = 5.0;             // 最大线加速度 (mm/s^2)
    //     cart_param.maxDec = -5.0;            // 最大线减速度 (mm/s^2)
    //     cart_param.maxAttitudeVel = 180.0;   // 最大姿态角速度 (deg/s)
    //     cart_param.speedLimitMode = 1;       // 速度限制方式: 0=位姿, 1=位置

    //     std::cout << std::fixed << std::setprecision(2);
    //     ret = set_cartesian_params(sock, cart_param);
    //     if (ret == Result::SUCCESS) {
    //         std::cout << "[成功] 设置笛卡尔参数" << std::endl;
    //         std::cout << "  最大线速度: " << cart_param.maxSpeed << " mm/s" <<
std::endl;
    //         std::cout << "  最大线加速度: " << cart_param.maxAcc << " mm/s^2" <<
std::endl;
    //         std::cout << "  最大线减速度: " << cart_param.maxDec << " mm/s^2" <<
std::endl;
    //         std::cout << "  最大姿态角速度: " << cart_param.maxAttitudeVel << " deg/s"
<< std::endl;
    //         std::cout << "  速度限制方式: " << (cart_param.speedLimitMode == 0 ? "位
姿" : "位置") << std::endl;
    //     } else {
    //         std::cerr << "[ERROR] 设置笛卡尔参数失败，错误码: " << ret << std::endl;
    //     }
    // }
    // std::cout << std::endl;
```

6.5.8 restore_network_factory_settings - 恢复网络出厂设置

方法：Result restore_network_factory_settings(SOCKETFD socketFd);

功能描述：执行控制器网络参数恢复出厂操作，重置通讯 IP 地址为默认固定地址，属于高危配置操作。

参数：

- socketFd：已连接的句柄。

返回值：

- Result：
- 0：成功。
- 非0：失败

使用示例：

```
// ============================================================
// 10. 恢复网络出厂设置（危险操作，默认注释）
// ============================================================
// ⚠️ 危险：恢复网络出厂设置会重置控制器 IP 为 192.168.1.13
//    取消下面代码的注释前，请确认当前网络环境允许 IP 变更
//
// print_separator("恢复网络出厂设置");
// ret = restore_network_factory_settings(sock);
// print_result("restore_network_factory_settings", ret);
// std::cout << std::endl;
```

6.5.9 set_robot_joint_param - 设置当前机器人关节参数

方法：Result set_robot_joint_param(SOCKETFD socketFd, int id, RobotJointParam& param);

功能描述：对指定单关节进行运动与硬限位参数配置，设定各关节最大运行速度、加速度及正负向机械位置软限位，约束关节运动边界保障本体运行安全。

参数：

- socketFd：已连接的句柄。
- id：关节编号，取值范围 [1,6]（6轴，对应 J1~J6）；7轴为 [1,7]（对应 J1~J7）。
- param：结构体参数。

返回值：

- Result：
- 0：成功。
- 非0：失败

使用示例：

```
// // ============================================================
// // 7. 设置关节参数（遍历关节 1~6）
// // ============================================================
```

```
    // print_separator("设置关节参数");

    // {
    //     RobotJointParam joint_param;
    //     joint_param.reducRatio = 1;           // 减速比
    //     joint_param.encoderResolution = 2;    // 编码器位数
    //     joint_param.posSWLimit = 3;           // 轴正限位
    //     joint_param.negSWLimit = -4;          // 轴反限位
    //     joint_param.ratedRotSpeed = 5;        // 电机额定正转速
    //     joint_param.ratedDeRotSpeed = -6;     // 电机额定反转速
    //     joint_param.maxRotSpeed = 2;          // 电机最大正转速
    //     joint_param.maxDeRotSpeed = -2;       // 电机最大反转速
    //     joint_param.ratedVel = 300;           // 额定正速度 (度/秒)
    //     joint_param.deRatedVel = -300;        // 额定反速度 (度/秒)
    //     joint_param.maxAcc = 2;               // 最大加速度 (度/秒^2)
    //     joint_param.maxDecel = -2;            // 最大减速度 (度/秒^2)
    //     joint_param.direction = -1;           // 模型方向: 1=正向, -1=反向

    //     // 遍历关节 1~6
    //     for (int joint_id = 1; joint_id <= 6; ++joint_id) {
    //         std::cout << "--- 关节 " << joint_id << " ---" << std::endl;
    //         ret = set_robot_joint_param(sock, joint_id, joint_param);
    //         if (ret == Result::SUCCESS) {
    //             std::cout << "  [成功] 设置关节 " << joint_id << " 参数" << std::endl;
    //         } else {
    //             std::cout << "  [失败] 设置关节 " << joint_id << " 参数，错误码: " << ret
<< std::endl;
    //         }
    //     }

    //     // 打印参数（仅供示意，TODO: 各关节应根据实际需求独立设置参数值）
    //     std::cout << "\n========== 机器人关节参数设置 (仅供参考) ==========" <<
std::endl;
    //     std::cout << "  减速比: " << joint_param.reducRatio << std::endl;
    //     std::cout << "  编码器位数: " << joint_param.encoderResolution << std::endl;
    //     std::cout << "  正限位: " << joint_param.posSWLimit << " 度" << std::endl;
    //     std::cout << "  反限位: " << joint_param.negSWLimit << " 度" << std::endl;
    //     std::cout << "  额定正转速: " << joint_param.ratedRotSpeed << " RPM" <<
std::endl;
    //     std::cout << "  额定反转速: " << joint_param.ratedDeRotSpeed << " RPM" <<
std::endl;
    //     std::cout << "  最大正转速: " << joint_param.maxRotSpeed << " RPM" <<
std::endl;
    //     std::cout << "  最大反转速: " << joint_param.maxDeRotSpeed << " RPM" <<
std::endl;
    //     std::cout << "  额定正速度: " << joint_param.ratedVel << " 度/秒" << std::endl;
    //     std::cout << "  额定反速度: " << joint_param.deRatedVel << " 度/秒" <<
std::endl;
    //     std::cout << "  最大加速度: " << joint_param.maxAcc << " 度/秒^2" << std::endl;
    //     std::cout << "  最大减速度: " << joint_param.maxDecel << " 度/秒^2" <<
std::endl;
    //     std::cout << "  模型方向: " << (joint_param.direction == 1 ? "正向" : "反向") <<
```

```
std::endl;
    //     std::cout << "================================================" << std::endl;
    // }
    // std::cout << std::endl;
```

6.5.10 set_controller_ip - 配置控制器有线网口IP

方法：Result set_controller_ip(SOCKETFD socketFd, const std::string& name, const std::string& address, const std::string& gateway, const std::string& dns);

功能描述：配置控制器有线网口 IP

参数：

- socketFd：已连接的句柄。
- name：网卡设备名称。
- address：ip地址。
- gateway：网关地址。
- dns：DNS服务器地址。

返回值：

- Result：
- 0：成功。
- 非0：失败

使用示例：

```
    // ============================================================
    // 9. 配置有线网口 IP 地址（危险操作，默认注释）
    // ============================================================
    // ⚠️ 危险：错误配置会导致控制器网络断连，仅能在已知网络参数时使用
    //    取消下面代码的注释前，请确认 IP/网关/DNS 参数正确
    //
    // print_separator("配置有线网口 IP 地址");
    // ret = set_controller_ip(sock, "eth0", "192.168.1.14", "192.168.1.1", "8.8.8.8");
    // print_result("set_controller_ip", ret);
    // print_separator("配置有线网口 IP 地址完成, 新IP: 192.168.1.14");
    // std::cout << std::endl;
```

6.5.11 set_robot_dh_param - 设置机器人 DH 参数

方法：Result set_robot_dh_param(SOCKETFD socketFd, const RobotDHParam& param);

功能描述：设置当前机器人DH参数。

参数：

- socketFd：已连接的句柄。
- param：alpha[6]{0} 1-6轴DH参数alpha，单位：deg；a[6]{0} 1-6轴DH参数a，单位：mm；theta[6]{0}1-6轴DH参数theta，单位：deg；d[6]{0} 1-6轴DH参数d，单位：mm；eulerAngle{0} 欧拉角模式等参数。

返回值：

- 0：成功。
- 非0：失败

6.5.12 set_zero_pos_deviation - 设置零位偏差

方法：Result set_zero_pos_deviation(SOCKETFD socketFd, int axis, double shift);

功能描述：设置指定关节的零位偏差（零点偏移）。

参数：

- socketFd：已连接的句柄。
- axis：轴号（1-7）。
- shift：偏移量（-360° < shift < 360°）。

返回值：

- 0：成功。
- 非 0：失败。

6.5.13 set_axis_zero_position - 指定关节当前位置为零位/标定当前状态下的零位

方法：Result set_axis_zero_position(SOCKETFD socketFd, int axis);

功能描述：将当前关节位置记录为零位。axis=0 表示所有轴，axis=1~6 表示单轴。

警告：仅在机械臂物理对齐零刻度线后调用，否则将导致坐标系错乱，引发撞机风险！

参数：

- socket_fd：已连接的句柄。
- axis：轴号（0=全轴，1~6=单轴）

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// ----设置指定关节零位（危险操作，默认注释）----
// ⚠️ 危险：set_axis_zero_position 会修改机器人关节零位标定，
//    错误的值会导致所有后续运动出现定位偏差，严重时可能造成机械碰撞。
//    仅在专业指导下、确认当前零位确需校正时使用。
//    取消下面代码的注释前，请确认 axis 值正确且已理解全部后果。
//
// std::cout << "[INFO] **********设置指定关节零位**********" << std::endl;
// int axis = 1;
// ret = set_axis_zero_position(socket_fd, axis);
// if (ret != Result::SUCCESS) {
//     std::cerr << "[ERROR] 设置指定关节零位失败，程序退出!" << std::endl;
//     return -1;
// }
// std::cout << "[INFO] 设置指定关节零位成功" << std::endl;
// std::cout << std::endl;
```

6.5.14 set_receive_error_or_warnning_message_callback - 设置错误警告回调

方法：Result set_receive_error_or_warnning_message_callback(SOCKETFD socketFd, void (*function)(int messageType, const char *message, int messageCode));

功能描述：注册异步错误/警告回调函数。当控制器产生错误或警告时，会异步调用该回调。

参数：

- socketFd：已连接的句柄。
- function：指向回调函数的指针。该回调函数在接收到错误或警告信息时被调用。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// ---- 注册错误/警告回调 ----
print_separator("注册错误回调");
ret = tl::set_receive_error_or_warnning_message_callback(sock, on_error_or_warning);
print_result("set_receive_error_or_warnning_message_callback", ret);
```

```
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 注册回调失败" << std::endl;
        tl::disconnect_robot(sock);
        return -1;
    }
```

6.6 六维力传感器查询接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_six_axis_force.cpp、ex_six_axis_force_drag.cpp、ex_error_zero.cpp

6.6.1 set_six_dimensional_force_communication_params - 设置六维力传感器通讯参数

方法：Result set_six_dimensional_force_communication_params(SOCKETFD socketFd, SixDimensionalForceCommunicationParams& params);

功能描述：配置机械臂末端六维力传感器的通讯参数（通讯类型、拖拽使能、自动连接、方向及 EtherCAT/Modbus 等参数）。

参数：

- socketFd：已连接的句柄。
- params：输入参数，SixDimensionalForceCommunicationParams 结构体，主要成员：
    - sensorDragEnable：传感器拖拽使能（默认 true）
    - originDataInitialPara：原始数据初始参数
    - sensorCommunicationType：通讯类型，0=EtherCAT，1=Modbus RTU，2=Modbus TCP
    - startupAutoConnectSensor：启动时自动连接传感器（默认 false）
    - YDirection：Y 方向（1=正，-1=反）
    - ZDirection：Z 方向（1=正，-1=反）
    - etherCat_mapNum：EtherCAT 映射数量
    - modbus_rtu_slaveID / modbus_rtu_port / modbus_rtu_baudRate 等：Modbus RTU 通讯参数
    - modbus_tcp_IP / modbus_tcp_port 等：Modbus TCP 通讯参数

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 1. 设置六维力传感器通讯参数（EtherCAT 通讯类型）
    tl::SixDimensionalForceCommunicationParams comm_params;
    comm_params.sensorCommunicationType = 0;      // 0: EtherCAT
    comm_params.startupAutoConnectSensor = true; // 启动自动连接传感器
    comm_params.sensorDragEnable = true;          // 传感器拖拽使能
    comm_params.etherCat_mapNum = 1;              // EtherCAT 映射数量
    comm_params.YDirection = -1;                  // Y 方向（-1 反）
    comm_params.ZDirection = -1;                  // Z 方向（-1 反）

    auto ret = tl::set_six_dimensional_force_communication_params(socket_fd,
comm_params);
    if (ret == tl::Result::SUCCESS) {
        std::cout << "[成功] 设置六维力传感器通讯参数" << std::endl;
    } else {
        std::cerr << "[失败] 设置六维力传感器通讯参数失败! ret = " << ret << std::endl;
    }
```

6.6.2 get_six_dimensional_force_communication_params - 获取六维力传感器通讯参数

方法：Result get_six_dimensional_force_communication_params(SOCKETFD socketFd, SixDimensionalForceCommunicationParams& params);

功能描述：获取六维力传感器当前的通讯参数配置。

参数：

- socketFd：已连接的句柄。
- params：输出参数，SixDimensionalForceCommunicationParams 结构体，成员同 6.6.1。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询当前通讯参数（可在此基础上修改字段后写回，避免覆盖无关配置）
    tl::SixDimensionalForceCommunicationParams comm_params;
```

```
    auto ret = tl::get_six_dimensional_force_communication_params(socket_fd,
comm_params);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 查询六维力传感器通讯参数失败! ret = " << ret << std::endl;
        return -1;
    }
    std::cout << "  通讯类型: " << comm_params.sensorCommunicationType
              << " (0=EtherCAT 1=ModbusRTU 2=ModbusTCP)" << std::endl;
    std::cout << "  传感器拖拽使能: " << (comm_params.sensorDragEnable ? "开启" : "关闭")
<< std::endl;
```

6.6.3 get_sensor_base_param - 获取六维力传感器的基础参数

方法：Result get_sensor_base_param(SOCKETFD socketFd, SensorBaseParam& baseParam);

功能描述：获取六维力传感器的基础参数（质量、质心、标零状态）。

参数：

- socketFd：已连接的句柄。
- baseParam：输出参数，SensorBaseParam 结构体，主要成员：
  - sensorMass：传感器质量，单位 [kg]
  - sensorMassCenterX：质心 X，单位 [mm]
  - sensorMassCenterY：质心 Y，单位 [mm]
  - sensorMassCenterZ：质心 Z，单位 [mm]
  - saveZero：是否已标零（true=已标零）

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询标定后的基础参数
    tl::SensorBaseParam base_param;
    auto ret = tl::get_sensor_base_param(socket_fd, base_param);
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  传感器质量: " << base_param.sensorMass << std::endl;
        std::cout << "  质心 - X: " << base_param.sensorMassCenterX
                  << ", Y: " << base_param.sensorMassCenterY
                  << ", Z: " << base_param.sensorMassCenterZ << std::endl;
        std::cout << "  标零状态: " << (base_param.saveZero ? "已标零" : "未标零") <<
std::endl;
    }
```

6.6.4 mark_base_sensor - 执行六维力传感器标定

方法：Result mark_base_sensor(SOCKETFD socketFd, bool& success);

功能描述：完成六维力传感器零点偏移与正交耦合误差校准，消除自重及安装偏差带来的力信号零漂，保证后续力控采集数据精度。

参数：

- socketFd：已连接的句柄。
- success：输出参数，标定是否成功（true=成功）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 标定六维力传感器零位/重心
// 操作前请确保：
//   1. 机械臂末端无负载或已安装标准负载
//   2. 机械臂处于竖直姿态（关节各轴在合理位置）
//   3. 周围安全，标定过程中机械臂会缓慢运动
std::cout << "[INFO] 即将执行六维力传感器标定，请确认 (y/n): ";
char confirm = 'n';
std::cin >> confirm;

if (confirm == 'y' || confirm == 'Y') {
    bool calib_success = false;
    auto ret = tl::mark_base_sensor(socket_fd, calib_success);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 标定六维力传感器失败，错误码: " << ret << std::endl;
    } else if (calib_success) {
        std::cout << "[INFO] 标定六维力传感器成功" << std::endl;
    } else {
        std::cerr << "[ERROR] 标定六维力传感器失败，标定结果为 false" << std::endl;
    }
}
```

6.6.5 get_sensor_6d_data - 获取六维力传感器数据

方法：Result get_sensor_6d_data(SOCKETFD socketFd, Sensor6DData& sensorData);

功能描述：获取机械臂末端六维力传感器数据（原始数据与去皮后数据）。

参数：

- socketFd：已连接的句柄。
- sensorData：输出参数，Sensor6DData 结构体，主要成员：
  - sensorConnected：传感器连接状态（true=已连接）
  - fxData / fyData / fzData：原始力数据 Fx / Fy / Fz，单位 [N]
  - mxData / myData / mzData：原始力矩数据 Mx / My / Mz，单位 [N・m]
  - fxDataSubBase / fyDataSubBase / fzDataSubBase：去皮后力数据，单位 [N]
  - mxDataSubBase / myDataSubBase / mzDataSubBase：去皮后力矩数据，单位 [N・m]

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取六维力传感器数据
tl::Sensor6DData sensor_data;
auto ret = tl::get_sensor_6d_data(socket_fd, sensor_data);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取六维力传感器数据失败! ret = " << ret << std::endl;
    return -1;
}

std::cout << "  传感器连接状态: "
          << (sensor_data.sensorConnected ? "已连接" : "未连接") << std::endl;
std::cout << std::fixed << std::setprecision(3);

std::cout << "  --- 传感器原始数据 ---" << std::endl;
std::cout << "  Fx = " << sensor_data.fxData << " N" << std::endl;
std::cout << "  Fy = " << sensor_data.fyData << " N" << std::endl;
std::cout << "  Fz = " << sensor_data.fzData << " N" << std::endl;
std::cout << "  Mx = " << sensor_data.mxData << " N·m" << std::endl;
std::cout << "  My = " << sensor_data.myData << " N·m" << std::endl;
std::cout << "  Mz = " << sensor_data.mzData << " N·m" << std::endl;

std::cout << "  --- 去皮后数据 ---" << std::endl;
std::cout << "  Fx = " << sensor_data.fxDataSubBase << " N" << std::endl;
std::cout << "  Fy = " << sensor_data.fyDataSubBase << " N" << std::endl;
std::cout << "  Fz = " << sensor_data.fzDataSubBase << " N" << std::endl;
std::cout << "  Mx = " << sensor_data.mxDataSubBase << " N·m" << std::endl;
std::cout << "  My = " << sensor_data.myDataSubBase << " N·m" << std::endl;
std::cout << "  Mz = " << sensor_data.mzDataSubBase << " N·m" << std::endl;
```

6.6.6 get_payload_param_by_sensor - 获取六维力传感器负载参数

方法：Result get_payload_param_by_sensor(SOCKETFD socketFd, PayloadParamBySensor& param);

功能描述：获取基于六维力传感器标定的负载参数（负载质量与质心）。

参数：

- socketFd：已连接的句柄。
- param：输出参数，PayloadParamBySensor 结构体，主要成员：
  - payloadMass：负载质量，单位 [kg]
  - payloadMassCenterX：负载质心 X，单位 [mm]
  - payloadMassCenterY：负载质心 Y，单位 [mm]
  - payloadMassCenterZ：负载质心 Z，单位 [mm]

返回值：

- 0：成功。
- 非0：失败。

6.7 碰撞检测接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_collision_detect.cpp

6.7.1 set_collision - 设置碰撞防护参数

方法：Result set_collision(SOCKETFD socketFd, CollisionPara& param);

功能描述：设置碰撞防护参数（24.03 版本新增接口）。可将各轴碰撞检测系数写入控制器；指令位置响应时间（position_delay_time_ms_value）影响碰撞保护实时性，不建议手动修改，建议沿用查询到的原值。

参数：

- socketFd：已连接的句柄。
- param：输入参数，CollisionPara 结构体主要成员：
  - collisionDetection_run：碰撞检测阈值（指令），第几位为第几轴，参数范围 [1,10000]
  - collisionDetection_teach：碰撞检测阈值（点动），第几位为第几轴，参数范围 [1,10000]

- position_delay_time_ms_value：指令位置响应时间，参数范围 (0,99] ms
- error_enable_time_ms_value：误差允许时间，参数范围 [0,99] ms
- axisum：机器人轴数，默认为六轴（6）

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询当前碰撞参数，基于原值修改后写回，避免覆盖无关配置
    tl::CollisionPara cur_param;
    auto ret = tl::get_collision(sock, cur_param);
    if (ret != tl::Result::SUCCESS || cur_param.collisionDetection_run.empty()) {
        std::cerr << "[ERROR] 查询碰撞参数失败（当前固件可能不支持 get_collision）" << std::end
l;
        return -1;
    }

    tl::CollisionPara set_param = cur_param;
    // 演示：所有轴碰撞检测系数统一设为 98（系数无量纲）
    set_param.collisionDetection_run.assign(set_param.collisionDetection_run.size(), 98.
0);
    set_param.collisionDetection_teach = set_param.collisionDetection_run;  // 点动阈值同
步
    // 指令位置响应时间建议沿用查询原值，不手动设置

    ret = tl::set_collision(sock, set_param);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 设置碰撞参数失败（当前固件可能不支持 set_collision）" << std::end
l;
        return -1;
    }
    std::cout << "[信息] 设置碰撞参数成功（系数=98）" << std::endl;
```

6.7.2 get_collision - 获取碰撞防护参数

方法：Result get_collision(SOCKETFD socketFd, CollisionPara& param);

功能描述：获取碰撞防护参数（24.03 版本新增接口）。

说明：与 get_collision_detection_param 功能相同，为推荐使用的查询接口。

参数：

- socketFd：已连接的句柄。

- param：输出参数，CollisionPara 结构体，成员同 6.7.1。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询当前碰撞防护参数
    tl::CollisionPara collision_param;
    auto ret = tl::get_collision(sock, collision_param);
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  碰撞检测阈值（指令）: ";
        for (const auto& val : collision_param.collisionDetection_run) {
            std::cout << val << " ";
        }
        std::cout << std::endl;
        std::cout << "  碰撞检测阈值（点动）: ";
        for (const auto& val : collision_param.collisionDetection_teach) {
            std::cout << val << " ";
        }
        std::cout << std::endl;
        std::cout << "  指令位置响应时间: " <<
collision_param.position_delay_time_ms_value << " ms" << std::endl;
        std::cout << "  误差允许时间: " << collision_param.error_enable_time_ms_value <<
" ms" << std::endl;
        std::cout << "  机器人轴数: " << collision_param.axisum << std::endl;
    }
```

6.8 电机与关节监控接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_info_query.cpp

6.8.1 get_current_motor_speed - 获取各关节电机当前转速

方法：Result get_current_motor_speed(SOCKETFD socketFd, std::vector< int >& motor_speed, std::vector< int >& motor_speed_sync);

功能描述：获取当前各关节电机转速。

参数：

- socketFd：已连接的句柄。
- motor_speed：输出参数，机器人本体电机转速，长度 7，单位 [RPM]。

- motor_speed_sync：输出参数，外部轴电机转速，长度 5，单位 [RPM]。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询各关节电机当前转速----
    std::cout << "[INFO] **********查询各关节电机当前转速**********" << std::endl;
    {
        std::vector<int> motor_speed;        // 机器人本体电机转速，长度 7
        std::vector<int> motor_speed_sync;   // 外部轴电机转速，长度 5
        auto ret = tl::get_current_motor_speed(socket_fd, motor_speed, motor_speed_sync)
;
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取电机转速失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 电机转速 (本体, RPM): ";
            for (const auto& val : motor_speed) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
            std::cout << "[INFO] 电机转速 (外部轴, RPM): ";
            for (const auto& val : motor_speed_sync) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.8.2 get_current_motor_torque - 获取各关节电机当前扭矩

方法：Result get_current_motor_torque(SOCKETFD socketFd, std::vector< int >& motor_torque, std::vector< int >& motor_torque_sync);

功能描述：获取当前各关节电机扭矩。

参数：

- socketFd：已连接的句柄。
- motor_torque：输出参数，机器人本体电机扭矩，长度 7，单位 [%]。
- motor_torque_sync：输出参数，外部轴电机扭矩，长度 5，单位 [%]。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询各关节电机当前扭矩----
    std::cout << "[INFO] **********查询各关节电机当前扭矩**********" << std::endl;
    {
        std::vector<int> motor_torque;       // 机器人本体电机扭矩，长度 7
        std::vector<int> motor_torque_sync;  // 外部轴电机扭矩，长度 5
        auto ret = tl::get_current_motor_torque(socket_fd, motor_torque, motor_torque_sy
nc);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取电机扭矩失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 电机扭矩 (本体, %): ";
            for (const auto& val : motor_torque) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
            std::cout << "[INFO] 电机扭矩 (外部轴, %): ";
            for (const auto& val : motor_torque_sync) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.8.3 get_current_motor_current - 获取各关节电机当前电流

方法：Result get_current_motor_current(SOCKETFD socketFd, std::vector< double >& motor_current, std::vector< double >& motor_current_sync);

功能描述：获取当前各关节电机电流（24.03/DEV 接口）。

参数：

- socketFd：已连接的句柄。
- motor_current：输出参数，机器人本体电机电流，长度 7，单位 ‰。
- motor_current_sync：输出参数，外部轴电机电流，长度 5，单位 ‰。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
    // ----查询各关节电机当前电流----
    std::cout << "[INFO] **********查询各关节电机当前电流**********" << std::endl;
    {
        std::vector<double> motor_current;       // 机器人本体电机电流，长度 7
        std::vector<double> motor_current_sync;  // 外部轴电机电流，长度 5
        auto ret = tl::get_current_motor_current(socket_fd, motor_current, motor_current
_sync);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取电机电流失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 电机电流 (本体, %): ";
            for (const auto& val : motor_current) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
            std::cout << "[INFO] 电机电流 (外部轴, %): ";
            for (const auto& val : motor_current_sync) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.8.4 get_current_motor_payload - 获取各关节电机当前负载

方法：Result get_current_motor_payload(SOCKETFD socketFd, std::vector< double >& motor_payload, std::vector< double >& motor_payload_sync);

功能描述：获取当前各关节电机负载。

参数：

- socketFd：已连接的句柄。
- motor_payload：输出参数，机器人本体电机负载，长度 7，单位 [%]。
- motor_payload_sync：输出参数，外部轴电机负载，长度 5，单位 [%]。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询各关节电机当前负载----
    std::cout << "[INFO] **********查询各关节电机当前负载**********" << std::endl;
    {
        std::vector<double> motor_payload;       // 机器人本体电机负载，长度 7
        std::vector<double> motor_payload_sync;  // 外部轴电机负载，长度 5
        auto ret = tl::get_current_motor_payload(socket_fd, motor_payload, motor_payload
_sync);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取电机负载失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 电机负载 (本体, %): ";
            for (const auto& val : motor_payload) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
            std::cout << "[INFO] 电机负载 (外部轴, %): ";
            for (const auto& val : motor_payload_sync) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.8.5 get_joint_voltage - 获取各关节电压

方法：Result get_joint_voltage(SOCKETFD socketFd, std::vector< double >& joint_voltage, std::vector< double >& positioner_voltage);

功能描述：获取当前各关节电压。

参数：

- socketFd：已连接的句柄。
- joint_voltage：输出参数，机器人本体各关节电压。
- positioner_voltage：输出参数，外部轴各关节电压。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询各关节电压----
    std::cout << "[INFO] **********查询各关节电压**********" << std::endl;
    {
```

```
        std::vector<double> joint_voltage;       // 机器人本体各关节电压
        std::vector<double> positioner_voltage;  // 外部轴各关节电压
        auto ret = tl::get_joint_voltage(socket_fd, joint_voltage, positioner_voltage);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取关节电压失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 关节电压 (本体): ";
            for (const auto& val : joint_voltage) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
            std::cout << "[INFO] 关节电压 (外部轴): ";
            for (const auto& val : positioner_voltage) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.8.6 get_joint_temperature - 获取各关节电机温度

方法：Result get_joint_temperature(SOCKETFD socketFd, std::vector< double >& temperatures);

功能描述：获取当前各关节电机温度（单位：摄氏度 ℃）。

参数：

- socketFd：已连接的句柄。
- temperatures：输出参数，各关节温度列表。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // ----查询各关节电机温度----
    std::cout << "[INFO] **********查询各关节电机温度**********" << std::endl;
    {
        std::vector<double> temperatures;  // 各关节温度
        auto ret = tl::get_joint_temperature(socket_fd, temperatures);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取关节温度失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 关节温度 (℃): ";
            for (const auto& val : temperatures) {
                std::cout << val << " ";
```

```
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

6.9 作业运动控制接口

头文件：include/tl_job_operate.h 相关示例代码：examples\cpp\ex_job_control.cpp

6.9.1 job_create - 创建作业文件名

方法：Result job_create(SOCKETFD socketFd, const std::string& jobName);

功能描述：在控制器内新建一个空的作业文件（自动防止重名）。

参数：

- socketFd：已连接的句柄。
- jobName：作业文件名。

返回值：

- 0：成功。
- 非0：失败（如文件已存在）。

使用示例：

```
    // 创建作业文件
    std::string job_name = "TEST_CPP";
    auto ret = tl::job_create(socket_fd, job_name);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 创建作业文件失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 创建作业文件成功" << std::endl;
```

6.9.2 job_open - 打开指定作业文件

方法：Result job_open(SOCKETFD socketFd, const std::string& jobName);

功能描述：打开已存在的作业文件，使其成为当前激活程序（后续插入指令将写入该文件）。

参数：

- socketFd：已连接的句柄。
- jobName：作业文件名（如 "MY_JOB" 或 "MY_JOB.JOB"）。

返回值：

- 0：成功。
- 非0：失败（文件不存在等）。

使用示例：

```
// 打开作业文件
auto ret = tl::job_open(socket_fd, job_name);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 打开作业文件失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 打开作业文件成功" << std::endl;
```

6.9.3 job_insert_moveJ - 作业里添加关节空间运动

方法：Result job_insert_moveJ(SOCKETFD socketFd, int line, MoveCmd moveCmd);

功能描述：在作业文件指定行插入一条关节空间运动（MoveJ）指令。

参数：

- socketFd：已连接的句柄。
- line：行号（从1开始）。
- moveCmd：运动命令结构体，需设置以下成员：
    - targetPosType：目标位置类型，PosType 枚举（data / PType / E_TYPE / RP_TYPE / AP_TYPE / GPType / GEType）
    - targetPosName：目标位置名称（如 "P0001"）
    - coord：坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）
    - velocity：速度
    - acc：加速度
    - dec：减速度

- pl：平滑度（0表示精确到达）
- targetPosValue：目标位置数据，长度 14（前 7 位本体值，后 7 位外部轴值）

返回值：

- 0：成功。
- 非0：失败（行号越界、参数错误等）。

使用示例：

```
// 作业文件插入 MoveJ（P 变量模式）
int line = 1;
tl::MoveCmd cmd;
cmd.targetPosType = tl::PosType::PType;   // 使用 P 变量
cmd.targetPosName = "P0001";
cmd.coord         = tl::Coord::JOINT;     // 关节坐标系
cmd.velocity      = 40;
cmd.acc           = 10;
cmd.dec           = 10;
cmd.pl            = 0;                    // 精确到达
cmd.targetPosValue = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
                      0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};

auto ret = tl::job_insert_moveJ(socket_fd, line, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 作业文件插入 MoveJ 失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 作业文件插入 MoveJ 成功" << std::endl;
```

6.9.4 job_insert_moveL - 作业里添加笛卡尔空间直线运动

方法：Result job_insert_moveL(SOCKETFD socketFd, int line, MoveCmd moveCmd);

功能描述：插入笛卡尔空间直线运动（MoveL）指令。

参数：

- socketFd：已连接的句柄。
- line：行号（从1开始）。
- moveCmd：运动命令结构体，成员同 6.9.3。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
// 作业文件插入 MoveL（P 变量模式）
int line = 2;
tl::MoveCmd cmd;
cmd.targetPosType = tl::PosType::PType;
cmd.targetPosName = "P0002";
cmd.coord         = tl::Coord::BASE;      // 基坐标系（直角）
cmd.velocity      = 40;
cmd.acc           = 10;
cmd.dec           = 10;
cmd.pl            = 0;
cmd.targetPosValue = {280, 0.0, 380, 3.14, 0.0, 0.0};  // X, Y, Z, Rx, Ry, Rz

auto ret = tl::job_insert_moveL(socket_fd, line, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 作业文件插入 MoveL 失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 作业文件插入 MoveL 成功" << std::endl;
```

6.9.5 job_insert_moveC - 作业里添加笛卡尔空间圆弧运动

方法：Result job_insert_moveC(SOCKETFD socketFd, int line, MoveCmd moveCmd);

功能描述：插入笛卡尔空间圆弧运动（MoveC）指令。通常需要连续插入两条 MoveC 指令，第一条为圆弧中间点，第二条为终点。

参数：

- socketFd：已连接的句柄。
- line：行号（从1开始）。
- moveCmd：运动命令结构体，成员同 6.9.3。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 作业文件插入 MoveC（圆弧中间点 + 终点，共两条）
int line = 3;
tl::MoveCmd cmd;
cmd.targetPosType = tl::PosType::PType;
```

```
    cmd.targetPosName = "P0003";
    cmd.coord         = tl::Coord::TOOL;      // 工具坐标系
    cmd.velocity      = 40;
    cmd.acc           = 10;
    cmd.dec           = 10;
    cmd.pl            = 0;

    // 第一条：圆弧中间点
    cmd.targetPosValue = {270, 0.0, 380, 3.14, 0.0, 0.0};
    auto ret = tl::job_insert_moveC(socket_fd, line, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 作业文件插入 MoveC（中间点）失败，错误码: " << ret << std::endl;
        return -1;
    }

    // 第二条：圆弧终点
    cmd.targetPosValue = {240, 0.0, 380, 3.14, 0.0, 0.0};
    ret = tl::job_insert_moveC(socket_fd, line, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 作业文件插入 MoveC（终点）失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 作业文件插入 MoveC 成功" << std::endl;
```

6.9.6 job_run - 启动指定作业文件

方法：Result job_run(SOCKETFD socketFd, const std::string& jobName);

功能描述：启动运行已打开的作业文件（该功能需要在运行模式下使用，调用该接口时，控制器会自动进行伺服模式切换）。

参数：

- socketFd：已连接的句柄。
- jobName：作业文件名。

返回值：

- 0：成功。
- 非0：失败（文件未打开、未使能等）。

使用示例：

```
    // 运行作业文件
    auto ret = tl::job_run(socket_fd, job_name);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 运行作业文件失败，错误码: " << ret << std::endl;
        return -1;
```

```
    }
    std::cout << "[信息] 运行作业文件成功" << std::endl;
```

6.9.7 job_get_all_jobfile_name - 获取全部作业文件名

方法：Result job_get_all_jobfile_name(SOCKETFD socketFd, std::vector< std::vector< std::string >>& robotsFile);

功能描述：获取控制器内所有作业文件列表（按机器人分组，通常只有一组）。

参数：

- socketFd：已连接的句柄。
- robotsFile：输出参数，二维字符串数组，每个元素是一个分组内的文件名列表。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取所有作业文件名
    std::vector<std::vector<std::string>> all_job_names;
    auto ret = tl::job_get_all_jobfile_name(socket_fd, all_job_names);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取作业文件名失败，错误码: " << ret << std::endl;
        return -1;
    }

    int i = 1;
    for (const auto& names : all_job_names) {
        for (const auto& name : names) {
            std::cout << "[信息] 作业文件 " << i++ << " 名称: " << name << std::endl;
        }
    }
```

6.10 实时运动控制接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_move_control.cpp

6.10.1 robot_movej - 关节空间运动到目标位姿

方法：Result robot_movej(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：立即下发关节角度数据，驱动机械臂实时运动到目标位置（在线点对点控制）。不依赖作业文件（该功能需要在示教模式下使用）。

参数：

- socketFd：已连接的句柄。
- moveCmd：运动命令结构体，需设置以下成员：
  - targetPosType：目标位置类型，PosType 枚举（data / PType / E_TYPE / RP_TYPE / AP_TYPE / GPType / GEType）
  - targetPosName：目标位置名称（使用 P 变量模式时可为空）
  - coord：坐标系，Coord 枚举（JOINT / BASE / TOOL / USER），范围 0~3
  - velocity：速度，范围 0 < velocity ⩽ 100，单位 %
  - acc：加速度，范围 0 < acc ⩽ 100
  - dec：减速度，范围 0 < dec ⩽ 100
  - pl：平滑度（0 表示精确到达）
  - targetPosValue：目标位置数据，n 个轴就赋值前 n 位数组，其余置 0；关节坐标下单位为度

返回值：

- 0：成功。
- 非0：失败（如超限位、未使能等）。

使用示例：

```
// MoveJ 运动控制
tl::MoveCmd cmd;
cmd.targetPosType  = tl::PosType::PType;
cmd.targetPosName  = "";
cmd.coord          = tl::Coord::JOINT;   // 关节坐标系
cmd.velocity       = 40;                 // 速度，单位 %
cmd.acc            = 10;
cmd.dec            = 10;
cmd.pl             = 0;                  // 精确到达
cmd.targetPosValue = std::vector<double>{
    20.0, 10.0, -10.0, 0.0, 0.0, 0.0, 0.0}; // 关节坐标下单位为度

auto ret = tl::robot_movej(socket_fd, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] MoveJ运动控制失败，错误码: " << ret << std::endl;
    return -1;
```

```
}
// 等待运动到位
wait_motion_done(socket_fd, 30);
std::cout << "[信息] MoveJ运动控制成功" << std::endl;
```

6.10.2 robot_movel - 笛卡尔空间直线运动到目标位姿

方法：Result robot_movel(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：立即下发直角坐标数据，驱动机械臂末端沿直线运动到目标位姿。

参数：

- socketFd：已连接的句柄。
- moveCmd：运动命令结构体，需设置以下成员：
  - targetPosType：目标位置类型，PosType 枚举（data / PType / E_TYPE / RP_TYPE / AP_TYPE / GPType / GEType）
  - targetPosName：目标位置名称（使用 P 变量模式时可为空）
  - coord：坐标系，Coord 枚举（JOINT / BASE / TOOL / USER），范围 0~3
  - velocity：速度，范围 0 < velocity ⩽ 1000，单位 mm/s
  - acc：加速度，范围 0 < acc ⩽ 100
  - dec：减速度，范围 0 < dec ⩽ 100
  - pl：平滑度（0 表示精确到达）
  - targetPosValue：目标位置数据，直角坐标下位移单位 mm、旋转单位弧度，如 [X, Y, Z, RX, RY, RZ]

返回值：

- 0：成功。
- 非0：失败（如超限位、未使能等）。

使用示例：

```
// MoveL 运动控制
tl::MoveCmd cmd;
cmd.targetPosType  = tl::PosType::PType;
cmd.targetPosName  = "";
cmd.coord          = tl::Coord::BASE;    // 基坐标系（直角）
cmd.velocity       = 40;                 // 速度，单位 mm/s
```

```
    cmd.acc            = 10;
    cmd.dec            = 10;
    cmd.pl             = 0;
    cmd.targetPosValue = std::vector<double>{
        280, 90.0, 270, 3.14, 0.0, 0.0}; // 直角坐标下位移单位 mm，旋转单位弧度

    auto ret = tl::robot_movel(socket_fd, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] MoveL运动控制失败，错误码: " << ret << std::endl;
        return -1;
    }
    // 等待运动到位
    wait_motion_done(socket_fd, 30);
    std::cout << "[信息] MoveL运动控制成功" << std::endl;
```

6.10.3 robot_go_home - 机器人回原点（home）

方法：Result robot_go_home(SOCKETFD socketFd);

功能描述：回到设定的零点。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 回到设定的零点
    auto ret = tl::robot_go_home(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 回到零点失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 机器人回到零点成功" << std::endl;
```

6.10.4 robot_go_to_reset_position - 机器人回到复位位置

方法：Result robot_go_to_reset_position(SOCKETFD socketFd);

功能描述：回到控制器预设的复位点。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败（如控制器未配置复位点）。

使用示例：

```
// 回到控制器预设的复位点
auto ret = tl::robot_go_to_reset_position(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 回到复位点失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 机器人回到复位位置成功" << std::endl;
```

6.11 坐标系与用户坐标管理接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_coord_transform.cpp

6.11.1 set_user_coordinate_data - 设置用户坐标

方法：Result set_user_coordinate_data(SOCKETFD socketFd, int userNum, const UserCoordParam& param);

功能描述：设置（标定）用户坐标系参数。推荐使用 UserCoordParam 结构体重载版本（含类型/联动信息），旧版 vector< double > 参数已废弃。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。
- param：用户坐标参数结构体，可设置以下成员：
    - location_type：用户坐标类型，0：静态 1：联动 2：动态
    - name：用户坐标名称
    - position[6]：用户坐标参数 [X, Y, Z, A, B, C]
    - related_type：联动对象类型，0：机器人 1：外部轴

- related_num：联动对象编号
- related_coord_type：联动坐标类型，0：直角 1：工具 2：用户
- related_coord_num：联动坐标编号

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置用户坐标系（编号 1，与基坐标重合）
int userNum = 1;
tl::UserCoordParam user_coord;
user_coord.location_type = 0;          // 静态
user_coord.name          = "USER01";
user_coord.position[0]   = 0.0;        // X (mm)
user_coord.position[1]   = 0.0;        // Y (mm)
user_coord.position[2]   = 0.0;        // Z (mm)
user_coord.position[3]   = 0.0;        // A (rad)
user_coord.position[4]   = 0.0;        // B (rad)
user_coord.position[5]   = 0.0;        // C (rad)

auto ret = tl::set_user_coordinate_data(socket_fd, userNum, user_coord);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置用户坐标系失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置用户坐标系成功" << std::endl;
```

6.11.2 set_user_coord_number - 设置用户坐标编号

方法：Result set_user_coord_number(SOCKETFD socketFd, int userNum);

功能描述：设置当前使用的用户坐标编号，后续运动将基于该用户坐标系。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置用户坐标编号为 2
int userNum = 2;
auto ret = tl::set_user_coord_number(socket_fd, userNum);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置用户坐标编号失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置用户坐标编号成功" << std::endl;
```

6.11.3 get_user_coord_para - 获取当前使用的用户坐标的参数

方法：Result get_user_coord_para(SOCKETFD socketFd, int userNum, UserCoordParam& param);

功能描述：查询指定编号的用户坐标参数（返回 UserCoordParam 结构体，含类型/联动信息），旧版 vector< double > 输出已废弃。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。
- param：输出，用户坐标参数结构体（UserCoordParam）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取用户坐标参数（编号 1）
int userNum = 1;
tl::UserCoordParam user_para;
auto ret = tl::get_user_coord_para(socket_fd, userNum, user_para);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取用户坐标参数失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 用户坐标名称: " << user_para.name << std::endl;
std::cout << "[信息] 用户坐标位置: ["
          << user_para.position[0] << ", "
          << user_para.position[1] << ", "
          << user_para.position[2] << ", "
          << user_para.position[3] << ", "
```

```
              << user_para.position[4] << ", "
              << user_para.position[5] << "]" << std::endl;
```

6.11.4 get_user_coord_number - 获取当前使用的用户坐标编号

方法：Result get_user_coord_number(SOCKETFD socketFd, int& userNum);

功能描述：获取当前使用的用户坐标编号。

参数：

- socketFd：已连接的句柄。
- userNum：输出，当前使用的用户坐标编号。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取当前用户坐标编号
int userNum = 0;
auto ret = tl::get_user_coord_number(socket_fd, userNum);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取用户坐标编号失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 当前用户坐标编号: " << userNum << std::endl;
```

6.11.5 get_current_coord - 获取当前工作坐标系

方法：Result get_current_coord(SOCKETFD socketFd, Coord& coord);

功能描述：获取机器人当前工作坐标系（推荐使用 Coord 枚举重载版本，int 输出旧版已废弃）。

参数：

- socketFd：已连接的句柄。
- coord：输出，当前工作坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取当前工作坐标系
tl::Coord coord;
auto ret = tl::get_current_coord(socket_fd, coord);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取当前工作坐标系失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 当前坐标系编号: " << static_cast<int>(coord) << std::endl;
```

6.11.6 set_current_coord - 切换示教运动坐标系

方法：Result set_current_coord(SOCKETFD socketFd, Coord coord);

功能描述：切换示教运动坐标系（推荐使用 Coord 枚举重载版本，int 旧版已废弃）。

参数：

- socketFd：已连接的句柄。
- coord：目标坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 切换当前工作坐标系为基坐标系
auto ret = tl::set_current_coord(socket_fd, tl::Coord::BASE);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置当前工作坐标系失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置当前工作坐标系成功" << std::endl;
```

6.11.7 calibration_oxy - 三点法自动设置用户坐标系

方法：Result calibration_oxy(SOCKETFD socketFd, int userNum, std::string xyo);

功能描述：三点法用户坐标系标定。依次手动移动机械臂到原点 O、X 方向点、Y 方向点，分别调用本接口标定，最后用 calculate_user_coordinate 计算用户坐标。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。
- xyo：标定类型，取值 'X' / 'Y' / 'O'。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 三点法标定（需先手动移动机械臂到对应位置）
    // 步骤1：移动到原点 O，标定 O
    auto ret = tl::calibration_oxy(socket_fd, 1, "O");
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 标定原点 O 失败，错误码：" << ret << std::endl;
        return -1;
    }

    // 步骤2：移动到 X 方向点，标定 X
    ret = tl::calibration_oxy(socket_fd, 1, "X");
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 标定 X 方向点失败，错误码：" << ret << std::endl;
        return -1;
    }

    // 步骤3：移动到 Y 方向点，标定 Y
    ret = tl::calibration_oxy(socket_fd, 1, "Y");
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 标定 Y 方向点失败，错误码：" << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 三点标定完成，下一步请调用 calculate_user_coordinate 计算坐标" << std:
:endl;
```

6.11.8 calculate_user_coordinate - 计算用户坐标

方法：Result calculate_user_coordinate(SOCKETFD socketFd, int userNumber);

功能描述：根据三点法标定的 O / X / Y 数据计算用户坐标系（需先完成 calibration_oxy 标定）。

参数：

- socketFd：已连接的句柄。

- userNumber：用户坐标编号。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 计算用户坐标（三点法标定完成后调用）
    auto ret = tl::calculate_user_coordinate(socket_fd, 1);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 计算用户坐标失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 用户坐标计算成功" << std::endl;
```

6.11.9 get_target_coord_pos_value - 获取目标坐标位置值

方法：Result get_target_coord_pos_value(SOCKETFD socketFd, std::string name, Coord targetCoord, std::vector< double >& targetPos);

功能描述：获取指定点位在目标坐标系下的位置值（推荐使用 Coord 枚举重载版本，int 旧版已废弃）。

参数：

- socketFd：已连接的句柄。
- name：点位名（P 点、GP 点、E 点、GE 点），如 "GP0001"。
- targetCoord：目标坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）。
- targetPos：输出，转换后的位置，14 位点位（坐标系, 角度/弧度, 形态/左右手, 工具号, 用户坐标号, 预留, 预留, 1~7 轴）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取全局点位 GP0001 在基坐标系下的位置值
    std::vector<double> target_pos;
    auto ret = tl::get_target_coord_pos_value(socket_fd, "GP0001", tl::Coord::BASE, targ
et_pos);
    if (ret != tl::Result::SUCCESS) {
```

```
        std::cerr << "[ERROR] 获取目标坐标位置值失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] GP0001 在基坐标系下的位置（共 " << target_pos.size() << " 位）:" <<
std::endl;
    for (size_t i = 0; i < target_pos.size(); ++i) {
        std::cout << "  [" << i << "] " << target_pos[i] << std::endl;
    }
```

6.11.10 get_local_pos_p_value - 获取本地位置变量 P 值

方法：Result get_local_pos_p_value(SOCKETFD socketFd, std::string name, std::vector< double >& pos);

功能描述：获取本地位置变量 P 点的位置值。

参数：

- socketFd：已连接的句柄。
- name：点位名，范围 P0001 - P9999。
- pos：输出，转换后的位置，14 位点位（坐标系, 角度/弧度, 形态/左右手, 工具号, 用户坐标号, 预留, 预留, 1~7 轴）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取本地位置变量 P0001 的值
    std::vector<double> pos;
    auto ret = tl::get_local_pos_p_value(socket_fd, "P0001", pos);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取本地位置变量 P 值失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] P0001 的位置（共 " << pos.size() << " 位）:" << std::endl;
    for (size_t i = 0; i < pos.size(); ++i) {
        std::cout << "  [" << i << "] " << pos[i] << std::endl;
    }
```

6.11.11 set_default_cartesian_params - 设置笛卡尔参数为默认值

方法：Result set_default_cartesian_params(SOCKETFD socketFd);

功能描述：将机械臂末端（笛卡尔）相关参数恢复为默认值。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置机械臂末端参数为默认值
auto ret = tl::set_default_cartesian_params(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置笛卡尔参数为默认值失败，错误码： " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置笛卡尔参数为默认值成功" << std::endl;
```

6.12 工具手相关接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_tool_hand_number.cpp、examples\cpp\ex_coord_tool.cpp、examples\cpp\ex_info_query.cpp

6.12.1 set_tool_hand_number - 设置工具手编号

方法：Result set_tool_hand_number(SOCKETFD socketFd, int toolNum);

功能描述：设定末端工具对应的工具组编号，用于控制器匹配工具坐标系、负载参数及碰撞检测包络配置。

参数：

- socketFd：已连接的句柄。
- toolNum：工具手编号。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置工具手编号为 2
int tool_num = 2;
auto ret = tl::set_tool_hand_number(socket_fd, tool_num);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置工具手编号失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置工具手编号成功（toolNum = " << tool_num << "）" << std::endl;
```

6.12.2 get_tool_hand_number - 获取当前工具手编号

方法：Result get_tool_hand_number(SOCKETFD socketFd, int& toolNum);

功能描述：获取当前使用的工具手编号。

参数：

- socketFd：已连接的句柄。
- toolNum：输出，当前使用的工具手编号。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取当前工具手编号（set 后需加延迟再 get，确保读取到最新值）
std::this_thread::sleep_for(std::chrono::milliseconds(500));
int get_tool_num = -1;
auto ret = tl::get_tool_hand_number(socket_fd, get_tool_num);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取工具手编号失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 当前工具手编号: " << get_tool_num << std::endl;
```

6.12.3 set_tool_hand_param - 设置工具手参数

方法：Result set_tool_hand_param(SOCKETFD socketFd, int toolNum, ToolParam param);

功能描述：设置指定工具手的参数（工具坐标系偏移、负载等）。

参数：

- socketFd：已连接的句柄。

- toolNum：工具手编号。
- param：工具手参数结构体（ToolParam），可设置以下成员：
    - X / Y / Z：X / Y / Z 轴偏移，单位 mm
    - A / B / C：绕 A / B / C 轴旋转，单位 度
    - payloadMass：负载质量，单位 kg
    - payloadInertia：负载惯量
    - payloadMassCenter_X / Y / Z：负载质心，单位 mm
    - note：注释
    - maxToolRangeX / Y / Z、minToolRangeX / Y / Z：工具干涉区最大 / 最小范围
    - toolHandMethod：工具手方式，0：控制点 1：立方体

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置工具手参数（编号 1）
int tool_num = 1;
tl::ToolParam tool_param;
tool_param.X = 0;      // X 轴偏移 (mm)
tool_param.Y = 0;      // Y 轴偏移 (mm)
tool_param.Z = 0;      // Z 轴偏移 (mm)
tool_param.A = 0;      // 绕 A 轴旋转 (度)
tool_param.B = 0;      // 绕 B 轴旋转 (度)
tool_param.C = 0;      // 绕 C 轴旋转 (度)

auto ret = tl::set_tool_hand_param(socket_fd, tool_num, tool_param);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置工具手参数失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置工具手参数成功" << std::endl;
```

6.12.4 get_tool_hand_param - 获取当前工具手参数

方法：Result get_tool_hand_param(SOCKETFD socketFd, int toolNum, ToolParam& param);

功能描述：获取指定工具手的参数（结构体成员同 6.12.3）。

参数：

- socketFd：已连接的句柄。
- toolNum：工具手编号。
- param：输出，工具手参数结构体（ToolParam）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取工具手参数（编号 1）
int tool_num = 1;
tl::ToolParam tool_param;
auto ret = tl::get_tool_hand_param(socket_fd, tool_num, tool_param);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取工具手参数失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 工具手 " << tool_num << " 参数:" << std::endl;
std::cout << "  X=" << tool_param.X << " Y=" << tool_param.Y
          << " Z=" << tool_param.Z << std::endl;
std::cout << "  A=" << tool_param.A << " B=" << tool_param.B
          << " C=" << tool_param.C << std::endl;
```

6.12.5 set_tool_coordinate_range - 设置工具包络球参数（工具坐标范围参数）

方法：Result set_tool_coordinate_range(SOCKETFD socketFd, int tool_number, const ToolCoordinateRange& range_param);

功能描述：设置工具坐标范围参数（工具包络球），用于碰撞检测包络计算。

参数：

- socketFd：已连接的句柄。
- tool_number：工具编号。
- range_param：工具坐标范围参数结构体（ToolCoordinateRange）：
  - min_range_x / max_range_x：X 轴最小 / 最大范围
  - min_range_y / max_range_y：Y 轴最小 / 最大范围

- min_range_z / max_range_z：Z 轴最小 / 最大范围
- tool_hand_method：工具握持方式

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 设置工具坐标范围参数（编号 1）
    int tool_number = 1;
    tl::ToolCoordinateRange range_param;
    range_param.min_range_x = -500.0;
    range_param.max_range_x = 500.0;
    range_param.min_range_y = -400.0;
    range_param.max_range_y = 400.0;
    range_param.min_range_z = -300.0;
    range_param.max_range_z = 300.0;
    range_param.tool_hand_method = 1;

    auto ret = tl::set_tool_coordinate_range(socket_fd, tool_number, range_param);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 设置工具" << tool_number << "坐标范围参数失败，错误码： " <<
ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 设置工具坐标范围参数成功" << std::endl;
```

6.12.6 get_tool_coordinate_range - 获取工具包络球参数（工具坐标范围参数）

方法：Result get_tool_coordinate_range(SOCKETFD socketFd, int tool_number, ToolCoordinateRange& range_param);

功能描述：获取指定工具的坐标范围参数（工具包络球，结构体成员同 6.12.5）。

参数：

- socketFd：已连接的句柄。
- tool_number：工具编号。
- range_param：输出，工具坐标范围参数结构体（ToolCoordinateRange）。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
    // 获取工具坐标范围参数（编号 1）
    int tool_number = 1;
    tl::ToolCoordinateRange range_param;
    auto ret = tl::get_tool_coordinate_range(socket_fd, tool_number, range_param);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取工具" << tool_number << "坐标范围参数失败，错误码： " <<
ret << std::endl;
        return -1;
    }
    std::cout << std::fixed << std::setprecision(4);
    std::cout << "[信息] 工具 " << tool_number << " 坐标范围参数:" << std::endl;
    std::cout << "  X轴范围: [" << range_param.min_range_x
              << ", " << range_param.max_range_x << "] mm" << std::endl;
    std::cout << "  Y轴范围: [" << range_param.min_range_y
              << ", " << range_param.max_range_y << "] mm" << std::endl;
    std::cout << "  Z轴范围: [" << range_param.min_range_z
              << ", " << range_param.max_range_z << "] mm" << std::endl;
    std::cout << "  工具握持方式： " << range_param.tool_hand_method << std::endl;
```

6.13 示教操作接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_teach_control.cpp

6.13.1 robot_start_jogging - 开始点动

方法：Result robot_start_jogging(SOCKETFD socketFd, int axis, bool dir);

功能描述：开始示教点动，驱动指定轴沿指定方向连续运动（需在示教模式下使用，且会直接驱动机器人运动，运行前请确保安全区域无人）。

参数：

- socketFd：已连接的句柄。
- axis：轴号。
- dir：方向，true 正方向 / false 反方向。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 示教点动（需先切换为示教模式）
auto ret = tl::set_current_mode(socket_fd, tl::RobotMode::TEACH);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 切换示教模式失败，错误码: " << ret << std::endl;
    return -1;
}

// 开始点动：轴 1，正方向
int axis = 1;
bool direction = true;
ret = tl::robot_start_jogging(socket_fd, axis, direction);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 示教点动失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 示教点动成功" << std::endl;

// 点动运行一段时间后，调用 robot_stop_jogging 停止
std::this_thread::sleep_for(std::chrono::seconds(5));
```

6.13.2 robot_stop_jogging - 停止点动

方法：Result robot_stop_jogging(SOCKETFD socketFd, int axis);

功能描述：停止指定轴的点动运动。

参数：

- socketFd：已连接的句柄。
- axis：轴号。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 停止轴 1 的点动
int axis = 1;
auto ret = tl::robot_stop_jogging(socket_fd, axis);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 停止点动失败，错误码: " << ret << std::endl;
    return -1;
```

```
    }
    std::cout << "[信息] 停止点动成功" << std::endl;
```

6.14 全局路点管理接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_global_position.cpp

6.14.1 set_global_position - 新增/设置全局路点

方法：Result set_global_position(SOCKETFD socketFd, std::string posName, std::vector< double > pos);

功能描述：创建或修改一个持久化的全局位置变量（路点），可通过名称直接调用。

参数：

- socketFd：已连接的句柄。
- posName：全局位置名，如 "GP0001"。
- pos：14 位点位数组，含义如下：
    - [0]：坐标系，0：关节 1：基坐标 2：工具 3：用户
    - [1]：0 角度制 / 1 弧度制
    - [2]：形态
    - [3]：工具手坐标序号
    - [4]：用户坐标序号
    - [5] [6]：备用
    - [7-13]：点位信息

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 设置全局路点
    std::string posName = "GP_test0";
    std::vector<double> posInfo {0, 1, 2, 3, 4, 5, 6};
    auto ret = tl::set_global_position(socket_fd, posName, posInfo);
```

```
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置全局路点失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置全局路点成功" << std::endl;
```

6.14.2 get_global_position - 查询全局 GP 点位

方法：Result get_global_position(SOCKETFD socketFd, std::string posName, std::vector< double >& pos);

功能描述：查询指定名称的全局 GP 点位。

参数：

- socketFd：已连接的句柄。
- posName：全局位置名，如 "GP0001"。
- pos：输出，全局点位数组，长度 14；前 7 位为点位的坐标、姿态等信息，后 7 位为机器人位置。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 查询全局路点
std::vector<double> queriedPos;
auto ret = tl::get_global_position(socket_fd, posName, queriedPos);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 查询全局路点失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 查询全局路点成功，位置为:";
for (const auto& pos : queriedPos) {
    std::cout << " " << pos;
}
std::cout << std::endl;
```

6.14.3 set_global_string - 设置全局字符串变量

方法：Result set_global_string(SOCKETFD socketFd, const std::string& varName, const std::string& varValue);

功能描述：设置全局字符串变量的值。

参数：

- socketFd：已连接的句柄。
- varName：变量名，如 "GS001"。
- varValue：字符串变量值。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置全局字符串变量
auto ret = tl::set_global_string(socket_fd, "GS001", "hello");
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置全局字符串变量失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置全局字符串变量成功" << std::endl;
```

6.14.4 get_global_string - 获取全局字符串变量

方法：Result get_global_string(SOCKETFD socketFd, const std::string& varName, std::string& value);

功能描述：查询全局字符串变量的值。

参数：

- socketFd：已连接的句柄。
- varName：变量名，如 "GS001"。
- value：输出，变量值。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取全局字符串变量
std::string value;
auto ret = tl::get_global_string(socket_fd, "GS001", value);
```

```
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取全局字符串变量失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 全局字符串变量 GS001 = " << value << std::endl;
```

6.14.5 get_global_sync_position - 获取全局同步位置

方法：Result get_global_sync_position(SOCKETFD socketFd, const std::string& posName, std::vector< double >& pos);

功能描述：查询全局 GE 点位（含外部轴）。

参数：

- socketFd：已连接的句柄。
- posName：全局位置名，如 "GE0001"。
- pos：输出，全局点位数组，长度 21；前 7 位为点位的坐标、姿态等信息，中间 7 位为机器人位置，后 7 位为外部轴位置。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取全局同步位置（GE 点）
std::vector<double> pos;
auto ret = tl::get_global_sync_position(socket_fd, "GE0001", pos);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取全局同步位置失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] GE0001 位置（共 " << pos.size() << " 位）:" << std::endl;
for (size_t i = 0; i < pos.size(); ++i) {
    std::cout << "  [" << i << "] " << pos[i] << std::endl;
}
```

6.14.6 get_global_variant - 获取全局变量

方法：Result get_global_variant(SOCKETFD socketFd, const std::string& varName, double& value);

功能描述：查询全局变量的值。

参数：

- socketFd：已连接的句柄。
- varName：全局变量名，支持形式如 "GI001" / "GD001" / "GB001"。
- value：输出，变量值。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取全局变量
double value = 0.0;
auto ret = tl::get_global_variant(socket_fd, "GD001", value);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取全局变量失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 全局变量 GD001 = " << value << std::endl;
```

6.15 模式与 IO 控制接口

头文件：include/tl_interface.h、include/tl_io.h 相关示例代码：examples\cpp\ex_digital_io.cpp、examples\cpp\ex_move_control.cpp

6.15.1 set_current_mode - 设置机器人模式（切换示教模式和运动模式）

方法：Result set_current_mode(SOCKETFD socketFd, RobotMode mode);

功能描述：切换控制器运行模式：示教模式（手动）、远程模式、运行模式（自动运行作业）。推荐使用 RobotMode 枚举重载版本，int 旧版已废弃。切换为运行模式后将自动进入伺服上电状态，无须额外上电。

参数：

- socketFd：已连接的句柄。
- mode：目标模式，RobotMode 枚举：
  - RobotMode::TEACH：示教模式
  - RobotMode::REMOTE：远程模式

- RobotMode::RUN：运行模式

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 切换为示教模式
auto ret = tl::set_current_mode(socket_fd, tl::RobotMode::TEACH);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 切换示教模式失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 已切换为示教模式" << std::endl;
```

6.15.2 set_digital_output - 设置数字 IO 输出

方法：Result set_digital_output(SOCKETFD socketFd, int port, IoLevel value);

功能描述：设置指定数字输出端口的状态。推荐使用 IoLevel 枚举重载版本，int 旧版已废弃。

参数：

- socketFd：已连接的句柄。
- port：输出端口号，范围【1，最大端口数】。
- value：输出电平，IoLevel 枚举：
  - IoLevel::OFF：输出关闭（低电平）
  - IoLevel::ON：输出打开（高电平）

返回值：

- 0：成功。
- 非0：失败（如端口号越界）。

使用示例：

```
// 设置数字 IO 输出（端口 1 输出高电平）
int io_port = 1;
auto ret = tl::set_digital_output(socket_fd, io_port, tl::IoLevel::ON);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置数字 IO 输出失败，错误码: " << ret << std::endl;
```

```
        return -1;
    }
    std::cout << "[信息] 设置数字 IO 输出成功" << std::endl;
```

6.15.3 get_digital_output - 查询数字 IO 输出

方法：Result get_digital_output(SOCKETFD socketFd, std::vector< int >& out);

功能描述：一次获取所有数字输出端口的电平状态（通常 64 路）。

参数：

- socketFd：已连接的句柄。
- out：输出，存储各端口状态（0 或 1）的数组，长度 64。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询所有数字输出
    std::vector<int> output_values;
    auto ret = tl::get_digital_output(socket_fd, output_values);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 查询数字 IO 输出失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 查询数字 IO 输出成功，状态为:";
    for (const auto& state : output_values) {
        std::cout << " " << state;
    }
    std::cout << std::endl;
```

6.15.4 get_digital_input - 获取所有 IO 输入状态

方法：Result get_digital_input(SOCKETFD socketFd, std::vector< int >& in);

功能描述：一次获取所有数字输入端口的电平状态（通常 64 路）。

参数：

- socketFd：已连接的句柄。
- in：输出，存储各输入端口状态的数组，长度 64。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取所有数字输入状态
std::vector<int> input_values;
auto ret = tl::get_digital_input(socket_fd, input_values);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取数字 IO 输入失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 获取数字 IO 输入成功，状态为:";
for (const auto& state : input_values) {
    std::cout << " " << state;
}
std::cout << std::endl;
```

6.15.5 set_analog_output - 设置模拟量输出

方法：Result set_analog_output(SOCKETFD socketFd, int port, double value);

功能描述：设置指定模拟量输出端口的数值。

参数：

- socketFd：已连接的句柄。
- port：端口号。
- value：数值，范围 0≤value≤10。

返回值：

- 0：成功。
- 非0：失败（如数值越界）。

使用示例：

```
// 设置模拟量输出（端口 1，输出 5.0V）
int port = 1;
auto ret = tl::set_analog_output(socket_fd, port, 5.0);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置模拟量输出失败，错误码: " << ret << std::endl;
    return -1;
```

```
    }
    std::cout << "[信息] 设置模拟量输出成功" << std::endl;
```

6.15.6 get_analog_input - 获取模拟量输入

方法：Result get_analog_input(SOCKETFD socketFd, std::vector< double >& ain);

功能描述：一次获取所有模拟量输入端口的数值。

参数：

- socketFd：已连接的句柄。
- ain：输出，模拟量输入数组，最大长度 64。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取所有模拟量输入
    std::vector<double> analog_input;
    auto ret = tl::get_analog_input(socket_fd, analog_input);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取模拟量输入失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 获取模拟量输入成功，共 " << analog_input.size() << " 路" << std::end
l;
```

6.15.7 get_analog_output - 获取模拟量输出

方法：Result get_analog_output(SOCKETFD socketFd, std::vector< double >& aout);

功能描述：一次获取所有模拟量输出端口的数值。

参数：

- socketFd：已连接的句柄。
- aout：输出，模拟量输出数组，最大长度 64。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
    // 获取所有模拟量输出
    std::vector<double> analog_output;
    auto ret = tl::get_analog_output(socket_fd, analog_output);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取模拟量输出失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 获取模拟量输出成功，共 " << analog_output.size() << " 路" << std::en
dl;
```

6.15.8 get_io_type - 获取 IO 类型

方法：Result get_io_type(SOCKETFD socketFd, IOtype& io_type);

功能描述：获取 IO 型号和端口数量信息。

参数：

- socketFd：已连接的句柄。
- io_type：输出，IOtype 结构体：
    - num：IO 板数量
    - type：IO 型号列表
    - io_port_sum：IO 端口数量，二维数组，每块 IO 板为 [数字输入端口数, 数字输出端口数, 模拟输入端口数, 模拟输出端口数]

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取 IO 类型信息
    tl::IOtype io_type;
    auto ret = tl::get_io_type(socket_fd, io_type);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取 IO 类型失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] IO 板数量: " << io_type.num << std::endl;
```

```
    for (size_t i = 0; i < io_type.type.size(); ++i) {
        std::cout << "  IO 板 " << i << " 型号: " << io_type.type[i] << std::endl;
    }
```

6.15.9 get_IO_reset_function - 获取 IO 复位功能

方法：Result get_IO_reset_function(SOCKETFD socketFd, int robotNum, IoResetType type, std::vector< int >& enable, std::vector< int >& value);

功能描述：获取 IO 复位相关参数。推荐使用 IoResetType 枚举重载版本，int 旧版已废弃。

参数：

- socketFd：已连接的句柄。
- robotNum：机器人编号（1-4）。
- type：复位类型，IoResetType 枚举：
    - IoResetType::REMOTE_IO：远程 IO 复位
    - IoResetType::MODE_STOP：切模式停止
    - IoResetType::PROGRAM_ERROR：程序报错
- enable：输出，是否复位容器，大小为所有 IO 板输出端口数。
- value：输出，复位值容器，大小为所有 IO 板输出端口数。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 获取远程 IO 复位参数
    std::vector<int> enable;
    std::vector<int> value;
    auto ret = tl::get_IO_reset_function(socket_fd, 1, tl::IoResetType::REMOTE_IO, enabl
e, value);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取 IO 复位功能失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 获取 IO 复位功能成功，端口数: " << enable.size() << std::endl;
```

6.16 算法接口

头文件：include/tl_interface.h 相关示例代码：examples\cpp\ex_coord_transform.cpp

6.16.1 get_origin_coord_to_target_coord - 坐标转换算法

方法：Result get_origin_coord_to_target_coord(SOCKETFD socketFd, Coord originCoord, std::vector< double > originPos, Coord targetCoord, std::vector< double >& targetPos, bool& convert_state, int form = 0, const std::vector< double >& reference_pos = {});

功能描述：在不同坐标系之间进行空间坐标转换。从关节坐标系转换为直角坐标系时为正运动学解算，从直角坐标系转换为关节坐标系时为逆运动学解算。接口仅做纯数学数值运算，不会驱动机械臂运动、不会触发伺服下电；逆解失败（奇异点位姿、目标位姿超出可达空间）时返回 EXCEPTION（-5）。推荐使用 Coord 枚举重载版本，int 旧版已废弃。接口依赖机械臂通信链路，建立连接后才可调用。工具坐标系（TOOL）转换由控制器端实现（语义与控制器端一致，SDK 透传）。

> ⚠ **废弃提示**：带 `convert_state` 出参的重载已废弃——该出参冗余，逆解结果已反映在返回值。请使用无 `convert_state` 的枚举重载。

参数：

- socketFd：已连接的句柄。
- originCoord：原坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）。
- originPos：要进行转换的坐标值 [X, Y, Z, A, B, C]：
  - 关节坐标：0-6 轴，范围 [-10000, 10000]（度）
  - 基坐标：0-2 轴范围 [-10000, 10000]（mm），3-6 轴范围 [-3.1416, 3.1416]（rad）
  - 工具坐标：同基坐标
  - 用户坐标：同基坐标
- targetCoord：目标坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）。
- targetPos：输出，转换后的坐标值（点位数组）。
- convert_state：输出，true 逆解成功 / false 逆解失败。
- form：形态（默认 0，与控制器协议对齐）。
- reference_pos：参考点（默认空）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 正解：关节坐标 → 基坐标
    std::vector<double> joint_pos = {10, 20, 30, 0, 0, 0}; // 关节角度 [°]
    std::vector<double> cartesian_pos;
    bool convert_state = false;
    auto ret = tl::get_origin_coord_to_target_coord(
        socket_fd, tl::Coord::JOINT, joint_pos, tl::Coord::BASE, cartesian_pos, convert_
state);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 坐标转换失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 正解成功: 位移 X=" << cartesian_pos[0]
              << " Y=" << cartesian_pos[1] << " Z=" << cartesian_pos[2] << " (mm)" << st
d::endl;

    // 逆解：基坐标 → 关节坐标
    std::vector<double> joint_back;
    ret = tl::get_origin_coord_to_target_coord(
        socket_fd, tl::Coord::BASE, cartesian_pos, tl::Coord::JOINT, joint_back, convert
_state);
    if (ret != tl::Result::SUCCESS || !convert_state) {
        std::cerr << "[ERROR] 逆解失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 逆解成功，关节角度:";
    for (const auto& j : joint_back) {
        std::cout << " " << j;
    }
    std::cout << std::endl;
```

6.16.2 get_quat2rpy - 四元数 转 欧拉角

方法：Result get_quat2rpy(SOCKETFD socketFd, std::vector< double > quat_vector, std::vector< double >& rpy_res);

功能描述：将四元数转换为欧拉角，接口依赖机械臂通信链路，建立连接后才可调用。

参数：

- socketFd：已连接的句柄。
- quat_vector：输入，四元数向量，4 元素 [w, x, y, z]（w 在首位；须为单位四元数）。

- rpy_res：输出，欧拉角向量，3 元素 [rx, ry, rz]（弧度）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 四元数转欧拉角
std::vector<double> quat = {1.0, 0.0, 0.0, 0.0}; // [w, x, y, z]
std::vector<double> rpy;
auto ret = tl::get_quat2rpy(socket_fd, quat, rpy);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 四元数转欧拉角失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 欧拉角 [rad]: Rx=" << rpy[0]
          << " Ry=" << rpy[1] << " Rz=" << rpy[2] << std::endl;
```

6.16.3 get_rpy2quat - 欧拉角 转 四元数

方法：Result get_rpy2quat(SOCKETFD socketFd, std::vector< double > rpy_vector, std::vector< double >& quat_res);

功能描述：将欧拉角转换为四元数，接口依赖机械臂通信链路，建立连接后才可调用。

参数：

- socketFd：已连接的句柄。
- rpy_vector：输入，欧拉角向量，3 元素 [rx, ry, rz]（弧度）。
- quat_res：输出，四元数向量，4 元素 [w, x, y, z]（w 在首位）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 欧拉角转四元数
std::vector<double> rpy = {0.2, 0.3, 0.5}; // rad
std::vector<double> quat;
auto ret = tl::get_rpy2quat(socket_fd, rpy, quat);
if (ret != tl::Result::SUCCESS) {
```

```
        std::cerr << "[ERROR] 欧拉角转四元数失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 四元数 [w, x, y, z]: "
              << quat[0] << ", " << quat[1] << ", " << quat[2] << ", " << quat[3] <<
std::endl;
```

6.16.4 get_rpy2r - 欧拉角 转 旋转矩阵

方法：Result get_rpy2r(SOCKETFD socketFd, std::vector< double > rpy_vector, std::vector< double >& r_res);

功能描述：将欧拉角转换为旋转矩阵，接口依赖机械臂通信链路，建立连接后才可调用。

参数：

- socketFd：已连接的句柄。
- rpy_vector：输入，欧拉角向量，3 元素 [rx, ry, rz]（弧度）。
- r_res：输出，旋转矩阵 9 元素，行主序。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 欧拉角转旋转矩阵
    std::vector<double> rpy = {0.0, 0.0, M_PI_2}; // rad
    std::vector<double> rot_mat;
    auto ret = tl::get_rpy2r(socket_fd, rpy, rot_mat);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 欧拉角转旋转矩阵失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 旋转矩阵（3x3，行主序）:" << std::endl;
    for (int i = 0; i < 3; ++i) {
        std::cout << "  " << rot_mat[i * 3] << " " << rot_mat[i * 3 + 1] << " " << rot_m
at[i * 3 + 2] << std::endl;
    }
```

6.16.5 get_tr2r - 位姿 转 旋转矩阵

方法：Result get_tr2r(SOCKETFD socketFd, std::vector< double > tr_matrix, std::vector< double >& r_res);

功能描述：将位姿矩阵（4x4 齐次变换矩阵）转换为旋转矩阵（3x3），提取旋转部分，平移部分丢弃，接口依赖机械臂通信链路，建立连接后才可调用。

参数：

- socketFd：已连接的句柄。
- tr_matrix：输入，位姿矩阵，vector 长度 16（行主序，R 部分无量纲，T 部分单位 mm）。
- r_res：输出，旋转矩阵，vector 长度 9（行主序）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 位姿矩阵转旋转矩阵（构造绕 Z 转 90°、平移 (100, 200, 300) mm 的位姿矩阵）
    double cos90 = cos(M_PI_2), sin90 = sin(M_PI_2);
    std::vector<double> tr_in = {
        cos90, -sin90, 0.0, 100.0,
        sin90,  cos90, 0.0, 200.0,
        0.0,    0.0,   1.0, 300.0,
        0.0,    0.0,   0.0, 1.0
    };
    std::vector<double> R;
    auto ret = tl::get_tr2r(socket_fd, tr_in, R);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 位姿转旋转矩阵失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 旋转矩阵（3x3，行主序）:" << std::endl;
    for (int i = 0; i < 3; ++i) {
        std::cout << "  " << R[i * 3] << " " << R[i * 3 + 1] << " " << R[i * 3 + 2] << s
td::endl;
    }
```

6.16.6 get_r2tr - 旋转矩阵 转 位姿矩阵

方法：Result get_r2tr(SOCKETFD socketFd, std::vector< double > r_matrix, std::vector< double >& tr_res);

功能描述：将旋转矩阵（3x3）转换为位姿矩阵（4x4 齐次变换矩阵），平移部分输出为 0。接口依赖机械臂通信链路，建立连接后才可调用。

参数：

- socketFd：已连接的句柄。
- r_matrix：输入，旋转矩阵，vector 长度 9（行主序）。
- tr_res：输出，位姿矩阵，vector 长度 16（行主序，R 部分无量纲，T 部分为 0）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 旋转矩阵转位姿矩阵（R 为前面 get_tr2r 的输出）
std::vector<double> tr_out;
auto ret = tl::get_r2tr(socket_fd, R, tr_out);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 旋转矩阵转位姿矩阵失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 位姿矩阵（4x4，行主序，平移部分为 0）:" << std::endl;
for (int i = 0; i < 4; ++i) {
    std::cout << "  " << tr_out[i * 4] << " " << tr_out[i * 4 + 1] << " "
              << tr_out[i * 4 + 2] << " " << tr_out[i * 4 + 3] << std::endl;
}
```

6.17 modbus通信接口

头文件：include/tl_modbus.h 相关示例代码：examples\cpp\ex_inspire_claw.cpp

6.17.1 modbus_set_master_parameter - 设置 Modbus RTU 主站参数

方法：Result modbus_set_master_parameter(SOCKETFD socketFd, int id, const ModbusMasterParameter& param);

功能描述：配置 Modbus RTU 或 TCP 主站参数。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。

- param：主站参数结构体（ModbusMasterParameter）：
  - type：主站类型，支持 "TCP" / "RTU"
  - startAddress：false 起始地址为 1；true 起始地址为 0
  - TCP：Modbus TCP 参数（type="TCP" 时使用，含 IP、port、endian_type）
  - RTU：Modbus RTU 参数（type="RTU" 时使用）：
    - slaveId：从站号
    - port：端口
    - baudrate：波特率
    - checkBit：奇偶校验位，"None" / "Even" / "Odd"
    - dataBit：数据位，5/6/7/8
    - stopBit：停止位，1/2

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 配置 RTU 主站（配方 id=1）
int id = 1;
tl::ModbusMasterParameter param;
param.type          = "RTU";
param.startAddress  = true;   // 起始地址 0
param.RTU.slaveId   = 1;      // 从站号
param.RTU.port      = 1;      // 端口
param.RTU.baudrate  = 115200; // 波特率
param.RTU.checkBit  = "None"; // 奇偶校验位
param.RTU.dataBit   = 8;      // 数据位
param.RTU.stopBit   = 1;      // 停止位

auto ret = tl::modbus_set_master_parameter(socket_fd, id, param);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置 Modbus 主站参数失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置 Modbus 主站参数成功" << std::endl;
```

6.17.2 modbus_open_master - 打开 Modbus 主站

方法：Result modbus_open_master(SOCKETFD socketFd, int id);

功能描述：打开 Modbus 主站（需与 modbus_set_master_parameter 设置的 id 对应）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id，需与 modbus_set_master_parameter 设置的 id 对应。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 打开 Modbus 主站（配方 id=1）
int id = 1;
auto ret = tl::modbus_open_master(socket_fd, id);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 打开 Modbus 主站失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 打开 Modbus 主站成功" << std::endl;
```

6.17.3 modbus_write_multiple_holding_registers - 写多个保持寄存器

方法：Result modbus_write_multiple_holding_registers(SOCKETFD socketFd, int id, int address, const std::vector< int >& data);

功能描述：写多个保持寄存器（功能码 10H）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。
- address：起始寄存器地址。
- data：写入的寄存器数据。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
// 写多个保持寄存器（从地址 0 开始写 3 个寄存器）
int id = 1;
std::vector<int> data = {100, 200, 300};
auto ret = tl::modbus_write_multiple_holding_registers(socket_fd, id, 0, data);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 写多个保持寄存器失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 写多个保持寄存器成功" << std::endl;
```

6.17.4 modbus_read_holding_registers - Modbus RTU 读取多个保持寄存器

方法：Result modbus_read_holding_registers(SOCKETFD socketFd, int id, int address, int quantity, std::vector< int >& data);

功能描述：读取多个保持寄存器（功能码 03H）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。
- address：起始地址。
- quantity：读取数量。
- data：输出，读取数据（失败时输出向量被清空）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 读取保持寄存器（从地址 0 读 3 个）
int id = 1;
std::vector<int> data;
auto ret = tl::modbus_read_holding_registers(socket_fd, id, 0, 3, data);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 读取保持寄存器失败，错误码: " << ret << std::endl;
    return -1;
}
```

```
    std::cout << "[信息] 读取保持寄存器成功:";
    for (const auto& v : data) {
        std::cout << " " << v;
    }
    std::cout << std::endl;
```

6.17.5 modbus_write_signal_coil_status - Modbus RTU协议写线圈

方法：Result modbus_write_signal_coil_status(SOCKETFD socketFd, int id, int address, int data);

功能描述：写单个线圈状态（功能码 05H）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。
- address：线圈地址。
- data：写入的线圈状态值。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 写单个线圈（地址 0，置 1）
    int id = 1;
    auto ret = tl::modbus_write_signal_coil_status(socket_fd, id, 0, 1);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 写线圈状态失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 写线圈状态成功" << std::endl;
```

6.17.6 modbus_write_signal_holding_registers - Modbus RTU协议写保持寄存器

方法：Result modbus_write_signal_holding_registers(SOCKETFD socketFd, int id, int address, int data);

功能描述：写单个保持寄存器（功能码 06H）。

参数：

- socketFd：已连接的句柄。

- id：配方 id 参数，最多保存 9 个 id。
- address：寄存器地址。
- data：写入的寄存器数据。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 写单个保持寄存器（地址 10，写入 1000）
int id = 1;
auto ret = tl::modbus_write_signal_holding_registers(socket_fd, id, 10, 1000);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 写保持寄存器失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 写保持寄存器成功" << std::endl;
```

6.17.7 modbus_read_coil_status - Modbus读线圈状态

方法：Result modbus_read_coil_status(SOCKETFD socketFd, int id, int address, int quantity, std::vector< int >& data);

功能描述：读取线圈状态（功能码 01H）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。
- address：起始地址。
- quantity：读取数量。
- data：输出，读取数据（失败时输出向量被清空）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 读取线圈状态（从地址 0 读 8 个）
int id = 1;
std::vector<int> data;
auto ret = tl::modbus_read_coil_status(socket_fd, id, 0, 8, data);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 读取线圈状态失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 读取线圈状态成功:";
for (const auto& v : data) {
    std::cout << " " << v;
}
std::cout << std::endl;
```

6.17.8 modbus_read_input_status - Modbus读取输入状态

方法：Result modbus_read_input_status(SOCKETFD socketFd, int id, int address, int quantity, std::vector< int >& data);

功能描述：读取输入状态（功能码 02H）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。
- address：起始地址。
- quantity：读取数量。
- data：输出，读取数据（失败时输出向量被清空）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 读取输入状态（从地址 0 读 8 个）
int id = 1;
std::vector<int> data;
auto ret = tl::modbus_read_input_status(socket_fd, id, 0, 8, data);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 读取输入状态失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 读取输入状态成功:";
```

```
for (const auto& v : data) {
    std::cout << " " << v;
}
std::cout << std::endl;
```

6.17.9 modbus_read_input_registers - Modbus 读输入寄存器

方法：Result modbus_read_input_registers(SOCKETFD socketFd, int id, int address, int quantity, std::vector< int >& data);

功能描述：读取输入寄存器（功能码 04H）。

参数：

- socketFd：已连接的句柄。
- id：配方 id 参数，最多保存 9 个 id。
- address：起始地址。
- quantity：读取数量。
- data：输出，读取数据（失败时输出向量被清空）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 读取输入寄存器（从地址 0 读 3 个）
int id = 1;
std::vector<int> data;
auto ret = tl::modbus_read_input_registers(socket_fd, id, 0, 3, data);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 读取输入寄存器失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 读取输入寄存器成功:";
for (const auto& v : data) {
    std::cout << " " << v;
}
std::cout << std::endl;
```

6.18 队列运动接口

头文件：include/tl_queue_operate.h 相关示例代码：examples\cpp\ex_queue_motion.cpp

6.18.1 queue_motion_set_status - 打开、关闭队列运动模式

方法：Result queue_motion_set_status(SOCKETFD socketFd, bool status);

功能描述：打开或关闭控制器的队列运动模式。无论打开还是关闭，都会清空远端控制器已存储的队列。打开队列模式（status=true）时控制器会先执行伺服下电再上电；关闭队列模式（status=false）退出后控制器会执行伺服下电，后续如需继续运动需重新执行上电流程。

参数：

- socketFd：已连接的句柄。
- status：true 打开 / false 关闭。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 打开队列运动模式
auto ret = tl::queue_motion_set_status(socket_fd, true);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 打开队列运动模式失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 队列运动模式已打开" << std::endl;
```

6.18.2 queue_motion_push_back_moveJ - 队列运动模式的本地队列最后插入一条move J运动

方法：Result queue_motion_push_back_moveJ(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：在队列运动模式的本地队列末尾插入一条 MoveJ（关节空间运动）指令。

参数：

- socketFd：已连接的句柄。

- moveCmd：运动命令结构体，需设置以下成员：
    - coord：坐标系，Coord 枚举（JOINT / BASE / TOOL / USER）
    - targetPosValue：目标点坐标，长度 7
    - velocity：速度，范围 (1, 100]，单位 度/s
    - acc：加速度，范围 (1, 100]
    - dec：减速度，范围 (1, 100]
    - pl：多条轨迹之间平滑参数，范围 [0, 5]

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 插入一条 MoveJ 指令
tl::MoveCmd cmd;
cmd.coord        = tl::Coord::JOINT;   // 关节坐标系
cmd.velocity     = 30;                 // 度/s
cmd.acc          = 50;
cmd.dec          = 50;
cmd.pl           = 0;                  // 平滑参数
cmd.targetPosValue.resize(14, 0.0);
cmd.targetPosValue[0] = 30;            // J1 = 30°
cmd.targetPosValue[1] = 20;            // J2 = 20°

auto ret = tl::queue_motion_push_back_moveJ(socket_fd, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 插入 MoveJ 指令失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 插入 MoveJ 指令成功" << std::endl;
```

6.18.3 queue_motion_push_back_moveL - 队列运动模式的本地队列最后插入一条 move L运动

方法：Result queue_motion_push_back_moveL(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：在队列运动模式的本地队列末尾插入一条 MoveL（笛卡尔直线运动）指令。

参数：

- socketFd：已连接的句柄。
- moveCmd：运动命令结构体，成员同 6.18.2，velocity 范围 (1, 1000]，单位 mm/s。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 插入一条 MoveL 指令
tl::MoveCmd cmd;
cmd.coord        = tl::Coord::BASE;    // 基坐标系
cmd.velocity     = 100;                // mm/s
cmd.acc          = 50;
cmd.dec          = 50;
cmd.pl           = 0;
cmd.targetPosValue.resize(14, 0.0);
cmd.targetPosValue[0] = 300;           // X (mm)
cmd.targetPosValue[1] = 0;             // Y (mm)
cmd.targetPosValue[2] = 300;           // Z (mm)
cmd.targetPosValue[3] = 3.141;         // Rx (rad)
cmd.targetPosValue[4] = 0;             // Ry (rad)
cmd.targetPosValue[5] = 0;             // Rz (rad)

auto ret = tl::queue_motion_push_back_moveL(socket_fd, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 插入 MoveL 指令失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 插入 MoveL 指令成功" << std::endl;
```

6.18.4 queue_motion_push_back_moveC - 队列运动本地队列末尾插入一条 MoveC 运动

方法：Result queue_motion_push_back_moveC(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：在队列运动模式的本地队列末尾插入一条 MoveC（圆弧运动）指令。

参数：

- socketFd：已连接的句柄。
- moveCmd：运动命令结构体，成员同 6.18.2，velocity 范围 (1, 1000]，单位 mm/s。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 插入一条 MoveC 指令
tl::MoveCmd cmd;
cmd.coord        = tl::Coord::BASE;    // 基坐标系
cmd.velocity     = 100;                // mm/s
cmd.acc          = 50;
cmd.dec          = 50;
cmd.pl           = 0;
cmd.targetPosValue.resize(14, 0.0);
cmd.targetPosValue[0] = 280;           // X (mm)
cmd.targetPosValue[1] = 0;             // Y (mm)
cmd.targetPosValue[2] = 380;           // Z (mm)
cmd.targetPosValue[3] = 3.141;         // Rx (rad)
cmd.targetPosValue[4] = 0;
cmd.targetPosValue[5] = 0;

auto ret = tl::queue_motion_push_back_moveC(socket_fd, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 插入 MoveC 指令失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 插入 MoveC 指令成功" << std::endl;
```

6.18.5 queue_motion_push_back_moveS - 队列运动本地队列末尾插入一条 MoveS 运动

方法：Result queue_motion_push_back_moveS(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：在队列运动模式的本地队列末尾插入一条 MoveS（样条运动）指令。

参数：

- socketFd：已连接的句柄。
- moveCmd：运动命令结构体，成员同 6.18.2，velocity 范围 (1, 1000]，单位 mm/s。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 插入一条 MoveS 指令
tl::MoveCmd cmd;
cmd.coord        = tl::Coord::BASE;    // 基坐标系
cmd.velocity     = 100;                // mm/s
cmd.acc          = 50;
cmd.dec          = 50;
cmd.pl           = 0;
cmd.targetPosValue.resize(14, 0.0);
cmd.targetPosValue[0] = 300;           // X (mm)
cmd.targetPosValue[1] = 100;           // Y (mm)
cmd.targetPosValue[2] = 300;           // Z (mm)
cmd.targetPosValue[3] = 3.141;         // Rx (rad)

auto ret = tl::queue_motion_push_back_moveS(socket_fd, cmd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 插入 MoveS 指令失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 插入 MoveS 指令成功" << std::endl;
```

6.18.6 queue_motion_send_to_controller - 将本地队列的前size个数据发送到控制器

方法：Result queue_motion_send_to_controller(SOCKETFD socketFd, int size, bool isContinue = false);

功能描述：将本地队列的前 size 个数据发送到控制器。控制器接收到队列后会立即开始运动，调用前请先调用 queue_motion_set_status(true)。

参数：

- socketFd：已连接的句柄。
- size：要发送的队列大小，范围 [0, 31]；size = 0 时将当前运动队列全部指令发送。
- isContinue：是否继续发送，true 继续发送（可继续发送点位，机器人不运动）；false 不继续（机器人接收点位后立刻运动），默认 false。

返回值：

- 0：成功。
- 非0：失败（如传入的 size 超过队列长度）。

使用示例：

```
// 将本地队列全部指令发送到控制器
auto ret = tl::queue_motion_send_to_controller(socket_fd, 0, false);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 发送队列到控制器失败，错误码: " << ret << std::endl;
```

```
        return -1;
    }
    std::cout << "[信息] 发送队列到控制器成功" << std::endl;
```

6.18.7 queue_motion_get_status - 查询队列模式状态

方法：Result queue_motion_get_status(SOCKETFD socketFd, bool& status);

功能描述：查询控制器当前是否已打开队列运动模式。

参数：

- socketFd：已连接的句柄。
- status：输出，true 已打开 / false 关闭。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询队列模式状态
    bool status = false;
    auto ret = tl::queue_motion_get_status(socket_fd, status);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 查询队列模式状态失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 队列模式状态: " << (status ? "已打开" : "已关闭") << std::endl;
```

6.18.8 queue_motion_size - 查询当前运动队列的长度

方法：Result queue_motion_size(SOCKETFD socketFd, int& size);

功能描述：查询当前本地运动队列的长度。

参数：

- socketFd：已连接的句柄。
- size：输出，当前运动队列长度。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
// 查询本地队列长度
int size = 0;
auto ret = tl::queue_motion_size(socket_fd, size);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 查询队列长度失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 本地队列长度: " << size << " 条指令" << std::endl;
```

6.18.9 queue_motion_get_queuelen - 查询当前运动队列剩余的指令数量

方法：Result queue_motion_get_queuelen(SOCKETFD socketFd, int& len);

功能描述：查询控制器当前运动队列剩余的指令数量。

参数：

- socketFd：已连接的句柄。
- len：输出，当前运动队列剩余的指令数量。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 查询剩余队列指令数
int len = 0;
auto ret = tl::queue_motion_get_queuelen(socket_fd, len);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 查询剩余队列指令数失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 剩余队列指令数: " << len << std::endl;
```

6.18.10 queue_motion_suspend - 暂停连续运动（轨迹暂停）

方法：Result queue_motion_suspend(SOCKETFD socketFd);

功能描述：暂停连续运动（轨迹暂停）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 暂停连续运动
auto ret = tl::queue_motion_suspend(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 暂停连续运动失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 已暂停连续运动" << std::endl;
```

6.18.11 queue_motion_restart - 暂停后再次运行运动队列（继续当前轨迹运动）

方法：Result queue_motion_restart(SOCKETFD socketFd);

功能描述：暂停后再次运行运动队列（继续当前轨迹运动）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 恢复连续运动
auto ret = tl::queue_motion_restart(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 恢复连续运动失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 已恢复连续运动" << std::endl;
```

6.18.12 queue_motion_stop - 停止队列运动

方法：Result queue_motion_stop(SOCKETFD socketFd);

功能描述：停止连续运动。**该接口已废弃**，建议使用 queue_motion_stop_not_power_off 替代。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 停止队列运动（已废弃，推荐使用 queue_motion_stop_not_power_off）
auto ret = tl::queue_motion_stop(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 停止队列运动失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 已停止队列运动" << std::endl;
```

6.18.13 queue_motion_stop_not_power_off - 停止连续运动保持上电状态（轨迹缓停）

方法：Result queue_motion_stop_not_power_off(SOCKETFD socketFd);

功能描述：停止连续运动并保持上电状态（轨迹缓停）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 停止连续运动（保持上电）
auto ret = tl::queue_motion_stop_not_power_off(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 停止连续运动失败，错误码: " << ret << std::endl;
```

```
        return -1;
    }
    std::cout << "[信息] 已停止连续运动（保持上电）" << std::endl;
```

6.18.14 queue_motion_clear_Data - 清空本地队列数据

方法：Result queue_motion_clear_Data(SOCKETFD socketFd);

功能描述：清空缓存的运动队列数据。仅清除上位机侧缓存的队列数据，不清除已经下发给控制器的队列；控制器已接收的队列会继续执行，如需停止需调用 queue_motion_stop。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 清空本地队列数据
    auto ret = tl::queue_motion_clear_Data(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 清空本地队列数据失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 已清空本地队列数据" << std::endl;
```

6.19 拖拽示教与轨迹记录回放

头文件：include/tl_interface.h、include/tl_track.h 相关示例代码：examples\cpp\ex_track_queue_drag.cpp、examples\cpp\ex_six_axis_force_drag.cpp、examples\cpp\ex_teach_control.cpp

6.19.1 set_darg_mode - 设置拖拽示教的拖拽方式

方法：Result set_darg_mode(SOCKETFD socketFd, DragMode mode);

功能描述：配置机械臂拖拽示教执行模式，可选择力矩柔顺拖拽、位置锁定拖拽、3D 鼠标操控等不同拖动方式，适配现场不同示教操作需求。推荐使用 DragMode 枚举重载版本，int 旧版已废弃。需在切换拖拽类型（set_teach_type）之前设置。

参数：

- socketFd：已连接的句柄。
- mode：拖拽方式，DragMode 枚举：
  - DragMode::NONE：无（关闭拖拽）
  - DragMode::MOUSE_3D：3D 鼠标
  - DragMode::TORQUE：力矩模式
  - DragMode::POSITION：位置模式（22.07 版本无位置模式）

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置拖拽方式为 3D 鼠标
auto ret = tl::set_darg_mode(socket_fd, tl::DragMode::MOUSE_3D);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置拖拽模式失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置拖拽模式成功（MOUSE_3D）" << std::endl;
```

6.19.2 set_teach_type - 设置示教模式类型

方法：Result set_teach_type(SOCKETFD socketFd, TeachType type);

功能描述：设置示教模式类型。推荐使用 TeachType 枚举重载版本，int 旧版已废弃。

参数：

- socketFd：已连接的句柄。
- type：示教模式类型，TeachType 枚举：
  - TeachType::JOG：点动
  - TeachType::DRAG：拖拽

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
// 切换为拖拽示教类型
auto ret = tl::set_teach_type(socket_fd, tl::TeachType::DRAG);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置示教类型失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 已切换为拖拽示教类型" << std::endl;
```

6.19.3 get_teach_type - 查询示教模式类型

方法：Result get_teach_type(SOCKETFD socketFd, TeachType& type);

功能描述：查询当前的示教模式类型。推荐使用 TeachType 枚举重载版本，int 旧版已废弃。

参数：

- socketFd：已连接的句柄。
- type：输出，示教模式类型，TeachType 枚举（JOG / DRAG）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 查询示教模式类型
tl::TeachType type;
auto ret = tl::get_teach_type(socket_fd, type);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 查询示教模式类型失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 当前示教模式类型: " << static_cast<int>(type)
          << " (0=点动 1=拖拽)" << std::endl;
```

6.19.4 set_current_teach_sensitivity - 设置电流环拖动示教灵敏度

方法：Result set_current_teach_sensitivity(SOCKETFD socketFd, const std::vector< double >& sensitivity);

功能描述：设置电流环拖动示教灵敏度。

参数：

- socketFd：已连接的句柄。
- sensitivity：各关节电流环拖动示教灵敏度参数向量，按实际轴数排列（6 轴为 J1-J6，7 轴为 J1-J7），范围 [0, 3]，数值越小灵敏度越低。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 设置电流环拖动示教灵敏度（J1-J6 均为 1.5）
std::vector<double> sensitivity = {1.5, 1.5, 1.5, 1.5, 1.5, 1.5};
auto ret = tl::set_current_teach_sensitivity(socket_fd, sensitivity);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 设置电流环拖动示教灵敏度失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 设置电流环拖动示教灵敏度成功" << std::endl;
```

6.19.5 get_current_teach_sensitivity - 获取电流环拖动示教灵敏度

方法：Result get_current_teach_sensitivity(SOCKETFD socketFd, std::vector< double >& sensitivity);

功能描述：获取各关节电流环拖动示教灵敏度参数。

参数：

- socketFd：已连接的句柄。
- sensitivity：输出，各关节电流环拖动示教灵敏度参数向量，按实际轴数排列。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 获取电流环拖动示教灵敏度
std::vector<double> sensitivity;
auto ret = tl::get_current_teach_sensitivity(socket_fd, sensitivity);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取电流环拖动示教灵敏度失败，错误码: " << ret << std::endl;
```

```
        return -1;
    }
    std::cout << "[信息] 各关节灵敏度:";
    for (const auto& s : sensitivity) {
        std::cout << " " << s;
    }
    std::cout << std::endl;
```

6.19.6 get_drag_thread_is_end - 查询拖拽是否结束

方法：Result get_drag_thread_is_end(SOCKETFD socketFd, bool& endFlag);

功能描述：查询拖拽是否已结束。

参数：

- socketFd：已连接的句柄。
- endFlag：输出，true 拖拽结束 / false 拖拽中。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
    // 查询拖拽是否结束
    bool drag_finished = false;
    auto ret = tl::get_drag_thread_is_end(socket_fd, drag_finished);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 查询拖拽结束标志失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 拖拽是否结束: " << (drag_finished ? "是" : "否") << std::endl;
```

6.19.7 track_record_start - 轨迹记录开始

方法：Result track_record_start(SOCKETFD socketFd, double maxSamplingNum, double samplingInterval);

功能描述：开始记录拖动示教轨迹（需先进入拖拽示教模式）。

参数：

- socketFd：已连接的句柄。
- maxSamplingNum：最大采样点数，范围 [200, 12000]。

- samplingInterval：采样间隔，范围 [0.03, 1]。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 开始轨迹记录
double max_sampling_num = 10000;   // 最大采样点数
double sampling_interval = 1;      // 采样间隔 (s)
auto ret = tl::track_record_start(socket_fd, max_sampling_num, sampling_interval);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 轨迹记录开始失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 轨迹记录已开始" << std::endl;
```

6.19.8 get_track_record_status - 轨迹记录开启状态查询

方法：Result get_track_record_status(SOCKETFD socketFd, bool& recordStart);

功能描述：查询轨迹记录是否处于开启状态。

参数：

- socketFd：已连接的句柄。
- recordStart：输出，true 开启 / false 关闭。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 查询轨迹记录状态
bool record_status = false;
auto ret = tl::get_track_record_status(socket_fd, record_status);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 查询轨迹记录状态失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 轨迹记录状态: " << (record_status ? "开启" : "关闭") << std::endl;
```

6.19.9 track_record_stop - 轨迹记录关闭

方法：Result track_record_stop(SOCKETFD socketFd);

功能描述：关闭轨迹记录。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 关闭轨迹记录
auto ret = tl::track_record_stop(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 轨迹记录关闭失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 轨迹记录已关闭" << std::endl;
```

6.19.10 track_record_save - 保存拖动示教轨迹

方法：Result track_record_save(SOCKETFD socketFd, std::string trajName);

功能描述：将当前录制的轨迹保存到控制器。

参数：

- socketFd：已连接的句柄。
- trajName：保存的轨迹名称。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 保存轨迹
std::string traj_name = "traj_01";
auto ret = tl::track_record_save(socket_fd, traj_name);
```

```
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 轨迹记录保存失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 轨迹记录已保存（trajName = " << traj_name << "）" << std::endl;
```

6.19.11 track_record_playback - 轨迹回放

方法：Result track_record_playback(SOCKETFD socketFd, int vel);

功能描述：回放已保存的拖动示教轨迹（回放前建议先退出拖拽模式、切回点动）。

参数：

- socketFd：已连接的句柄。
- vel：回放速度。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 轨迹回放
int vel = 100;  // 回放速度
auto ret = tl::track_record_playback(socket_fd, vel);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 轨迹回放失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 轨迹回放开始（vel = " << vel << "）" << std::endl;
```

6.19.12 track_record_delete - 轨迹清除

方法：Result track_record_delete(SOCKETFD socketFd);

功能描述：清除控制器内缓存的当前录制轨迹数据，释放轨迹占用存储空间。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
// 轨迹清除
auto ret = tl::track_record_delete(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 轨迹清除失败，错误码： " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 轨迹清除成功" << std::endl;
```

6.19.13 drag_trajectory_export - 导出拖动轨迹

方法：Result drag_trajectory_export(SOCKETFD socketFd, const std::string& directoryPath);

功能描述：将控制器 `dragPos` 目录中的全部拖动轨迹（`.json` 文件）导出到本地目录，用于备份或迁移。

参数：

- socketFd：已连接的句柄。
- directoryPath：本地保存目录；末级目录不存在时自动创建，父目录须存在。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 导出控制器中的全部拖动轨迹
auto ret = tl::drag_trajectory_export(socket_fd, "/home/user/drag_traj");
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 拖动轨迹导出失败，错误码： " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 拖动轨迹已导出" << std::endl;
```

6.19.14 drag_trajectory_import - 导入拖动轨迹

方法：Result drag_trajectory_import(SOCKETFD socketFd, const std::string& directoryPath);

功能描述：将本地目录中的全部 `.json` 拖动轨迹导入控制器 `dragPos` 目录。

参数：

- socketFd：已连接的句柄。
- directoryPath：包含 `.json` 轨迹文件的本地目录。

返回值：

- 0：成功。
- 非0：失败。

注意：控制器中存在同名轨迹时将被覆盖。

使用示例：

```
// 导入本地目录中的全部拖动轨迹
auto ret = tl::drag_trajectory_import(socket_fd, "/home/user/drag_traj");
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 拖动轨迹导入失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 拖动轨迹已导入" << std::endl;
```

6.19.15 drag_trajectory_playback - 回放指定拖动轨迹

方法：Result drag_trajectory_playback(SOCKETFD socketFd, const std::string& trajectoryName, int velocity);

功能描述：回放控制器 `dragPos` 目录中指定的拖动轨迹文件。回放前建议先退出拖拽模式。

参数：

- socketFd：已连接的句柄。
- trajectoryName：控制器 `dragPos` 目录中的轨迹文件名，包含 `.json` 后缀。
- velocity：回放速度，范围 [1,100]。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 以 50% 速度回放指定的拖动轨迹
auto ret = tl::drag_trajectory_playback(socket_fd, "traj_001.json", 50);
```

```
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 拖动轨迹回放失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 拖动轨迹回放已开始" << std::endl;
```

6.19.16 drag_trajectory_delete - 删除指定拖动轨迹

方法：Result drag_trajectory_delete(SOCKETFD socketFd, const std::string& trajectoryName);

功能描述：删除控制器 `dragPos` 目录中指定的拖动轨迹文件。

参数：

- socketFd：已连接的句柄。
- trajectoryName：控制器 `dragPos` 目录中的轨迹文件名，包含 `.json` 后缀。

返回值：

- 0：成功。
- 非0：失败。

注意：删除后不可恢复。

使用示例：

```
    // 删除指定的拖动轨迹
    auto ret = tl::drag_trajectory_delete(socket_fd, "traj_001.json");
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 拖动轨迹删除失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 拖动轨迹已删除" << std::endl;
```

七．7000端口接口

7.1 以下功能在连接7000端口后使用

头文件：include/tl_servo_ext.h 相关示例代码：examples\cpp\ex_servoj.cpp

7.1.1 open_servoJ - 打开关节跟踪模式

方法：Result open_servoJ(SOCKETFD socketFd, std::vector< double > vmax, std::vector< double > amax, std::vector< double > jmax);

功能描述：打开关节跟踪模式（底层 servoJ 透传）。连接 7000 端口后调用本接口即可进入透传模式，之后可高频调用 set_servoJ_pos 实时下发关节角。

参数：

- socketFd：socket（7000 伺服端口）。
- vmax：关节最大速度约束（固定 7 元素，°/s）。
- amax：关节最大加速度约束（固定 7 元素，°/s²）。
- jmax：关节最大加加速度约束（固定 7 元素，°/s³）。

返回值：

- 0：成功。
- 非0：失败（任一约束向量长度 ≠ 7 时返回参数错误）。

使用示例：

```
    // 打开 servoJ 关节实时跟踪模式
    // 注意: SDK 底层协议要求 servoJ 相关向量固定为 7 元素,
    //       6 轴机器人在第 7 元素（索引 6）补 0，7 轴机器人为真实 J7 约束值
    std::vector<double> vmax = {30.0, 30.0, 30.0, 30.0, 30.0, 30.0, 30.0};  // °/s
    std::vector<double> amax = {60.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0};  // °/s²
    std::vector<double> jmax = {100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0}; // °/
s³

    auto ret = tl::open_servoJ(socket_fd, vmax, amax, jmax);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] open_servoJ 失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 关节跟踪模式已打开" << std::endl;
```

7.1.2 close_servoJ - 关闭关节跟踪模式

方法：Result close_servoJ(SOCKETFD socketFd);

功能描述：关闭关节跟踪模式（底层 servoJ 透传），停止实时关节角下发。

参数：

- socketFd：socket（7000 伺服端口）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 关闭 servoJ 关节跟踪模式
auto ret = tl::close_servoJ(socket_fd);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[WARN] close_servoJ 返回: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 关节跟踪模式已关闭" << std::endl;
```

7.1.3 set_servoJ_pos - 设置实时跟踪关节位置

方法：Result set_servoJ_pos(SOCKETFD socketFd, std::vector< double > q);

功能描述：实时发送目标关节角到控制器（透传模式，10ms 周期高频调用）。控制器直接响应，实现平滑实时跟踪。

参数：

- socketFd：socket（7000 伺服端口）。
- q：目标关节角，7 元素向量（度）；6 轴机器人在第 7 元素（索引 6）补 0（外部轴），7 轴机器人第 7 元素为真实 J7 关节角。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
// 高频发送关节角（J1 从 0° 平滑转到 30°）
std::vector<double> q = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
for (int i = 0; i < 200; ++i) {
    q[0] += (30.0 / 200.0);       // 每次增加 0.15°
    auto ret = tl::set_servoJ_pos(socket_fd, q);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] set_servoJ_pos 失败，错误码: " << ret << std::endl;
```

```
            return -1;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(10)); // 100Hz
    }
    std::cout << "[信息] J1=30° 到位" << std::endl;
```

7.1.4 open_servo_csv_mode - 打开周期同步速度模式

方法：Result open_servo_csv_mode(SOCKETFD socketFd);

功能描述：打开周期同步速度模式（CSV），之后可按固定周期调用 set_servo_csv_velocity 下发关节目标速度。

参数：

- socketFd：socket（7000 伺服端口）。

返回值：

- 0：成功。
- 非0：失败。

注意：需控制器 24.03 及以上版本；需先连接 7000 端口，并与 close_servo_csv_mode 成对调用。

使用示例：

```
    // 打开周期同步速度模式
    auto ret = tl::open_servo_csv_mode(socket_udp);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] open_servo_csv_mode 失败，错误码: " << ret << std::endl;
        return -1;
    }
    std::cout << "[信息] 周期同步速度模式已打开" << std::endl;
```

7.1.5 close_servo_csv_mode - 关闭周期同步速度模式

方法：Result close_servo_csv_mode(SOCKETFD socketFd);

功能描述：关闭周期同步速度模式，停止速度下发。

参数：

- socketFd：socket（7000 伺服端口）。

返回值：

- 0：成功。

- 非0：失败。

使用示例：

```
auto ret = tl::close_servo_csv_mode(socket_udp);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[WARN] close_servo_csv_mode 返回: " << ret << std::endl;
}
```

7.1.6 set_servo_csv_velocity - 下发周期同步速度

方法：Result set_servo_csv_velocity(SOCKETFD socketFd, const std::vector< double >& targetVelocity);

功能描述：在周期同步速度模式下，按固定周期下发各关节目标速度。

参数：

- socketFd：socket（7000 伺服端口）。
- targetVelocity：各关节目标速度（°/s），固定 7 元素；6 轴机器人第 7 元素（索引 6）补 0（外部轴），7 轴机器人第 7 元素为真实 J7 速度。

返回值：

- 0：成功。
- 非0：失败（向量长度 ≠ 7 时返回参数错误）。

使用示例：

```
// 以 10°/s 正向旋转 J1，其余轴保持 0
std::vector<double> velocity = {10.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
auto ret = tl::set_servo_csv_velocity(socket_udp, velocity);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] set_servo_csv_velocity 失败，错误码: " << ret << std::endl;
    return -1;
}
```

7.1.7 open_servo_cst_mode - 打开周期同步扭矩模式

方法：Result open_servo_cst_mode(SOCKETFD socketFd);

功能描述：打开周期同步扭矩模式（CST），之后可按固定周期调用 set_servo_cst_torque 下发关节目标扭矩。

参数：

- socketFd：socket（7000 伺服端口）。

返回值：

- 0：成功。
- 非0：失败。

注意：需控制器 24.03 及以上版本；需先连接 7000 端口，并与 close_servo_cst_mode 成对调用。

使用示例：

```
// 打开周期同步扭矩模式
auto ret = tl::open_servo_cst_mode(socket_udp);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] open_servo_cst_mode 失败，错误码: " << ret << std::endl;
    return -1;
}
std::cout << "[信息] 周期同步扭矩模式已打开" << std::endl;
```

7.1.8 close_servo_cst_mode - 关闭周期同步扭矩模式

方法：Result close_servo_cst_mode(SOCKETFD socketFd);

功能描述：关闭周期同步扭矩模式，停止扭矩下发。

参数：

- socketFd：socket（7000 伺服端口）。

返回值：

- 0：成功。
- 非0：失败。

使用示例：

```
auto ret = tl::close_servo_cst_mode(socket_udp);
if (ret != tl::Result::SUCCESS) {
    std::cerr << "[WARN] close_servo_cst_mode 返回: " << ret << std::endl;
}
```

7.1.9 set_servo_cst_torque - 下发周期同步扭矩

方法：Result set_servo_cst_torque(SOCKETFD socketFd, const std::vector< double >& targetTorque);

功能描述：在周期同步扭矩模式下，按固定周期下发各关节目标扭矩。

参数：

- socketFd：socket（7000 伺服端口）。
- targetTorque：各关节目标扭矩，固定 7 元素；单位为额定扭矩的 0.1%，范围 [-1000,1000]；6 轴机器人第 7 元素（索引 6）补 0（外部轴），7 轴机器人第 7 元素为真实 J7 扭矩。

返回值：

- 0：成功。
- 非0：失败（向量长度 ≠ 7 时返回参数错误）。

使用示例：

```
    // 给 J1 施加 5% 额定扭矩（50 × 0.1%）
    std::vector<double> torque = {50.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
    auto ret = tl::set_servo_cst_torque(socket_udp, torque);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] set_servo_cst_torque 失败，错误码: " << ret << std::endl;
        return -1;
    }
```

8.1 Modbus主站通信综合示例

本节为 Modbus 主站通信综合示例（因时电爪 Inspire EG2-4C2，通过 RS485 串口以 Modbus RTU 协议控制），完整源码见 `examples/cpp/ex_inspire_claw.cpp`。完整代码如下：

```
/**
 * ex_inspire_claw.cpp
 * @brief 因时电爪（Inspire EG2-4C2，RS485 / Modbus RTU）控制测试示例
 * @attention
 *   - 确保控制器网络通信正常，夹爪通过 RS485 串口连接控制器
 *   - 主站参数（从站 ID、串口号、波特率等）请在下方"RTU 主站配置区"自行修改
 *   - 开度范围 0~1000：0=闭合完，1000=张开完；写目标开度后夹爪立即动作，
 *     需轮询实际开度寄存器等待到位
 *   - 夹爪会真实动作：默认测试序列首位置 0 为全闭合，运行前请确保夹爪空闲、
 *     指间无物体
 * @note 运行步骤
 *       编译: cd examples/cpp/build && cmake .. && make
 *       运行: ./ex_inspire_claw
 */
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <string>
```

```cpp
#include <thread>
#include <vector>

#include "tl_api.h"

#ifdef _WIN32
#include <windows.h>
#endif

// ============================================================
// 控制器连接配置（按实际修改）
// ============================================================
static std::string ROBOT_IP = "192.168.1.13";   // 机械臂控制器 IP（可用 -ip 参数覆盖）
const char* ROBOT_PORT = "6001";         // 控制器端口

// ============================================================
// RTU 主站配置区（请根据实际接线自行修改）
// ============================================================
constexpr int RTU_MASTER_ID = 1;         // 控制器侧主站编号
constexpr int RTU_SLAVE_ID = 1;          // 夹爪从站 ID
constexpr int RTU_PORT = 2;              // 控制器串口号（按接线修改）
constexpr int RTU_BAUDRATE = 115200;     // 波特率（EG2 默认 115200）
const std::string RTU_CHECK_BIT = "None"; // 校验位: "None"/"Even"/"Odd"
constexpr int RTU_DATA_BIT = 8;          // 数据位
constexpr int RTU_STOP_BIT = 1;          // 停止位

// ============================================================
// EG2-4C2 夹爪保持寄存器（控制器侧地址，EG2-4C2 固件实测）
// 注意: 通用手册（EG2-4XX V1.0.6）编号为 1010/1011/1012/1061/1064/1065,
//       本机 EG2-4C2 实测目标开度寄存器为地址 10（写 500 即张开到半开）,
//       其余寄存器按同段顺延，若读数异常请按夹爪实际固件调整
// 功能码: 03H 读 / 06H 写单个 / 10H 写多个，CRC16-REV=0xA001
// ============================================================
constexpr int REG_OPENLEN_SET = 10;      // 目标开度 0~1000（0=闭合完 1000=张开完），写后立即动作
constexpr int REG_SPEED_SET = 11;        // 速度 10~1000，写后不动作
constexpr int REG_FORCE_SET = 12;        // 抓力 100~1000，写后不动作
constexpr int REG_OPENLEN_ACT = 61;      // 实际开度 0~1000（只读）
constexpr int REG_ERRORCODE = 64;        // 错误码（只读，位0堵转 位1过温 位2过流 位3运行故障 位4内部通信故障）
constexpr int REG_STATUS = 65;           // 状态码（只读，1张开到位 2闭合到位 3停止 4夹取中 5张开中 6夹取到位）
// 其它只读寄存器按需扩展: 60 实际抓力 / 62 电流 / 63 温度

constexpr int CLAW_SPEED = 300;          // 夹爪动作速度
constexpr int CLAW_FORCE = 300;          // 抓力阈值
// 测试开度序列: 0=闭合完, 1000=张开完；首位置 0 为全闭合，运行前确保指间无物体
const std::vector<int> CLAW_TEST_POS = {0, 1000};

// ============================================================
// 工具函数
// ============================================================
```

```cpp
// @brief 打印操作结果
void print_result(const std::string& msg, tl::Result ret)
{
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  [成功] " << msg << std::endl;
    } else {
        std::cout << "  [失败] " << msg << ", 错误码: " << static_cast<int>(ret) << std::e
ndl;
    }
}

// @brief 打印分隔标题
void print_separator(const std::string& title)
{
    std::cout << "\n========== " << title << " ==========" << std::endl;
}

// @brief 带重试的读取：最多尝试 3 次，每次间隔 500ms
tl::Result read_with_retry(tl::SOCKETFD sock, int id, int addr, int qty,
                        std::vector<int>& data, const std::string& desc)
{
    tl::Result ret = tl::Result::SUCCESS;
    for (int attempt = 1; attempt <= 3; attempt++) {
        data.clear();
        ret = tl::modbus_read_holding_registers(sock, id, addr, qty, data);
        if (ret == tl::Result::SUCCESS) {
            std::cout << "  [成功] " << desc << "（第" << attempt << "次）" << std::endl;
            return ret;
        }
        std::cout << "  [重试] " << desc << " 第" << attempt << "次失败，500ms后重试..."
                  << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
    }
    std::cout << "  [失败] " << desc << ", 错误码: " << static_cast<int>(ret)
              << "（重试3次均失败）" << std::endl;
    return ret;
}

// @brief 轮询实际开度寄存器（REG_OPENLEN_ACT）直到接近目标或超时
// @param last 输出: 最后一次成功读到的实际开度（无成功读数时为 -1）
// @return true 到位 / false 超时
bool wait_openlen_reach(tl::SOCKETFD sock, int master_id, int target, int& last,
                        int timeout_ms = 10000)
{
    auto deadline = std::chrono::steady_clock::now()
                  + std::chrono::milliseconds(timeout_ms);
    last = -1;
    bool read_ok = false;
    while (std::chrono::steady_clock::now() < deadline) {
        std::vector<int> data;
        auto ret = tl::modbus_read_holding_registers(sock, master_id, REG_OPENLEN_ACT,
1, data);
        if (ret == tl::Result::SUCCESS && !data.empty()) {
```

```
            read_ok = true;
            last = data[0];
            std::cout << "  当前开度: " << last << " (目标 " << target << ") \r" << std::fl
ush;
            if (std::abs(last - target) <= 5) {
                std::cout << std::endl;
                return true;
            }
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(200));
    }
    std::cout << std::endl;
    if (!read_ok) last = -1;
    return false;
}

// ============================================================
// 主函数
// ============================================================
int main(int argc, char* argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif

    // 解析命令行参数: ./ex_xxx [-ip 192.168.1.13]，默认 192.168.1.13
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ROBOT_IP = argv[i + 1];
            break;
        }
    }

    // ---- 获取 SDK 版本 ----
    std::string version = tl::get_library_version();
    std::cout << "[信息] SDK 版本: " << version << std::endl;

    // ---- 连接控制器 ----
    print_separator("连接控制器");
    tl::SOCKETFD sock = tl::connect_robot(ROBOT_IP, ROBOT_PORT);
    if (sock < 0) {
        std::cerr << "[ERROR] 连接控制器失败" << std::endl;
        return -1;
    }
    std::cout << "[信息] 连接成功 (socket = " << sock << ")" << std::endl;

    tl::Result ret;

    // ---- 配置 RTU 主站 ----
    print_separator("配置 RTU 主站");
    {
        tl::ModbusMasterParameter param;
        param.type = "RTU";
        param.startAddress = true;              // 起始地址 0
```

```
        param.RTU.slaveId = RTU_SLAVE_ID;
        param.RTU.port = RTU_PORT;
        param.RTU.baudrate = RTU_BAUDRATE;
        param.RTU.checkBit = RTU_CHECK_BIT;
        param.RTU.dataBit = RTU_DATA_BIT;
        param.RTU.stopBit = RTU_STOP_BIT;

        ret = tl::modbus_set_master_parameter(sock, RTU_MASTER_ID, param);
        print_result("tl::modbus_set_master_parameter(id=1, RTU)", ret);
        if (ret != tl::Result::SUCCESS) {
            tl::disconnect_robot(sock);
            return -1;
        }
        std::cout << "  从站ID: " << param.RTU.slaveId
                  << ", 串口: " << param.RTU.port
                  << ", 波特率: " << param.RTU.baudrate << std::endl;
        std::cout << "  校验: " << param.RTU.checkBit
                  << ", 数据位: " << param.RTU.dataBit
                  << ", 停止位: " << param.RTU.stopBit << std::endl;
    }

    // ---- 打开Modbus 主站 ----
    ret = tl::modbus_open_master(sock, RTU_MASTER_ID);
    print_result("tl::modbus_open_master(id=1)", ret);
    if (ret != tl::Result::SUCCESS) {
        tl::disconnect_robot(sock);
        return -1;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));

    // ---- 03H 读当前开度 ----
    print_separator("读取夹爪当前开度");
    {
        std::vector<int> data;
        ret = read_with_retry(sock, RTU_MASTER_ID, REG_OPENLEN_ACT, 1, data, "读实际开度")
;
        if (ret != tl::Result::SUCCESS) {
            std::cout << "  [提示] 读取实际开度失败（地址 " << REG_OPENLEN_ACT
                      << " 为同段顺延假设，若持续失败请调整REG_OPENLEN_ACT）" << std::endl;
        } else {
            std::cout << "  当前开度: " << data[0] << "（0=闭合完, 1000=张开完）" << std::en
dl;
        }
    }

    // ---- 06H 设置速度与抓力（地址 11/12 为同段顺延，失败不阻塞动作） ----
    print_separator("设置速度与抓力");
    ret = tl::modbus_write_signal_holding_registers(sock, RTU_MASTER_ID, REG_SPEED_SET,
CLAW_SPEED);
    print_result("写速度 (addr=" + std::to_string(REG_SPEED_SET) + ", val="
                 + std::to_string(CLAW_SPEED) + ")", ret);
    if (ret != tl::Result::SUCCESS) {
        std::cout << "  [提示] 速度寄存器可能不在此地址，跳过（不影响开度动作）" << std::endl;
```

```
    }
    ret = tl::modbus_write_signal_holding_registers(sock, RTU_MASTER_ID, REG_FORCE_SET,
CLAW_FORCE);
    print_result("写抓力 (addr=" + std::to_string(REG_FORCE_SET) + ", val="
                 + std::to_string(CLAW_FORCE) + ")", ret);
    if (ret != tl::Result::SUCCESS) {
        std::cout << "  [提示] 抓力寄存器可能不在此地址，跳过（不影响开度动作）" << std::endl;
    }

    // ---- 06H 写目标开度（addr=10 为实测地址），并轮询实际开度到位 ----
    for (size_t i = 0; i < CLAW_TEST_POS.size(); i++) {
        int target = CLAW_TEST_POS[i];
        std::string title = "测试位置 " + std::to_string(i + 1) + "/"
                          + std::to_string(CLAW_TEST_POS.size())
                          + ": 目标开度=" + std::to_string(target);
        print_separator(title);

        ret = tl::modbus_write_signal_holding_registers(sock, RTU_MASTER_ID, REG_OPENLEN
_SET, target);
        print_result("tl::modbus_write_signal_holding_registers(addr="
                     + std::to_string(REG_OPENLEN_SET) + ", val=" + std::to_string(targe
t) + ")", ret);
        if (ret != tl::Result::SUCCESS) {
            tl::disconnect_robot(sock);
            return -1;
        }

        int last = -1;
        bool reached = wait_openlen_reach(sock, RTU_MASTER_ID, target, last);
        if (!reached) {
            std::cout << "  [提示] 未能确认到位（实际开度地址 " << REG_OPENLEN_ACT
                      << " 为同段顺延假设，若读数异常请调整 REG_OPENLEN_ACT；开度命令已下发）"
                      << std::endl;
        } else {
            std::cout << "  [到位] 开度 " << last << " 到达目标 " << target << std::endl;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(300));
    }

    // ---- 03H 读状态码与错误码 ----
    print_separator("读取夹爪状态");
    {
        std::vector<int> data;
        ret = read_with_retry(sock, RTU_MASTER_ID, REG_STATUS, 1, data, "读状态码");
        if (ret == tl::Result::SUCCESS) {
            int status = data[0];
            const char* desc = "未知";
            switch (status) {
                case 1: desc = "张开到位"; break;
                case 2: desc = "闭合到位"; break;
                case 3: desc = "停止"; break;
                case 4: desc = "夹取中"; break;
                case 5: desc = "张开中"; break;
```

```
                case 6: desc = "夹取到位"; break;
                default: break;
            }
            std::cout << "  状态码: " << status << " (" << desc << ") " << std::endl;
        }
    }
    {
        std::vector<int> data;
        ret = read_with_retry(sock, RTU_MASTER_ID, REG_ERRORCODE, 1, data, "读错误码");
        if (ret != tl::Result::SUCCESS) {
            std::cout << "  [提示] 读取错误码失败（地址 " << REG_ERRORCODE
                      << " 为同段顺延假设）" << std::endl;
            tl::disconnect_robot(sock);
            return 0;
        }
        int err = data[0];
        if (err == 0) {
            std::cout << "  错误码: 0（无错误）" << std::endl;
        } else {
            static const std::vector<std::pair<int, const char*>> bits = {
                {0, "堵转"}, {1, "过温"}, {2, "过流"}, {3, "运行故障"}, {4, "内部通信故障"}};
            std::string faults;
            for (const auto& b : bits) {
                if (err & (1 << b.first)) {
                    if (!faults.empty()) faults += ", ";
                    faults += b.second;
                }
            }
            std::cout << "  错误码: " << err << " (" << faults << ") " << std::endl;
        }
    }

    // ---- 断开连接 ----
    print_separator("断开连接");
    tl::disconnect_robot(sock);
    std::cout << "[信息] 已断开连接" << std::endl;

    std::cout << "\n[信息] 因时夹爪测试示例运行完毕" << std::endl;
    return 0;
}
```

8.2 关节实时跟踪servoJ使用示例

本节为关节实时跟踪（servoJ 透传）使用示例，完整源码见 `examples/cpp/ex_servoj.cpp`。该示例演示双端口连接（6001 控制 + 7000 伺服高频透传）、切换运行模式、open_servoJ 进入透传、循环 set_servoJ_pos 平滑运动、close_servoJ 退出。完整代码如下：

```
/**
 * ex_servoj.cpp
 * @brief 关节实时跟踪（servoJ）示例程序
```

```
 * @attention
 *   - 确保机械臂供电正常、网络通信正常
 *   - 需要双端口连接: 6001（控制）+ 7000（伺服高频透传）
 *   - servoJ 周期约 10ms，回调内不要做耗时操作
 *   - 可用命令行参数 -ip 指定控制器 IP（默认 192.168.1.13）
 *   - 运行模式: 透传模式（实时的将关节数据下发给控制器，控制器直接响应。连接7000端口后，并调用
open_servoJ即可进入透传模式。在此之前建议set_current_mode(sock, RobotMode::RUN)切换到运行模式）
 *   - 开始前需: 双端口连接(6001+7000) → 上电 → open_servoJ
 *     （注意: servoJ 相关向量必须为 7 元素，传 6 元素会导致协议解析错乱断开连接）
 *   - 退出前需: close_servoJ → tl::set_current_mode(RobotMode::TEACH) →
tl::disconnect_robot(双端口)
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_servoj
 */

#include "tl_api.h"
#include <iostream>
#include <thread>
#include <chrono>
#include <vector>
#include <string>

#ifdef _WIN32
#include <windows.h>
#endif
// ============================================================
// 工具函数
// ============================================================

void print_result(const std::string& msg, int ret)
{
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  [成功] " << msg << std::endl;
    } else {
        std::cout << "  [失败] " << msg << ", 错误码: " << ret << std::endl;
    }
}

void print_separator(const std::string& title)
{
    std::cout << "\n========== " << title << " ==========" << std::endl;
}

// ============================================================
// 主函数
// ============================================================

int main(int argc, char* argv[])
{
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
```

```
    // 解析命令行参数: ./ex_servoj [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // ---- 获取 SDK 版本 ----
    {
        std::string version = tl::get_library_version();
        std::cout << "[信息] SDK 版本: " << version << std::endl;
    }

    // ---- 双端口连接 ----
    print_separator("双端口连接");
    tl::SOCKETFD sock = tl::connect_robot(ip, "6001");
    if (sock <= 0) {
        std::cerr << "[ERROR] 连接 6001 端口失败" << std::endl;
        return -1;
    }
    tl::SOCKETFD sock_servo = tl::connect_robot(ip, "7000");
    if (sock_servo <= 0) {
        std::cerr << "[ERROR] 连接 7000 端口失败" << std::endl;
        tl::disconnect_robot(sock);
        return -1;
    }
    std::cout << "[信息] 双端口连接成功 (6001=" << sock << ", 7000=" << sock_servo << ")" <
< std::endl;

    tl::Result ret;

    // ---- 清错与下电复位 ----
    tl::clear_error(sock);
    std::this_thread::sleep_for(std::chrono::milliseconds(300));
    tl::set_servo_poweroff(sock);
    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    // ---- 切换示教模式并设置伺服就绪 (RobotMode::TEACH) ----
    print_separator("切换示教模式");
    ret = tl::set_current_mode(sock, tl::RobotMode::TEACH);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 切换示教模式失败: " << ret << std::endl;
        tl::disconnect_robot(sock);
        tl::disconnect_robot(sock_servo);
        return -1;
    }
    std::cout << "  [信息] 已切换为示教模式" << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    std::cout << "  [信息] 设置伺服就绪..." << std::endl;
    ret = tl::set_servo_state(sock, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
```

```
        std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
        tl::disconnect_robot(sock);
        tl::disconnect_robot(sock_servo);
        return -1;
    }
    print_result("tl::set_servo_state(ServoState::READY)", ret);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));

    // ---- 切换为运行模式（RobotMode::RUN）----
    print_separator("切换运行模式");
    ret = tl::set_current_mode(sock, tl::RobotMode::RUN);
    print_result("tl::set_current_mode(RobotMode::RUN)", ret);
    std::this_thread::sleep_for(std::chrono::milliseconds(2000));

    // ============================================================
    // 打开 servoJ 模式
    // ============================================================
    print_separator("servoJ 关节实时跟踪");

    // 速度/加速度/加加速度约束（7 元素，单位 °/s, °/s², °/s³）
    // 注意: SDK 底层协议要求 servoJ 相关向量固定为 7 元素，传 6 元素会导致
    //       C++ vector marshaling 时长度不足，控制器协议解析错乱并断开 7000 连接
    std::vector<double> vmax = {30.0, 30.0, 30.0, 30.0, 30.0, 30.0, 30.0};
    std::vector<double> amax = {60.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0};
    std::vector<double> jmax = {100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0};

    ret = tl::open_servoJ(sock_servo, vmax, amax, jmax);
    print_result("open_servoJ", ret);
    if (ret != tl::Result::SUCCESS) {
        tl::set_servo_poweroff(sock);
        tl::disconnect_robot(sock);
        tl::disconnect_robot(sock_servo);
        return -1;
    }

    // ---- 动作 1: J1 从 0° 平滑转到 30° ----
    std::cout << "\n--- 动作 1: J1 转到 30° ---" << std::endl;
    {
        // q 必须为 7 元素，与 open_servoJ 一致
        std::vector<double> q = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
        for (int i = 0; i < 200; ++i) {
            q[0] += (30.0 / 200.0);       // 每次增加 0.15°
            tl::set_servoJ_pos(sock_servo, q);
            std::this_thread::sleep_for(std::chrono::milliseconds(10)); // 100Hz
        }
        std::cout << "  [信息] J1=30° 到位" << std::endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    // ---- 动作 2: J1 回到 0°，同时 J2 转到 20° ----
    std::cout << "\n--- 动作 2: J1 回零, J2 转到 20° ---" << std::endl;
    {
        // q 必须为 7 元素
```

```
        std::vector<double> q = {30.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
        for (int i = 0; i < 200; ++i) {
            q[0] -= (30.0 / 200.0);       // J1 逐步回到 0°
            q[1] += (20.0 / 200.0);       // J2 逐步增加到 20°
            tl::set_servoJ_pos(sock_servo, q);
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
        std::cout << "  [信息] J1=0°, J2=20° 到位" << std::endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    // ---- 动作 3: J1=0°, J2 回到 0°, J3 转到 15° ----
    std::cout << "\n--- 动作 3: J2 回零, J3 转到 15° ---" << std::endl;
    {
        // q 必须为 7 元素
        std::vector<double> q = {0.0, 20.0, 0.0, 0.0, 0.0, 0.0, 0.0};
        for (int i = 0; i < 200; ++i) {
            q[1] -= (20.0 / 200.0);       // J2 逐步回到 0°
            q[2] += (15.0 / 200.0);       // J3 逐步增加到 15°
            tl::set_servoJ_pos(sock_servo, q);
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
        std::cout << "  [信息] J2=0°, J3=15° 到位" << std::endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    // ---- 动作 4: 所有轴回到零位 ----
    std::cout << "\n--- 动作 4: 所有轴回零 ---" << std::endl;
    {
        std::vector<double> q = {0.0, 0.0, 15.0, 0.0, 0.0, 0.0, 0.0};
        for (int i = 0; i < 150; ++i) {
            q[2] -= (15.0 / 150.0);       // J3 逐步回到 0°
            tl::set_servoJ_pos(sock_servo, q);
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
        std::cout << "  [信息] 所有轴回零到位" << std::endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    // ---- 关闭 servoJ 模式 ----
    ret = tl::close_servoJ(sock_servo);
    print_result("close_servoJ", ret);

    std::cout << "\n  [信息] servoJ 运动全部完成" << std::endl;

    // ============================================================
    // 切回示教模式（自动下电）& 断开
    // ============================================================
    print_separator("切回示教模式");

    ret = tl::set_current_mode(sock, tl::RobotMode::TEACH);
    print_result("tl::set_current_mode(RobotMode::TEACH) 切回示教模式（自动下电）", ret);
    std::this_thread::sleep_for(std::chrono::milliseconds(300));
```

```
    tl::disconnect_robot(sock);
    tl::disconnect_robot(sock_servo);
    std::cout << "[信息] 已断开连接" << std::endl;

    std::cout << "\n[信息] servoJ 示例程序运行完毕" << std::endl;
    return 0;
}
```

8.3 关节运动到目标点robot_movej与直线运动到目标位置robot_movel使用示例

本节为运动控制（MoveJ / MoveL）使用示例，完整源码见 `examples/cpp/ex_move_control.cpp`。该示例演示连接控制器、切换示教模式、伺服上电、robot_movej 关节空间运动、robot_movel 笛卡尔直线运动、下电断开。完整代码如下：

```
/**
 * ex_move_control.cpp
 * @brief 运动控制相关接口（包括MoveJ、MoveL）
 * @attention
 *   - 运行模式：示教模式
 *   - 开始前需: connect_robot → tl::set_servo_state(ServoState::READY) →
set_servo_poweron
 *   - 退出前需: set_servo_poweroff → disconnect_robot
 *   - 确保机械臂供电正常、网络通信正常
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_move_control
 */

#include <iostream>
#include <thread>
#include <chrono>
#include "tl_api.h"

#ifdef _WIN32
#include <windows.h>
#endif
void enable_servo(const int socket_fd)
{
  std::cout << "--- 检查伺服状态 ---" << std::endl;

  // 1. 先清除控制器错误和报警，再下电复位确保状态同步
  tl::clear_error(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));
  tl::set_servo_poweroff(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));

  // 2. 检查伺服状态
```

```
  tl::ServoState state;
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state) << " (ServoState::STOP/
READY/ALARM/RUNNING)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
  }

  // 3. 上电流程
  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }

  std::cout << "  [信息] 上电使能..." << std::endl;
  ret = tl::set_servo_poweron(socket_fd);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
    return;
  }
  std::this_thread::sleep_for(std::chrono::seconds(2));

  // 4. 确认结果
  tl::get_servo_state(socket_fd, state);
  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂使能上电成功（servo_state = " << static_cast<int>(state) <<
") " << std::endl;
  } else {
    std::cerr << "[ERROR] 机械臂使能上电失败（servo_state = " << static_cast<int>(state)
<< ") " << std::endl;
  }
}

/// @brief 轮询机器人运行状态等待运动完成，替代固定延时
/// @param sock TCP socket
/// @param timeout_sec 超时秒数
/// @return true 运动完成 / false 超时
```

```
static bool wait_motion_done(tl::SOCKETFD sock, int timeout_sec)
{
    const auto deadline = std::chrono::steady_clock::now()
                        + std::chrono::seconds(timeout_sec);
    int last_print = 0;

    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    while (std::chrono::steady_clock::now() < deadline) {
        tl::RunState state;
        tl::Result ret = tl::get_robot_running_state(sock, state);
        if (ret == tl::Result::SUCCESS && state == tl::RunState::STOP) {
            std::cout << "  [信息] 运动完成" << std::endl;
            return true;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(200));

        int elapsed = static_cast<int>(std::chrono::duration_cast<std::chrono::seconds>(
            std::chrono::steady_clock::now() -
            (deadline - std::chrono::seconds(timeout_sec))).count());
        if (elapsed - last_print >= 5) {
            std::cout << "  [信息] 等待运动中... (已过 " << elapsed << "s)" << std::endl;
            last_print = elapsed;
        }
    }
    std::cerr << "  [WARN] 等待运动超时 (" << timeout_sec << "s)" << std::endl;
    return false;
}

void disable_servo(const int socket_fd)
{
  tl::ServoState state;
  tl::get_servo_state(socket_fd, state);
  switch (state)
  {
    case tl::ServoState::READY:
        std::cout << "[INFO] 机械臂已使能下电" << std::endl;
        break;
    case tl::ServoState::RUNNING:
        tl::set_servo_poweroff(socket_fd);
        tl::get_servo_state(socket_fd, state);
        std::cout << "[INFO] 机械臂使能下电成功（servo_state = " <<
static_cast<int>(state) << "）" << std::endl;
        return;
    default:
        break;
  }
  std::cout << "[INFO] 机械臂使能下电失败（servo_state = " << static_cast<int>(state) << "）
" << std::endl;
}

int main(int argc, char* argv[]) {
#ifdef _WIN32
```

```
    SetConsoleOutputCP(CP_UTF8);
#endif
    // 解析命令行参数: ./ex_move_control [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // 定义端口
    std::string port = "6001";

    // 获取 SDK 版本
    std::string version = tl::get_library_version();
    std::cout << "[INFO] SDK 版本: " << version << std::endl;

    // 连接控制器
    std::cout << "[INFO] 连接控制器..." << std::endl;
    auto socket_fd = tl::connect_robot(ip, port);
    if (socket_fd < 0) {
        std::cerr << "[ERROR] 连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 连接成功 (socket=" << socket_fd << ")" << std::endl;

    // ----切换为示教模式----
    std::cout << "--- 切换示教模式 ---" << std::endl;
    auto ret = tl::set_current_mode(socket_fd, tl::RobotMode::TEACH);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 切换示教模式失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 已切换为示教模式" << std::endl;
    std::cout << std::endl;

    // 使能上电
    enable_servo(socket_fd);

    // 设置全局速度
    ret = tl::set_speed(socket_fd, 30);
    std::this_thread::sleep_for(std::chrono::milliseconds(200));

    // ----MoveJ运动控制----
    std::cout << "[INFO] **********MoveJ运动控制**********" << std::endl;
    tl::MoveCmd cmd;
    cmd.targetPosType  = tl::PosType::PType;
    cmd.targetPosName  = "";
    cmd.coord          = tl::Coord::JOINT;
    cmd.velocity       = 40;
    cmd.acc            = 10;
    cmd.dec            = 10;
    cmd.pl             = 0;
```

```
    cmd.targetPosValue = std::vector<double>{
      20.0, 10.0, -10.0,
      0.0, 0.0, 0.0, 0.0}; // 关节坐标下单位为度
    ret = tl::robot_movej(socket_fd, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] MoveJ运动控制失败，程序退出!" << std::endl;
        return -1;
    }
    // 等待运动到位
    wait_motion_done(socket_fd, 30);

    std::cout << "[INFO] MoveJ运动控制成功" << std::endl;
    std::cout << std::endl;

    // ----MoveL运动控制----
    std::cout << "[INFO] **********MoveL运动控制**********" << std::endl;
    cmd.targetPosName  = "";
    cmd.coord          = tl::Coord::BASE;
    cmd.targetPosValue = std::vector<double>{
      280, 90.0, 270,
      3.14, 0.0, 0.0}; // 基坐标下位移单位是mm，旋转单位是弧度
    ret = tl::robot_movel(socket_fd, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] MoveL运动控制失败，程序退出!" << std::endl;
        return -1;
    }
    // 等待运动到位
    wait_motion_done(socket_fd, 30);

    std::cout << "[INFO] MoveL运动控制成功" << std::endl;
    std::cout << std::endl;

    // 使能下电
    disable_servo(socket_fd);

    // 断开控制器连接
    std::cout << "[INFO] 断开控制器连接..." << std::endl;
    ret = tl::disconnect_robot(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 断开连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 断开连接成功" << std::endl;

    return 0;
}
```

8.4 点动使用示例

本节为示教控制（点动）使用示例，完整源码见 `examples/cpp/ex_teach_control.cpp`。该示例演示连接控制器、切换示教模式、robot_start_jogging 开始点动、robot_stop_jogging 停止点动，以及电流环拖动示教灵敏度的设置与查询。完整代码如下：

```
/**
 * ex_teach_control.cpp
 * @brief 示教控制相关接口
 * @attention
 *   - 运行模式: 示教模式
 *   - 开始前需: connect_robot → tl::set_servo_state(tl::ServoState::READY) →
set_servo_poweron
 *   - 退出前需: set_servo_poweroff → disconnect_robot
 *   - 确保机械臂供电正常、网络通信正常
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_teach_control
 */

#include <iostream>
#include <thread>
#include <chrono>
#include "tl_api.h"

#ifdef _WIN32
#include <windows.h>
#endif
void enable_servo(const int socket_fd)
{
  std::cout << "--- 检查伺服状态 ---" << std::endl;

  // 1. 先清除控制器错误和报警，再下电复位确保状态同步
  tl::clear_error(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));
  tl::set_servo_poweroff(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));

  // 2. 检查伺服状态
  tl::ServoState state;
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state)
            << " (ServoState::STOP/READY/ALARM/RUNNING)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
```

```
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
  }

  // 3. 上电流程
  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }

  std::cout << "  [信息] 上电使能..." << std::endl;
  ret = tl::set_servo_poweron(socket_fd);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
    return;
  }
  std::this_thread::sleep_for(std::chrono::seconds(2));

  // 4. 确认结果
  tl::get_servo_state(socket_fd, state);
  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂使能上电成功（servo_state = " << static_cast<int>(state) <<
") " << std::endl;
  } else {
    std::cerr << "[ERROR] 机械臂使能上电失败（servo_state = " << static_cast<int>(state)
<< ") " << std::endl;
  }
}

void disable_servo(const int socket_fd)
{
  tl::ServoState state;
  tl::get_servo_state(socket_fd, state);
  switch (state)
  {
    case tl::ServoState::READY:
        std::cout << "[INFO] 机械臂已使能下电" << std::endl;
        break;
    case tl::ServoState::RUNNING:
        tl::set_servo_poweroff(socket_fd);
        tl::get_servo_state(socket_fd, state);
        std::cout << "[INFO] 机械臂使能下电成功（servo_state = " <<
static_cast<int>(state) << ") " << std::endl;
        return;
    default:
```

```
        break;
  }
  std::cout << "[INFO] 机械臂使能下电失败（servo_state = " << static_cast<int>(state) << ")
" << std::endl;
}

int main(int argc, char* argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    // 解析命令行参数: ./ex_teach_control [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // 定义端口
    std::string port = "6001";

    // 获取 SDK 版本
    std::string version = tl::get_library_version();
    std::cout << "[INFO] SDK 版本: " << version << std::endl;

    // 连接控制器
    std::cout << "[INFO] 连接控制器..." << std::endl;
    auto socket_fd = tl::connect_robot(ip, port);
    if (socket_fd < 0) {
        std::cerr << "[ERROR] 连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 连接成功 (socket=" << socket_fd << ")" << std::endl;

    // 使能上电
    enable_servo(socket_fd);
    std::this_thread::sleep_for(std::chrono::seconds(2));

    // ----切换为示教模式----
    // 示教点动需要在示教模式下执行（tl::RobotMode::TEACH）
    std::cout << "--- 切换示教模式 ---" << std::endl;
    auto ret = tl::set_current_mode(socket_fd, tl::RobotMode::TEACH);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 切换示教模式失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 已切换为示教模式" << std::endl;

    // ----示教点动（注意安全）----
    // ⚠️  robot_start_jogging 会直接驱动机器人运动，运行前确保安全区域无人
    std::cout << "[INFO] **********示教点动**********" << std::endl;
    int axis = 1;
    bool direction = true;
```

```
    ret = tl::robot_start_jogging(socket_fd, axis, direction);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 示教点动失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 示教点动成功" << std::endl;

    std::this_thread::sleep_for(std::chrono::seconds(5));

    // ----停止点动----
    ret = tl::robot_stop_jogging(socket_fd, axis);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 停止点动失败，错误码: " << ret << std::endl;
    } else {
        std::cout << "[INFO] 停止点动成功" << std::endl;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    // ----设置电流环拖动示教灵敏度----
    // 范围 [0, 3]，数值越小灵敏度越低
    std::cout << "[INFO] **********设置电流环拖动示教灵敏度**********" << std::endl;
    {
        std::vector<double> friction_offsets = {1.5, 1.5, 1.5, 1.5, 1.5, 1.5};
        ret = tl::set_current_teach_sensitivity(socket_fd, friction_offsets);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 设置电流环拖动示教灵敏度失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 设置电流环拖动示教灵敏度成功:" << std::endl;
            std::cout << "  [J1~J6]: ";
            for (const auto& val : friction_offsets) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;

    // ----查询电流环拖动示教灵敏度----
    std::cout << "[INFO] **********查询电流环拖动示教灵敏度**********" << std::endl;
    {
        std::vector<double> verify_offsets;
        ret = tl::get_current_teach_sensitivity(socket_fd, verify_offsets);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 查询电流环拖动示教灵敏度失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 当前电流环拖动示教灵敏度 (J1~J6): ";
            for (const auto& val : verify_offsets) {
                std::cout << val << " ";
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;
```

```
    // 使能下电
    disable_servo(socket_fd);

    // 断开控制器连接
    std::cout << "[INFO] 断开控制器连接..." << std::endl;
    ret = tl::disconnect_robot(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 断开连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 断开连接成功" << std::endl;

    return 0;
}
```

8.5 作业相关接口使用示例

本节为作业控制使用示例，完整源码见 `examples/cpp/ex_job_control.cpp`。该示例演示创建作业文件、打开作业文件、插入 MoveJ/MoveL/MoveC 指令、获取全部作业文件名、切换运行模式、运行作业文件。完整代码如下：

```
/**
 * ex_job_control.cpp
 * @brief 作业控制相关接口（包括作业文件插入MoveJ、MoveL、MoveC，打开作业文件，创建作业文件，获取全部作业文件名，运行作业文件）
 * @attention
 *   - 插入作业之前需要先打开一个作业文件
 *   - 运行一个作业文件需要切换到运行模式
 *   - 确保机械臂供电正常、网络通信正常
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_job_control
 */

#include <iostream>
#include <thread>
#include <chrono>
#include "tl_api.h"

#ifdef _WIN32
#include <windows.h>
#endif
void enable_servo(const int socket_fd)
{
  std::cout << "--- 检查伺服状态 ---" << std::endl;

  // 1. 先清除控制器错误和报警，再下电复位确保状态同步
  tl::clear_error(socket_fd);
```

```
  std::this_thread::sleep_for(std::chrono::milliseconds(300));
  tl::set_servo_poweroff(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));

  // 2. 检查伺服状态
  tl::ServoState state;
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state)
            << " (STOP=停止 READY=就绪 ALARM=报警 RUNNING=运行)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
  }

  // 3. 上电流程
  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }

  std::cout << "  [信息] 上电使能..." << std::endl;
  ret = tl::set_servo_poweron(socket_fd);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
    return;
  }
  std::this_thread::sleep_for(std::chrono::seconds(2));

  // 4. 确认结果
  tl::get_servo_state(socket_fd, state);
  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂使能上电成功（servo_state = " << static_cast<int>(state) <<
") " << std::endl;
  } else {
    std::cerr << "[ERROR] 机械臂使能上电失败（servo_state = " << static_cast<int>(state)
<< ") " << std::endl;
  }
}
```

```cpp
void disable_servo(const int socket_fd)
{
  tl::ServoState state;
  tl::get_servo_state(socket_fd, state);
  switch (state)
  {
    case tl::ServoState::READY:
        std::cout << "[INFO] 机械臂已使能下电" << std::endl;
        break;
    case tl::ServoState::RUNNING:
        tl::set_servo_poweroff(socket_fd);
        tl::get_servo_state(socket_fd, state);
        std::cout << "[INFO] 机械臂使能下电成功（servo_state = " <<
static_cast<int>(state) << "）" << std::endl;
        return;
    default:
        break;
  }
  std::cout << "[INFO] 机械臂使能下电失败（servo_state = " << static_cast<int>(state) << "）
" << std::endl;
}

int main(int argc, char* argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    // 解析命令行参数: ./ex_job_control [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // 定义端口
    std::string port = "6001";

    // 获取 SDK 版本
    std::string version = tl::get_library_version();
    std::cout << "[INFO] SDK 版本: " << version << std::endl;

    // 连接控制器
    std::cout << "[INFO] 连接控制器..." << std::endl;
    auto socket_fd = tl::connect_robot(ip, port);
    if (socket_fd < 0) {
        std::cerr << "[ERROR] 连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 连接成功 (socket=" << socket_fd << ")" << std::endl;

    // 使能上电
    enable_servo(socket_fd);
```

```
    // ----创建作业文件----
    std::string jobName = "TEST_CPP";
    std::cout << "[INFO] **********创建作业文件**********" << std::endl;
    auto ret = tl::job_create(socket_fd, jobName);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 创建作业文件失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 创建作业文件成功" << std::endl;
    std::cout << std::endl;

    // ----打开作业文件----
    std::cout << "[INFO] **********打开作业文件**********" << std::endl;
    ret = tl::job_open(socket_fd, jobName);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 打开作业文件失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 打开作业文件成功" << std::endl;
    std::cout << std::endl;

    // ----作业文件插入MoveJ----
    std::cout << "[INFO] **********作业文件插入MoveJ**********" << std::endl;
    int line = 1;
    tl::MoveCmd cmd;
    cmd.targetPosType  = tl::PosType::PType;
    cmd.targetPosName  = "P0001";
    cmd.coord          = tl::Coord::JOINT;
    cmd.velocity       = 40;
    cmd.acc            = 10;
    cmd.dec            = 10;
    cmd.pl             = 0;
    cmd.targetPosValue = std::vector<double>{0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
    ret = tl::job_insert_moveJ(socket_fd, line, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 作业文件插入MoveJ失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 作业文件插入MoveJ成功" << std::endl;
    std::cout << std::endl;

    // ----作业文件插入MoveL----
    std::cout << "[INFO] **********作业文件插入MoveL**********" << std::endl;
    line = 2;
    cmd.targetPosName  = "P0002";
    cmd.coord          = tl::Coord::BASE;
    cmd.targetPosValue = std::vector<double>{280, 0.0, 380, 3.14, 0.0, 0.0};
    ret = tl::job_insert_moveL(socket_fd, line, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 作业文件插入MoveL失败，程序退出!" << std::endl;
        return -1;
```

```
    }
    std::cout << "[INFO] 作业文件插入MoveL成功" << std::endl;
    std::cout << std::endl;

    // ----作业文件插入MoveC（圆弧中间点）----
    std::cout << "[INFO] **********作业文件插入MoveC**********" << std::endl;
    line = 3;
    cmd.targetPosName  = "P0003";
    cmd.coord          = tl::Coord::TOOL;
    cmd.targetPosValue = std::vector<double>{270, 0.0, 380, 3.14, 0.0, 0.0};
    ret = tl::job_insert_moveC(socket_fd, line, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 作业文件插入MoveC失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 作业文件插入MoveC成功" << std::endl;
    std::cout << std::endl;

    // ----作业文件插入MoveC（圆弧终点）----
    std::cout << "[INFO] **********作业文件插入MoveC**********" << std::endl;
    line = 3;
    cmd.targetPosName  = "P0003";
    cmd.coord          = tl::Coord::TOOL;
    cmd.targetPosValue = std::vector<double>{240, 0.0, 380, 3.14, 0.0, 0.0};
    ret = tl::job_insert_moveC(socket_fd, line, cmd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 作业文件插入MoveC失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 作业文件插入MoveC成功" << std::endl;
    std::cout << std::endl;

    // ----获取所有作业文件名----
    std::cout << "[INFO] **********获取所有作业文件名**********" << std::endl;
    std::vector<std::vector<std::string>> AllJobNames;
    ret = tl::job_get_all_jobfile_name(socket_fd, AllJobNames);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 获取作业文件名失败，程序退出!" << std::endl;
        return -1;
    }
    int i = 1;
    for (const auto& JobNames : AllJobNames) {
        for (const auto& name : JobNames) {
            std::cout << "[INFO] 作业文件 " << i++ << " 名称: " << name << std::endl;
        }
    }

    // ----切换为运行模式----
    // 运行作业文件需要在运行模式下执行（RUN=运行）
    std::cout << "--- 切换运行模式 ---" << std::endl;
    ret = tl::set_current_mode(socket_fd, tl::RobotMode::RUN);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 切换运行模式失败，程序退出!" << std::endl;
```

```
        return -1;
    }
    std::cout << "[INFO] 已切换为运行模式" << std::endl;

    // ----运行作业文件----
    std::cout << "[INFO] **********运行作业文件**********" << std::endl;
    ret = tl::job_run(socket_fd, jobName);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 运行作业文件失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 运行作业文件成功" << std::endl;
    std::cout << std::endl;

    // 使能下电
    disable_servo(socket_fd);

    // 断开控制器连接
    std::cout << "[INFO] 断开控制器连接..." << std::endl;
    ret = tl::disconnect_robot(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 断开连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 断开连接成功" << std::endl;

    return 0;
}
```

8.6 队列运动使用示例

本节为队列运动使用示例，完整源码见 `examples/cpp/ex_queue_motion.cpp`。该示例演示队列运动的完整工作流：开启队列模式、插入 MoveJ/MoveL 指令、发送队列到控制器、等待执行完成，以及暂停/恢复/停止保持上电等控制。完整代码如下：

```
/**
 * ex_queue_motion.cpp
 * @brief 队列运动示例程序 —— 展示队列运动完整工作流
 * @attention
 *   - 确保机械臂供电正常、网络通信正常
 *   - 可用命令行参数 -ip 指定控制器 IP（默认 192.168.1.13）
 *   - 队列运动模式下，所有指令先在本地缓存，调用 send_to_controller 后一次性下发执行
 *   - 演示三种队列运动：纯 MoveJ、纯 MoveL、MoveJ+MoveL 混合
 *   - 运行模式: 队列模式（在queue_motion_set_status(true)之后，即可进入队列模式，在此之前建议
set_current_mode(sock, RobotMode::RUN)切换到运行模式）
 *   - 开始前需: connect_robot → 上电 → tl::queue_motion_set_status(true)
 *   - 退出前需: tl::queue_motion_set_status(false) →
tl::set_current_mode(RobotMode::TEACH) → set_servo_poweroff → disconnect_robot
 * @note 运行步骤
```

```
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_queue_motion
 */

#include "tl_api.h"
#include <iostream>
#include <thread>
#include <chrono>
#include <vector>
#include <string>
#include <functional>

#ifdef _WIN32
#include <windows.h>
#endif
// ============================================================
// 工具函数
// ============================================================

void print_result(const std::string& msg, int ret)
{
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  [成功] " << msg << std::endl;
    } else {
        std::cout << "  [失败] " << msg << ", 错误码: " << ret << std::endl;
    }
}

void print_separator(const std::string& title)
{
    std::cout << "\n========== " << title << " ==========" << std::endl;
}

/**
 * @brief 构建 MoveCmd（关节坐标模式）
 * @param joints 6 个关节角度值，单位 °（度），顺序 [J1, J2, J3, J4, J5, J6]
 * @param vel    百分比 % (1,100]，最大关节速度的百分比，实际受全局速度限制
 * @param acc    百分比 % (1,100]
 * @param dec    百分比 % (1,100]
 * @param pl     平滑参数 [0,5]
 * @note 实际运动速度 = 全局速度 × (vel / 100)
 */
tl::MoveCmd make_movej_cmd(const std::vector<double>& joints,
                           double vel = 30, double acc = 50, double dec = 50, int pl = 0)
{
    tl::MoveCmd cmd;
    cmd.coord        = tl::Coord::JOINT; // 关节坐标系
    cmd.velocity     = vel;
    cmd.acc          = acc;
    cmd.dec          = dec;
    cmd.pl           = pl;
    cmd.targetPosValue.resize(14, 0.0);
    for (size_t i = 0; i < joints.size() && i < 7; ++i) {
```

```
        cmd.targetPosValue[i] = joints[i];
    }
    return cmd;
}

/**
 * @brief 构建 MoveCmd（基坐标模式）
 * @param pose 位姿 [X, Y, Z, Rx, Ry, Rz]，位置单位 mm，姿态单位 °
 * @param vel    百分比 % (1,1000]，最大 TCP 线速度的百分比
 * @param acc    百分比 % (1,100]
 * @param dec    百分比 % (1,100]
 * @param pl     平滑参数 [0,5]
 * @note 实际运动速度 = 全局速度 × (vel / 100)
 */
tl::MoveCmd make_movel_cmd(const std::vector<double>& pose,
                           double vel = 100, double acc = 50, double dec = 50, int pl = 0)
{
    tl::MoveCmd cmd;
    cmd.coord        = tl::Coord::BASE; // 基坐标系，可切换到 tl::Coord::TOOL /
tl::Coord::USER
    cmd.velocity     = vel;
    cmd.acc          = acc;
    cmd.dec          = dec;
    cmd.pl           = pl;
    cmd.targetPosValue.resize(14, 0.0);
    for (size_t i = 0; i < pose.size() && i < 7; ++i) {
        cmd.targetPosValue[i] = pose[i];
    }
    return cmd;
}

// ============================================================
// 队列运动执行器
// ============================================================

/**
 * @brief 执行一轮队列运动（通用执行器）
 * @param sock 控制端口 socket
 * @param name 演示名称
 * @param fn_push 回调：向队列插入运动指令
 * @note 该函数演示完整的队列运动流程：
 *       1. queue_motion_set_status(true) 开启队列模式（会清空远端队列）
 *       2. 通过 fn_push 回调插入运动指令
 *       3. queue_motion_size 查询本地队列长度
 *       4. queue_motion_send_to_controller(sock, 0, false) 发送全部指令到控制器
 *       5. 轮询 get_robot_running_state 等待运动完成
 *       6. queue_motion_set_status(false) 关闭队列模式
 * @return true 成功 / false 失败
 */
bool run_queue_motion(tl::SOCKETFD sock, const std::string& name,
                      std::function<bool(tl::SOCKETFD)> fn_push)
{
    print_separator("队列运动：" + name);
```

```
    // 1. 打开队列运动模式（会清空远端队列）
    tl::Result ret = tl::queue_motion_set_status(sock, true);
    print_result("tl::queue_motion_set_status(true)", ret);
    if (ret != tl::Result::SUCCESS) return false;
    std::this_thread::sleep_for(std::chrono::milliseconds(200));

    // 确认队列模式已开启
    {
        bool status = false;
        tl::queue_motion_get_status(sock, status);
        std::cout << "  [信息] 队列模式状态: " << (status ? "已开启" : "已关闭") <<
std::endl;
    }

    // 2. 插入运动指令
    std::cout << "  [信息] 插入运动指令..." << std::endl;
    if (!fn_push(sock)) {
        std::cerr << "[ERROR] 插入运动指令失败" << std::endl;
        tl::queue_motion_set_status(sock, false);
        return false;
    }

    // 3. 查询本地队列长度
    {
        int size = 0;
        ret = tl::queue_motion_size(sock, size);
        if (ret == tl::Result::SUCCESS) {
            std::cout << "  [信息] 本地队列长度: " << size << " 条指令" << std::endl;
        }
    }

    // 4. 发送队列到控制器并等待执行完成
    std::cout << "  [信息] 发送队列到控制器..." << std::endl;
    ret = tl::queue_motion_send_to_controller(sock, 0, false);
    print_result("send_to_controller", ret);
    if (ret != tl::Result::SUCCESS) {
        tl::queue_motion_set_status(sock, false);
        return false;
    }

    std::cout << "  [信息] 等待运动执行完成..." << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    while (true) {
        tl::RunState running_state;
        ret = tl::get_robot_running_state(sock, running_state);
        if (ret == tl::Result::SUCCESS && running_state == tl::RunState::STOP) break;
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
    }
    std::cout << "  [信息] 队列运动执行完毕" << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));

    // 5. 关闭队列运动模式
```

```
    tl::queue_motion_set_status(sock, false);
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    return true;
}

// ============================================================
// 主函数
// ============================================================

int main(int argc, char* argv[])
{
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    // 解析命令行参数: ./ex_queue_motion [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // ---- 获取 SDK 版本 ----
    {
        std::string version = tl::get_library_version();
        std::cout << "[信息] SDK 版本: " << version << std::endl;
    }

    // ---- 连接控制器 ----
    print_separator("连接控制器");
    tl::SOCKETFD sock = tl::connect_robot(ip, "6001");
    if (sock < 0) {
        std::cerr << "[ERROR] 连接控制器失败" << std::endl;
        return -1;
    }
    std::cout << "[信息] 连接成功 (socket = " << sock << ")" << std::endl;

    // ---- 清错 ----
    tl::clear_error(sock);
    std::this_thread::sleep_for(std::chrono::milliseconds(300));
    tl::set_servo_poweroff(sock);
    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    tl::Result ret;

    // ---- 检查模式并确保伺服运行 ----
    {
        // 先获取当前模式
        tl::RobotMode current_mode;
        tl::Result ret = tl::get_current_mode(sock, current_mode);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取当前模式失败: " << ret << std::endl;
            tl::disconnect_robot(sock);
```

```
            return -1;
        }
        std::cout << "  [信息] 当前模式: " << static_cast<int>(current_mode) << " (RobotMode::TEACH/REMOTE/RUN)" << std::endl;

        if (current_mode == tl::RobotMode::TEACH) {
            // 示教模式 → 先确保伺服就绪上电，再切换为运行模式
            tl::ServoState servo_state;
            ret = tl::get_servo_state(sock, servo_state);
            if (ret != tl::Result::SUCCESS) {
                std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
                tl::disconnect_robot(sock);
                return -1;
            }
            std::cout << "  [信息] 伺服状态: " << static_cast<int>(servo_state) << " (ServoState::STOP/READY/ALARM/RUNNING)" << std::endl;

            if (servo_state != tl::ServoState::RUNNING) {
                // 伺服未运行 → 执行上电流程
                std::cout << "  [信息] 伺服未运行，执行上电流程..." << std::endl;
                ret = tl::set_servo_state(sock, tl::ServoState::READY);
                if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
                    std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
                    tl::disconnect_robot(sock);
                    return -1;
                }
                std::this_thread::sleep_for(std::chrono::milliseconds(500));

                ret = tl::set_servo_poweron(sock);
                if (ret != tl::Result::SUCCESS) {
                    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
                    tl::disconnect_robot(sock);
                    return -1;
                }
                std::cout << "  [信息] 上电成功" << std::endl;
                std::this_thread::sleep_for(std::chrono::seconds(2));
            } else {
                std::cout << "  [信息] 伺服已在运行状态" << std::endl;
            }

            // 切换为运行模式
            std::cout << "  [信息] 切换为运行模式..." << std::endl;
            ret = tl::set_current_mode(sock, tl::RobotMode::RUN);
            if (ret != tl::Result::SUCCESS) {
                std::cerr << "[ERROR] 切换运行模式失败: " << ret << std::endl;
                tl::disconnect_robot(sock);
                return -1;
            }
            std::cout << "  [信息] 已切换为运行模式" << std::endl;
            std::this_thread::sleep_for(std::chrono::seconds(1));
        } else if (current_mode == tl::RobotMode::RUN) {
            // 已在运行模式，直接检查伺服状态
```

```
            tl::ServoState servo_state;
            ret = tl::get_servo_state(sock, servo_state);
            if (ret != tl::Result::SUCCESS || servo_state != tl::ServoState::RUNNING) {
                std::cerr << "[ERROR] 伺服未在运行状态 (servo_state=" << static_cast<int>(s
ervo_state) << ")" << std::endl;
                tl::disconnect_robot(sock);
                return -1;
            }
            std::cout << "  [信息] 已在运行模式，伺服运行中 (servo_state=RUNNING)" << std::en
dl;
        } else {
            std::cerr << "[ERROR] 不支持的模式: " << static_cast<int>(current_mode) << ",
期望RobotMode::TEACH 或 RobotMode::RUN模式" << std::endl;
            tl::disconnect_robot(sock);
            return -1;
        }
    }

    // 延时 2 秒，保证状态完全就绪
    std::this_thread::sleep_for(std::chrono::seconds(2));

    // ---- 设置全局速度 ----
    // 控制器默认全局速度为 5%，设为 30% 提升运动效率
    print_separator("设置全局速度");
    ret = tl::set_speed(sock, 30);
    print_result("tl::set_speed(30)", ret);

    // ============================================================
    // 演示 1: 纯 MoveJ 队列运动
    // ============================================================
    run_queue_motion(sock, "纯 MoveJ", [](tl::SOCKETFD sock) -> bool {
        tl::Result r;

        std::cout << "  push_back_moveJ:  J1=30°" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({30, 0, 0, 0, 0, 0}, 3
0));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;

        std::cout << "  push_back_moveJ:  J1=30°, J2=20°" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({30, 20, 0, 0, 0, 0},
30));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;

        std::cout << "  push_back_moveJ:  J1=0°, J2=20°" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({0, 20, 0, 0, 0, 0}, 3
0));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;

        std::cout << "  push_back_moveJ:  回零" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({0, 0, 0, 0, 0, 0},
```

```
20));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;

        return true;
    });

    // ============================================================
    // 演示 2: 纯 MoveL 队列运动
    // ============================================================
    run_queue_motion(sock, "纯 MoveL", [](tl::SOCKETFD sock) -> bool {
        tl::Result r;

        std::cout << "  push_back_moveL:  X=300, Y=0, Z=300" << std::endl;
        r = tl::queue_motion_push_back_moveL(sock, make_movel_cmd({300, 0, 300, 3.141,
0, 0}, 30));
        print_result("push_back_moveL", r);
        if (r != tl::Result::SUCCESS) return false;

        std::cout << "  push_back_moveL:  X=400, Y=100, Z=300" << std::endl;
        r = tl::queue_motion_push_back_moveL(sock, make_movel_cmd({400, 100, 300,
3.141, 0, 0}, 30));
        print_result("push_back_moveL", r);
        if (r != tl::Result::SUCCESS) return false;

        std::cout << "  push_back_moveL:  X=300, Y=200, Z=300" << std::endl;
        r = tl::queue_motion_push_back_moveL(sock, make_movel_cmd({300, 200, 300,
3.141, 0, 0}, 30));
        print_result("push_back_moveL", r);
        if (r != tl::Result::SUCCESS) return false;

        std::cout << "  push_back_moveL:  回起始点" << std::endl;
        r = tl::queue_motion_push_back_moveL(sock, make_movel_cmd({229.882, 0, 358.680,
3.141, 0, 0}, 20));
        print_result("push_back_moveL", r);
        if (r != tl::Result::SUCCESS) return false;

        return true;
    });

    // ============================================================
    // 演示 3: MoveJ + MoveL 混合队列运动
    // ============================================================
    run_queue_motion(sock, "MoveJ + MoveL 混合", [](tl::SOCKETFD sock) -> bool {
        tl::Result r;

        // 先用 MoveJ 调整姿态
        std::cout << "  push_back_moveJ:  J1=45°（调整关节姿态）" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({45, 0, 0, 0, 0, 0}, 3
0));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;
```

```
        // 再用 MoveL 走直线
        std::cout << "  push_back_moveL:  X=400, Y=0, Z=300" << std::endl;
        r = tl::queue_motion_push_back_moveL(sock, make_movel_cmd({400, 0, 300, 3.141,
0, 0}, 30));
        print_result("push_back_moveL", r);
        if (r != tl::Result::SUCCESS) return false;

        // MoveJ 换向
        std::cout << "  push_back_moveJ:  J1=-45°（换向）" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({-45, 0, 0, 0, 0, 0},
30));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;

        // MoveL 走另一条直线
        std::cout << "  push_back_moveL:  X=300, Y=200, Z=300" << std::endl;
        r = tl::queue_motion_push_back_moveL(sock, make_movel_cmd({300, 200, 300,
3.141, 0, 0}, 30));
        print_result("push_back_moveL", r);
        if (r != tl::Result::SUCCESS) return false;

        // MoveJ 回零
        std::cout << "  push_back_moveJ:  回零" << std::endl;
        r = tl::queue_motion_push_back_moveJ(sock, make_movej_cmd({0, 0, 0, 0, 0, 0},
20));
        print_result("push_back_moveJ", r);
        if (r != tl::Result::SUCCESS) return false;

        return true;
    });

    // ============================================================
    // 演示 4: 队列运动控制（暂停/恢复/停止保持上电）
    // ============================================================
    {
        print_separator("队列运动控制：暂停/恢复/停止");

        // 辅助：等待队列剩余数减少到 target_len
        auto wait_queue_len = [&](int target_len) {
            int len = -1;
            do {
                std::this_thread::sleep_for(std::chrono::milliseconds(200));
                tl::queue_motion_get_queuelen(sock, len);
            } while (len > target_len && len != -1);
        };

        // 1. 开启队列模式
        tl::Result r = tl::queue_motion_set_status(sock, true);
        print_result("tl::queue_motion_set_status(true)", r);
        if (r != tl::Result::SUCCESS) return -1;
        std::this_thread::sleep_for(std::chrono::seconds(2));

        // 2. 插入 4 条 J5 摆动指令（简单清晰，便于观察控制效果）
```

```
        std::cout << "  [信息] 插入 4 条MoveJ 指令（J5 摆动）" << std::endl;
        std::vector<std::vector<double>> targets = {
            {0, 0, 0, 0, -30, 0, 0},    // J5 → -30°
            {0, 0, 0, 0,   0, 0, 0},    // J5 → 0°
            {0, 0, 0, 0, -30, 0, 0},    // J5 → -30°
            {0, 0, 0, 0,   0, 0, 0},    // J5 → 0°
        };
        for (size_t i = 0; i < targets.size(); ++i) {
            auto cmd = make_movej_cmd(targets[i], 30);
            r = tl::queue_motion_push_back_moveJ(sock, cmd);
            std::cout << "  第" << (i + 1) << "条: push_back_moveJ (J5→" << targets[i]
[4] << "°)"
                      << (r == tl::Result::SUCCESS ? " 成功" : " 失败") << std::endl;
        }

        // 3. 发送队列到控制器
        std::cout << "  [信息] 发送队列到控制器..." << std::endl;
        r = tl::queue_motion_send_to_controller(sock, 0);
        print_result("send_to_controller", r);
        if (r != tl::Result::SUCCESS) return -1;

        // 4. 第 1 条执行中 → 暂停
        std::cout << "  [信息] 第 1 条执行中，等待 2 秒后暂停..." << std::endl;
        std::this_thread::sleep_for(std::chrono::seconds(2));
        r = tl::queue_motion_suspend(sock);
        print_result("暂停 (suspend)", r);
        std::this_thread::sleep_for(std::chrono::seconds(2));

        // 5. 恢复
        r = tl::queue_motion_restart(sock);
        print_result("恢复 (restart)", r);
        std::cout << "  [信息] 等待第 1 条完成..." << std::endl;
        wait_queue_len(3);
        std::cout << "  [信息] 第 1 条完成" << std::endl;

        // 6. 等待第 2 条完成
        std::cout << "  [信息] 等待第 2 条完成..." << std::endl;
        wait_queue_len(2);
        std::cout << "  [信息] 第 2 条完成" << std::endl;

        // 7. 第 3 条执行中 → 暂停
        std::this_thread::sleep_for(std::chrono::seconds(1));
        r = tl::queue_motion_suspend(sock);
        print_result("再次暂停 (suspend)", r);
        std::this_thread::sleep_for(std::chrono::seconds(2));

        // 8. 恢复
        r = tl::queue_motion_restart(sock);
        print_result("再次恢复 (restart)", r);
        std::cout << "  [信息] 等待第 3 条完成..." << std::endl;
        wait_queue_len(1);
        std::cout << "  [信息] 第 3 条完成" << std::endl;
```

```
        // 9. 第 4 条执行中 → 停止（保持上电）
        std::this_thread::sleep_for(std::chrono::seconds(2));
        r = tl::queue_motion_stop_not_power_off(sock);
        print_result("停止保持上电 (stop_not_power_off)", r);
        std::this_thread::sleep_for(std::chrono::seconds(1));

        // 10. 关闭队列模式
        tl::queue_motion_set_status(sock, false);
        std::cout << "  [信息] 队列运动控制演示完毕" << std::endl;
    }

    // ---- 切回示教模式并断开连接 ----
    print_separator("切回示教模式 + 断开连接");
    ret = tl::set_current_mode(sock, tl::RobotMode::TEACH);
    print_result("tl::set_current_mode(RobotMode::TEACH) 切回示教模式", ret);
    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    tl::disconnect_robot(sock);
    std::cout << "[信息] 已断开连接" << std::endl;

    std::cout << "\n[信息] 队列运动示例程序运行完毕" << std::endl;
    return 0;
}
```

8.7 坐标转换算法示例

本节为坐标变换算法使用示例，完整源码见 `examples/cpp/ex_coord_transform.cpp`。该示例演示四元数↔欧拉角、欧拉角→旋转矩阵、位姿矩阵↔旋转矩阵、坐标系转换（正解/逆解）以及逆解结果通过 servoJ 下发运动。完整代码如下：

```
/**
 * ex_coord_transform.cpp
 * @brief 坐标变换算法示例程序 —— 展示常用的坐标系转换接口
 * @attention
 *   - 需要控制器连接正常、已上电
 *   - 可用命令行参数 -ip 指定控制器 IP（默认 192.168.1.13）
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_coord_transform
 */

// MSVC: 启用 M_PI / M_PI_2 等 POSIX 数学常量
#define _USE_MATH_DEFINES
#include "tl_api.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <cmath>
#include <thread>
```

```cpp
#include <chrono>

#ifdef _WIN32
#include <windows.h>
#endif
// 跨平台兜底：部分编译器在严格 C++ 模式下不定义 M_PI / M_PI_2
#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif
#ifndef M_PI_2
#define M_PI_2 1.57079632679489661923
#endif

// ============================================================
// 工具函数
// ============================================================

void print_result(const std::string& msg, int ret)
{
    if (ret == tl::Result::SUCCESS) {
        std::cout << "  [成功] " << msg << std::endl;
    } else {
        std::cout << "  [失败] " << msg << ", 错误码: " << ret << std::endl;
    }
}

void print_separator(const std::string& title)
{
    std::cout << "\n========== " << title << " ==========" << std::endl;
}

/// @brief 打印 double 向量
void print_vector(const std::string& name, const std::vector<double>& v, int width = 6)
{
    std::cout << "  " << name << " [" << v.size() << "] = {";
    for (size_t i = 0; i < v.size(); ++i) {
        std::cout << std::fixed << std::setprecision(4) << v[i];
        if (i < v.size() - 1) std::cout << ", ";
    }
    std::cout << "}" << std::endl;
}

/// @brief 打印矩阵（行主序）
void print_matrix(const std::string& name, const std::vector<double>& m, int rows, int c
ols)
{
    std::cout << "  " << name << " [" << m.size() << "]:" << std::endl;
    for (int r = 0; r < rows; ++r) {
        std::cout << "    ";
        for (int c = 0; c < cols; ++c) {
            int idx = r * cols + c;
            if (idx < (int)m.size()) {
                std::cout << std::fixed << std::setprecision(4) << std::setw(10) << m[id
```

```
x] << " ";
            }
        }
        std::cout << std::endl;
    }
}

// ============================================================
// 主函数
// ============================================================

int main(int argc, char* argv[])
{
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    // 解析命令行参数: ./ex_coord_transform [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // ---- 获取 SDK 版本 ----
    {
        std::string version = tl::get_library_version();
        std::cout << "[信息] SDK 版本: " << version << std::endl;
    }

    // ---- 双端口连接 ----
    print_separator("连接控制器");
    tl::SOCKETFD sock = tl::connect_robot(ip, "6001");
    if (sock < 0) {
        std::cerr << "[ERROR] 连接 6001 端口失败" << std::endl;
        return -1;
    }
    tl::SOCKETFD sock_servo = tl::connect_robot(ip, "7000");
    if (sock_servo < 0) {
        std::cerr << "[ERROR] 连接 7000 端口失败" << std::endl;
        tl::disconnect_robot(sock);
        return -1;
    }
    std::cout << "[信息] 双端口连接成功 (6001=" << sock << ", 7000=" << sock_servo << ")" <
< std::endl;

    tl::Result ret;

    // ---- 上电（servoJ 需要上电状态）----
    {
        print_separator("清错与上电");
        tl::set_current_mode(sock, tl::RobotMode::TEACH);
        std::this_thread::sleep_for(std::chrono::milliseconds(300));
```

```
        tl::clear_error(sock);
        std::this_thread::sleep_for(std::chrono::milliseconds(300));
        tl::set_servo_poweroff(sock);
        std::this_thread::sleep_for(std::chrono::milliseconds(300));

        tl::ServoState servo_state;
        ret = tl::get_servo_state(sock, servo_state);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
            tl::disconnect_robot(sock);
            tl::disconnect_robot(sock_servo);
            return -1;
        }
        std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(servo_state)
                  << " (STOP=停止 READY=就绪 ALARM=报警 RUNNING=运行)" << std::endl;

        if (servo_state == tl::ServoState::ALARM) {
            std::cerr << "[ERROR] 伺服处于报警状态，无法上电" << std::endl;
            tl::disconnect_robot(sock);
            tl::disconnect_robot(sock_servo);
            return -1;
        }

        if (servo_state != tl::ServoState::RUNNING) {
            std::cout << "  [信息] 伺服未运行，执行上电流程..." << std::endl;
            ret = tl::set_servo_state(sock, tl::ServoState::READY);
            if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED)
{
                std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
                tl::disconnect_robot(sock);
                tl::disconnect_robot(sock_servo);
                return -1;
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(500));

            ret = tl::set_servo_poweron(sock);
            if (ret != tl::Result::SUCCESS) {
                std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
                tl::disconnect_robot(sock);
                tl::disconnect_robot(sock_servo);
                return -1;
            }
            std::this_thread::sleep_for(std::chrono::seconds(2));
            std::cout << "  [信息] 上电成功" << std::endl;
        } else {
            std::cout << "  [信息] 伺服已运行 (servo_state=RUNNING)" << std::endl;
        }
    }

    // ============================================================
    // 1. 四元数 ↔ 欧拉角 (RPY) — 正反验证
    // ============================================================
    // 单位说明:
```

```
//   四元数 [x, y, z, w]       — 无量纲（归一化）
//   欧拉角 [roll, pitch, yaw] — 弧度 rad
//   旋转矩阵 (3×3 行主序)     — 无量纲
print_separator("四元数 ↔ 欧拉角");

// 1a. 欧拉角 → 四元数 → 欧拉角（正反验证）
{
    std::cout << "--- 1a. RPY ↔ 四元数 正反验证 ---" << std::endl;
    std::vector<double> rpy_in = {0.2, 0.3, 0.5}; // rad
    std::vector<double> quat, rpy_out;

    ret = tl::get_rpy2quat(sock, rpy_in, quat);
    print_result("tl::get_rpy2quat(RPY→四元数)", ret);
    if (ret != tl::Result::SUCCESS) return -1;
    print_vector("  输入欧拉角 [rad]", rpy_in);
    print_vector("  中间四元数 [x, y, z, w]", quat);

    ret = tl::get_quat2rpy(sock, quat, rpy_out);
    print_result("tl::get_quat2rpy(四元数→RPY)", ret);
    if (ret != tl::Result::SUCCESS) return -1;
    print_vector("  还原欧拉角 [rad]", rpy_out);

    if (rpy_out.size() >= 3 && rpy_in.size() >= 3) {
        double err = fabs(rpy_out[0] - rpy_in[0])
                   + fabs(rpy_out[1] - rpy_in[1])
                   + fabs(rpy_out[2] - rpy_in[2]);
        std::cout << "  正反变换误差 (应接近0): " << err << std::endl;
    }
}

// 1b. 欧拉角 → 旋转矩阵
{
    std::cout << "\n--- 1b. get_rpy2r: 欧拉角转旋转矩阵 ---" << std::endl;
    std::vector<double> rpy = {0.0, 0.0, M_PI_2}; // rad
    std::vector<double> rot_mat;

    ret = tl::get_rpy2r(sock, rpy, rot_mat);
    print_result("get_rpy2r", ret);
    print_vector("输入欧拉角 [rad]", rpy);
    if (ret == tl::Result::SUCCESS) {
        print_matrix("输出旋转矩阵 (3x3, 行主序, 无量纲)", rot_mat, 3, 3);
    }
}

// ============================================================
// 2. 位姿矩阵 ↔ 旋转矩阵 — 正反验证
// ============================================================
// 单位说明:
//   旋转矩阵 R (3×3)     — 无量纲
//   平移向量 T [Tx,Ty,Tz] — 毫米 mm
//   位姿矩阵 TR (4×4)  — R 部分无量纲，T 部分单位 mm
// 位姿矩阵 (行主序 16 元素):
//   [Rxx Rxy Rxz Tx]  第 0 行
```

```
//   [Ryx Ryy Ryz Ty]  第 1 行
//   [Rzx Rzy Rzz Tz]  第 2 行
//   [  0   0   0  1]  第 3 行
print_separator("位姿矩阵 ↔ 旋转矩阵");

// 2a. TR → R → TR（正反验证）
{
    std::cout << "--- 2a. TR ↔ R 正反验证 ---" << std::endl;
    // 构造一个绕 Z 转 90°、平移 (100, 200, 300) mm 的位姿矩阵
    double cos90 = cos(M_PI_2), sin90 = sin(M_PI_2);
    std::vector<double> tr_in = {
        cos90, -sin90, 0.0, 100.0,
        sin90,  cos90, 0.0, 200.0,
        0.0,    0.0,   1.0, 300.0,
        0.0,    0.0,   0.0, 1.0
    };
    std::vector<double> R, tr_out;

    ret = tl::get_tr2r(sock, tr_in, R);
    print_result("tl::get_tr2r(TR→R)", ret);
    if (ret != tl::Result::SUCCESS) return -1;
    print_matrix("输入位姿矩阵 (4x4, R无量纲, T:mm)", tr_in, 4, 4);
    print_matrix("中间旋转矩阵 (3x3, 无量纲)", R, 3, 3);

    ret = tl::get_r2tr(sock, R, tr_out);
    print_result("tl::get_r2tr(R→TR)", ret);
    if (ret != tl::Result::SUCCESS) return -1;
    print_matrix("还原位姿矩阵 (4x4, R无量纲, T:mm)", tr_out, 4, 4);

    // 说明: get_tr2r 只提取旋转矩阵，平移 T 在此丢弃；
    // get_r2tr 的输入不含平移，还原出的 TR 平移部分恒为 0。
    // SDK 无 "R+T 构造 TR" 接口，完整位姿矩阵需由调用方自行拼装 T。
    std::cout << "  注: get_tr2r 仅输出旋转矩阵，平移 T 不参与往返；" << std::endl;
    std::cout << "      还原位姿矩阵的平移部分为 0，需调用方另行拼装。" << std::endl;

    // 对比旋转部分（TR→R→TR 链路仅保证旋转信息无损）
    // 注意: 行主序 4x4 中旋转块位于 0,1,2, 4,5,6, 8,9,10,
    //       位置 3/7/11 是平移列，不能按连续 0..8 遍历。
    if (tr_out.size() >= 16 && tr_in.size() >= 16) {
        const int rot_idx[9] = {0, 1, 2, 4, 5, 6, 8, 9, 10};
        double r_err = 0.0;
        for (int k = 0; k < 9; ++k) {
            r_err += fabs(tr_out[rot_idx[k]] - tr_in[rot_idx[k]]);
        }
        std::cout << "  旋转矩阵误差 (应接近0): " << r_err << std::endl;
    }
}

// ============================================================
// 3. 坐标系转换（获取机器人当前位姿在不同坐标系下的表示）
// ============================================================
// 单位说明:
//   关节坐标 [J1..J6] — 度 °
```

```
    //   基坐标 [X,Y,Z,Rx,Ry,Rz] — 位移 mm，姿态 rad
    //   工具/用户坐标 — 同基坐标
    print_separator("坐标系转换");
    {
        std::cout << "--- 3. get_origin_coord_to_target_coord ---" << std::endl;
        std::cout << "  坐标系: 0=关节(°)  1=基坐标(mm+rad)  2=工具(mm+rad)  3=用户
(mm+rad)" << std::endl;

        // 从关节坐标系(0) 转换到基坐标系(1)
        // 关节角度: J1=10°, J2=20°, J3=30°, J4=0°, J5=0°, J6=0°
        std::vector<double> joint_pos = {10, 20, 30, 0, 0, 0};
        std::vector<double> cartesian_pos;
        bool convert_state = false;

        ret = tl::get_origin_coord_to_target_coord(sock, tl::Coord::JOINT, joint_pos,
tl::Coord::BASE, cartesian_pos, convert_state);
        print_result("tl::get_origin_coord_to_target_coord(关节→基坐标) 正解", ret);
        print_vector("输入关节角度 [°]", joint_pos);
        if (ret == tl::Result::SUCCESS) {
            std::cout << "  输出基坐标 — 前3位为位移[mm]，后3位为姿态[rad]:" << std::endl;
            print_vector("  基坐标", cartesian_pos);
            if (cartesian_pos.size() >= 6) {
                std::cout << "    位移 X=" << cartesian_pos[0] << "mm, Y=" << cartesian_p
os[1]
                          << "mm, Z=" << cartesian_pos[2] << "mm" << std::endl;
                std::cout << "    姿态 Rx=" << cartesian_pos[3] << "rad, Ry=" << cartesia
n_pos[4]
                          << "rad, Rz=" << cartesian_pos[5] << "rad" << std::endl;
                std::cout << "    姿态换算(°): Rx=" << cartesian_pos[3] * 180.0 / M_PI
                          << ", Ry=" << cartesian_pos[4] * 180.0 / M_PI
                          << ", Rz=" << cartesian_pos[5] * 180.0 / M_PI << std::endl;
            }
        }

        // 逆解: 基坐标系(1) → 关节坐标系(0)
        // 使用正解输出的基坐标作为输入，验证正反解一致性
        if (ret == tl::Result::SUCCESS && cartesian_pos.size() >= 6) {
            std::vector<double> joint_back;
            ret = tl::get_origin_coord_to_target_coord(sock, tl::Coord::BASE, cartesian_
pos, tl::Coord::JOINT, joint_back, convert_state);
            print_result("tl::get_origin_coord_to_target_coord(基坐标→关节) 逆解", ret);
            std::cout << "  输入基坐标:" << std::endl;
            std::cout << "    位移 [mm]: X=" << cartesian_pos[0] << ", Y=" << cartesian_p
os[1]
                      << ", Z=" << cartesian_pos[2] << std::endl;
            std::cout << "    姿态 [rad]: Rx=" << cartesian_pos[3] << ", Ry=" << cartesia
n_pos[4]
                      << ", Rz=" << cartesian_pos[5] << std::endl;
            if (ret == tl::Result::SUCCESS) {
                print_vector("  逆解关节角度 [°]", joint_back);
                // 对比原始关节角度，计算误差
                double err = 0.0;
                for (size_t i = 0; i < joint_pos.size() && i < joint_back.size(); ++i) {
```

```
                    err += fabs(joint_pos[i] - joint_back[i]);
                }
                std::cout << "  正反解误差 (应接近0): " << err << std::endl;
            }
        }
    }

    // ============================================================
    // 4. 逆解关节坐标 → servoJ 下发运动
    // ============================================================
    // 演示流程: 给定基坐标 → 逆解到关节角 → 通过 servoJ 发送到控制器
    print_separator("逆解 + servoJ 运动");

    // 取一个基坐标点，逆解为关节角
    std::vector<double> target_pose = {300, 0, 100, 3.141, 0, 0}; // X=300mm, Y=0,
Z=100mm
    std::vector<double> target_joints;
    bool convert_state2 = false;

    ret = tl::get_origin_coord_to_target_coord(sock, tl::Coord::BASE, target_pose, tl::C
oord::JOINT, target_joints, convert_state2);
    print_result("逆解: 基坐标→关节", ret);
    if (ret == tl::Result::SUCCESS) {
        print_vector("目标基坐标 [mm, rad]", target_pose);
        print_vector("逆解关节角度 [°]", target_joints);

        // 通过 servoJ 发送关节角度到控制器
        // servoJ 需要连接 7000 端口，已在开头连接
        std::cout << "\n  --- 通过 servoJ 下发逆解结果 ---" << std::endl;

        // servoJ 约束参数（7 元素: 6 本体轴 + 1 外部轴）
        // 注意: SDK 底层协议要求 servoJ 相关向量固定为 7 元素
        std::vector<double> vmax = {30.0, 30.0, 30.0, 30.0, 30.0, 30.0, 30.0};
        std::vector<double> amax = {60.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0};
        std::vector<double> jmax = {100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0};

        ret = tl::open_servoJ(sock_servo, vmax, amax, jmax);
        print_result("open_servoJ", ret);
        if (ret == tl::Result::SUCCESS) {
            // 逐步发送逆解出的关节角度，确保 servoJ 跟踪到位
            // 分 50 步从当前角度平滑过渡到目标角度
            // q 必须为 7 元素，与 open_servoJ 一致
            std::vector<double> current = {0, 0, 0, 0, 0, 0, 0};
            int steps = 50;
            for (int i = 0; i < steps; ++i) {
                std::vector<double> q(7);
                size_t n_axes = target_joints.size();
                if (n_axes > q.size()) n_axes = q.size();
                for (size_t j = 0; j < n_axes; ++j) {
                    q[j] = current[j] + (target_joints[j] - current[j]) * (i + 1) / step
s;
                }
                // 多余元素由 q(7) 初始化保持 0，无需额外处理
```

```
                tl::set_servoJ_pos(sock_servo, q);
                std::this_thread::sleep_for(std::chrono::milliseconds(10));
            }
            // servoJ 是实时跟踪模式，等待固定时间确保运动到位
            std::cout << "  [信息] 等待 servoJ 运动到位..." << std::endl;
            std::this_thread::sleep_for(std::chrono::seconds(5));

            ret = tl::close_servoJ(sock_servo);
            std::this_thread::sleep_for(std::chrono::milliseconds(500));
            print_result("close_servoJ", ret);
        }
    }

    // ---- 断开连接 ----
    print_separator("断开连接");
    tl::set_current_mode(sock, tl::RobotMode::TEACH);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    tl::disconnect_robot(sock);
    tl::disconnect_robot(sock_servo);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    std::cout << "[信息] 已断开连接，机械臂切换到示教模式" << std::endl;

    std::cout << "\n[信息] 坐标变换示例程序运行完毕" << std::endl;
    return 0;
}
```

8.8 模式与 IO 控制接口示例

本节为模式与 IO 控制使用示例，完整源码见 `examples/cpp/ex_digital_io.cpp`。该示例演示设置机器人模式、设置数字 IO 输出、查询数字 IO 输出、获取所有 IO 输入状态以及查询流程图编程运行状态。完整代码如下：

```
/**
 * ex_digital_io.cpp
 * @brief  模式与 IO 控制接口（包括设置机械臂模式、设置数字 IO 输出、设置数字 IO 输出、获取所有 IO 输入状态）
 * @attention 确保机械臂供电正常、网络通信正常
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_digital_io
 */

#include <iostream>
#include <thread>
#include <chrono>
#include "tl_api.h"

#ifdef _WIN32
#include <windows.h>
```

```
#endif
void enable_servo(const int socket_fd)
{
  std::cout << "--- 检查伺服状态 ---" << std::endl;

  // 1. 先清除控制器错误和报警，再下电复位确保状态同步
  tl::clear_error(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));
  tl::set_servo_poweroff(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));

  // 2. 检查伺服状态
  tl::ServoState state;
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state)
            << " (ServoState::STOP/READY/ALARM/RUNNING)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
  }

  // 3. 上电流程
  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }

  std::cout << "  [信息] 上电使能..." << std::endl;
  ret = tl::set_servo_poweron(socket_fd);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
    return;
  }
  std::this_thread::sleep_for(std::chrono::seconds(2));

  // 4. 确认结果
  tl::get_servo_state(socket_fd, state);
  if (state == tl::ServoState::RUNNING) {
```

```
    std::cout << "[INFO] 机械臂使能上电成功（servo_state = " << static_cast<int>(state) <<
") " << std::endl;
  } else {
    std::cerr << "[ERROR] 机械臂使能上电失败（servo_state = " << static_cast<int>(state)
<< ") " << std::endl;
  }
}

void disable_servo(const int socket_fd)
{
  tl::ServoState state;
  tl::get_servo_state(socket_fd, state);
  switch (state)
  {
    case tl::ServoState::READY:
        std::cout << "[INFO] 机械臂已使能下电" << std::endl;
        break;
    case tl::ServoState::RUNNING:
        tl::set_servo_poweroff(socket_fd);
        tl::get_servo_state(socket_fd, state);
        std::cout << "[INFO] 机械臂使能下电成功（servo_state = " <<
static_cast<int>(state) << ") " << std::endl;
        return;
    default:
        break;
  }
  std::cout << "[INFO] 机械臂使能下电失败（servo_state = " << static_cast<int>(state) << ")
" << std::endl;
}

int main(int argc, char* argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    // 解析命令行参数: ./ex_digital_io [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
    }
    // 定义端口
    std::string port = "6001";

    // 获取 SDK 版本
    std::string version = tl::get_library_version();
    std::cout << "[INFO] SDK 版本: " << version << std::endl;

    // 连接控制器
    std::cout << "[INFO] 连接控制器..." << std::endl;
    auto socket_fd = tl::connect_robot(ip, port);
    if (socket_fd < 0) {
```

```
        std::cerr << "[ERROR] 连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 连接成功 (socket=" << socket_fd << ")" << std::endl;

    // 使能上电
    enable_servo(socket_fd);

    // ----设置机器人模式----
    std::cout << "[INFO] **********设置机器人模式**********" << std::endl;
    tl::RobotMode mode = tl::RobotMode::REMOTE;
    auto ret = tl::set_current_mode(socket_fd, mode);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 设置机器人模式失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 设置机器人模式成功" << std::endl;
    std::cout << std::endl;

    // ----设置数字 IO 输出----
    std::cout << "[INFO] **********设置数字 IO 输出**********" << std::endl;
    int io_port = 1;
    tl::IoLevel value = tl::IoLevel::OFF;
    ret = tl::set_digital_output(socket_fd, io_port, value);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 设置数字 IO 输出失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 设置数字 IO 输出成功" << std::endl;
    std::cout << std::endl;

    // ----查询数字 IO 输出----
    std::cout << "[INFO] **********查询数字 IO 输出**********" << std::endl;
    std::vector<int> outputValues;
    ret = tl::get_digital_output(socket_fd, outputValues);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 查询数字 IO 输出失败，程序退出!" << std::endl;
        return -1;
    }
    std::cout << "[INFO] 查询数字 IO 输出成功，状态为：";
    for (const auto& state : outputValues) {
        std::cout << state << " ";
    }
    std::cout << std::endl;

    // ----获取所有 IO 输入状态----
    std::cout << "[INFO] **********获取所有 IO 输入状态**********" << std::endl;
    std::vector<int> inputValues;
    ret = tl::get_digital_input(socket_fd, inputValues);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 查询数字 IO 输入失败，程序退出!" << std::endl;
        return -1;
    }
```

```
    std::cout << "[INFO] 查询数字 IO 输入成功，状态为：";
    for (const auto& state : inputValues) {
        std::cout << state << " ";
    }
    std::cout << std::endl;

    // ----查询流程图编程运行状态----
    std::cout << "[INFO] **********查询流程图编程运行状态**********" << std::endl;
    {
        tl::RunState run_status;
        ret = tl::get_robot_running_state(socket_fd, run_status);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 查询流程图编程运行状态失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 流程图编程运行状态: ";
            switch (run_status) {
                case tl::RunState::STOP: std::cout << "停止 (RunState::STOP)"; break;
                case tl::RunState::PAUSE: std::cout << "暂停 (RunState::PAUSE)"; break;
                case tl::RunState::RUNNING: std::cout << "运行 (RunState::RUNNING)"; brea
k;
                default: std::cout << "未知 (status=" << static_cast<int>(run_status) <<
")"; break;
            }
            std::cout << std::endl;
        }
    }
    std::cout << std::endl;

    // 使能下电
    tl::set_current_mode(socket_fd, tl::RobotMode::TEACH);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    disable_servo(socket_fd);

    // 断开控制器连接
    std::cout << "[INFO] 断开控制器连接..." << std::endl;
    ret = tl::disconnect_robot(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 断开连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 断开连接成功" << std::endl;

    return 0;
}
```

8.9 拖拽示教与轨迹记录/回放示例

本节为拖拽示教与轨迹记录/回放使用示例，完整源码见 `examples/cpp/ex_track_queue_drag.cpp`。该示例演示切换示教模式、设置拖拽方式、切换拖拽示教类型、轨迹记录（开始→手动拖拽→关闭）、轨迹保存、轨迹回放与轨迹清除。完整代码如下：

```cpp
/**
 * ex_track_queue_drag.cpp
 * @brief 拖拽示教 + 轨迹记录/回放示例
 * @attention
 *   - 确保机械臂供电正常、网络通信正常
 *   - 拖拽需在示教模式（tl::RobotMode::TEACH）下进行，示教模式下也可直接回放
 *   - 拖拽期间请注意安全，确保周围无人
 * @note 运行步骤
 *       编译: cd build && cmake .. && make
 *       运行: ./ex_track_queue_drag
 */

#include <iostream>
#include <thread>
#include <chrono>
#include <string>
#include <ctime>
#include "tl_api.h"

void enable_servo(const int socket_fd)
{
  std::cout << "--- 检查伺服状态 ---" << std::endl;

  tl::clear_error(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));
  tl::set_servo_poweroff(socket_fd);
  std::this_thread::sleep_for(std::chrono::milliseconds(300));

  tl::ServoState state;
  auto ret = tl::get_servo_state(socket_fd, state);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 获取伺服状态失败: " << ret << std::endl;
    return;
  }
  std::cout << "  [信息] 当前伺服状态: " << static_cast<int>(state)
            << " (ServoState::STOP/READY/ALARM/RUNNING)" << std::endl;

  if (state == tl::ServoState::ALARM) {
    std::cerr << "[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试" << std::endl;
    return;
  }

  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂已在使能上电状态" << std::endl;
    return;
  }

  if (state == tl::ServoState::STOP) {
    std::cout << "  [信息] 伺服处于停止状态，设置就绪..." << std::endl;
    ret = tl::set_servo_state(socket_fd, tl::ServoState::READY);
    if (ret != tl::Result::SUCCESS && ret != tl::Result::OPERATION_NOT_ALLOWED) {
      std::cerr << "[ERROR] 设置伺服就绪失败: " << ret << std::endl;
```

```
      return;
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
  }

  std::cout << "  [信息] 上电使能..." << std::endl;
  ret = tl::set_servo_poweron(socket_fd);
  if (ret != tl::Result::SUCCESS) {
    std::cerr << "[ERROR] 上电失败: " << ret << std::endl;
    return;
  }
  std::this_thread::sleep_for(std::chrono::seconds(2));

  tl::get_servo_state(socket_fd, state);
  if (state == tl::ServoState::RUNNING) {
    std::cout << "[INFO] 机械臂使能上电成功（servo_state = " << static_cast<int>(state) <<
") " << std::endl;
  } else {
    std::cerr << "[ERROR] 机械臂使能上电失败（servo_state = " << static_cast<int>(state)
<< ") " << std::endl;
  }
}

void disable_servo(const int socket_fd)
{
  tl::ServoState state;
  tl::get_servo_state(socket_fd, state);
  switch (state)
  {
    case tl::ServoState::READY:
        std::cout << "[INFO] 机械臂已使能下电" << std::endl;
        break;
    case tl::ServoState::RUNNING:
        tl::set_servo_poweroff(socket_fd);
        tl::get_servo_state(socket_fd, state);
        std::cout << "[INFO] 机械臂使能下电成功（servo_state = " <<
static_cast<int>(state) << "）" << std::endl;
        return;
    default:
        break;
  }
  std::cout << "[INFO] 机械臂使能下电失败（servo_state = " << static_cast<int>(state) << ")
" << std::endl;
}

int main(int argc, char* argv[]) {
    // 解析命令行参数: ./ex_track_queue_drag [-ip 192.168.1.13]，默认 192.168.1.13
    std::string ip = "192.168.1.13";
    for (int i = 1; i + 1 < argc; ++i) {
        if (std::string(argv[i]) == "-ip" || std::string(argv[i]) == "--ip") {
            ip = argv[i + 1];
            break;
        }
```

```
    }
    std::string port = "6001";

    std::string version = tl::get_library_version();
    std::cout << "[INFO] SDK 版本: " << version << std::endl;

    // 连接控制器
    std::cout << "[INFO] 连接控制器..." << std::endl;
    auto socket_fd = tl::connect_robot(ip, port);
    if (socket_fd < 0) {
        std::cerr << "[ERROR] 连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 连接成功 (socket=" << socket_fd << ")" << std::endl;

    // 使能上电
    enable_servo(socket_fd);

    // ========================================================================
    // 第一部分：拖拽示教 + 轨迹记录/回放
    // ========================================================================

    // ----切换为示教模式（拖拽需要在示教模式下执行）----
    std::cout << "[INFO] **********切换为示教模式**********" << std::endl;
    {
        auto ret = tl::set_current_mode(socket_fd, tl::RobotMode::TEACH);  //
RobotMode::TEACH（示教模式）
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 切换示教模式失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 已切换为示教模式" << std::endl;
        }
    }
    std::cout << std::endl;

    // ----设置拖拽方式（MOUSE_3D，需在切拖拽类型前设置）----
    std::cout << "[INFO] **********设置拖拽方式（MOUSE_3D）**********" << std::endl;
    {
        tl::DragMode mode = tl::DragMode::MOUSE_3D;  // MOUSE_3D（3D鼠标）
        auto ret = tl::set_darg_mode(socket_fd, mode);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 设置拖拽模式失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 设置拖拽模式成功（mode = " << static_cast<int>(mode) << "
，MOUSE_3D）" << std::endl;
        }
    }
    std::cout << std::endl;

    // ----设置示教类型为拖拽（0=点动 1=拖拽）----
    std::cout << "[INFO] **********设置示教类型为拖拽**********" << std::endl;
    {
        auto ret = tl::set_teach_type(socket_fd, tl::TeachType::DRAG);  //
```

```
TeachType::DRAG（拖拽）
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 设置示教类型失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 已切换为拖拽示教类型" << std::endl;
        }
    }
    std::cout << std::endl;

    // ----轨迹记录开始（记录拖拽路径）----
    std::cout << "[INFO] **********轨迹记录开始**********" << std::endl;
    {
        double maxSamplingNum = 10000;   // 最大采样点数 [200, 12000]
        double samplingInterval = 1;   // 采样间隔 [0.03, 1]
        auto ret = tl::track_record_start(socket_fd, maxSamplingNum, samplingInterval);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 轨迹记录开始失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 轨迹记录已开始（maxSamplingNum=" << maxSamplingNum
                      << ", samplingInterval=" << samplingInterval << "s) " <<
std::endl;
        }
    }
    std::cout << std::endl;

    // 查询轨迹记录状态
    {
        bool recordStatus = false;
        auto ret = tl::get_track_record_status(socket_fd, recordStatus);
        if (ret == tl::Result::SUCCESS) {
            std::cout << "[INFO] 轨迹记录状态: " << (recordStatus ? "开启" : "关闭") <<
std::endl;
        }
    }

    // 等待用户手动拖拽记录轨迹（用户拖拽完成后按回车继续）
    std::cout << "[INFO] 机械臂已进入拖拽状态，请手动拖拽机械臂记录轨迹..." << std::endl;
    std::cout << "[INFO] 拖拽完成后请按 Enter 键继续..." << std::endl;
    std::cin.get();

    // ----轨迹记录关闭----
    std::cout << "[INFO] **********轨迹记录关闭**********" << std::endl;
    {
        auto ret = tl::track_record_stop(socket_fd);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 轨迹记录关闭失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 轨迹记录已关闭" << std::endl;
        }
    }
    std::cout << std::endl;

    // ----退出拖拽模式（回放前需切回点动）----
```

```
    std::cout << "[INFO] **********退出拖拽模式**********" << std::endl;
    {
        auto ret = tl::set_teach_type(socket_fd, tl::TeachType::JOG);  // TeachType::JOG
（点动模式）
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 切回点动模式失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 已切回点动模式" << std::endl;
        }
    }
    std::cout << std::endl;

    // ----轨迹记录保存（先保存到控制器，再回放）----
    std::cout << "[INFO] 等待1秒后保存..." << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    std::cout << "[INFO] **********轨迹记录保存**********" << std::endl;
    {
        std::time_t now = std::time(nullptr);
        std::tm* local = std::localtime(&now);
        char buf[64];
        std::strftime(buf, sizeof(buf), "轨迹名-%m月%d日%H时%M分%S", local);
        std::string trajName(buf);
        auto ret = tl::track_record_save(socket_fd, trajName);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 轨迹记录保存失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 轨迹记录已保存（trajName = " << trajName << "）" << std::e
ndl;
        }
    }
    std::cout << std::endl;

    // ----轨迹回放（复现已保存的轨迹）----
    std::cout << "[INFO] 等待1秒后开始回放..." << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(4));
    std::cout << "[INFO] **********轨迹回放（复现拖拽路径）**********" << std::endl;
    {
        int vel = 100;  // 回放速度
        auto ret = tl::track_record_playback(socket_fd, vel);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 轨迹回放失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 轨迹回放开始（vel = " << vel << "），等待完成..." << std::e
ndl;
            tl::RunState runningState;
            int timeout = 0;
            do {
                std::this_thread::sleep_for(std::chrono::milliseconds(500));
                tl::get_robot_running_state(socket_fd, runningState);
                timeout++;
            } while (runningState != tl::RunState::STOP && timeout < 120);
            if (runningState == tl::RunState::STOP) {
                std::cout << "[INFO] 轨迹回放完成" << std::endl;
```

```
            }
        }
    }
    std::cout << std::endl;

    // ----轨迹清除（清除缓冲区）----
    std::cout << "[INFO] **********轨迹清除**********" << std::endl;
    {
        auto ret = tl::track_record_delete(socket_fd);
        if (ret != tl::Result::SUCCESS) {
            std::cerr << "[ERROR] 轨迹清除失败，错误码: " << ret << std::endl;
        } else {
            std::cout << "[INFO] 轨迹已清除" << std::endl;
        }
    }
    std::cout << std::endl;

    // 使能下电
    disable_servo(socket_fd);

    // 断开控制器连接
    std::cout << "[INFO] 断开控制器连接..." << std::endl;
    auto ret = tl::disconnect_robot(socket_fd);
    if (ret != tl::Result::SUCCESS) {
        std::cerr << "[ERROR] 断开连接失败，程序退出!"<< std::endl;
        return -1;
    }
    std::cout << "[INFO] 断开连接成功" << std::endl;

    return 0;
}
```

附录（一）

返回值说明

返回值	常量	说明
0	SUCCESS	成功
-1	RECEIVE_FAILED	接收失败
-2	DISCONNECT	未连接
-3	PARAM_ERR	参数错误
-4	OPERATION_NOT_ALLOWED	操作不被允许
-5	EXCEPTION	异常
-6	TIMEOUT	超时

返回值	常量	说明
---- ---- ----		

部分接口可以看cpp 文件的代码注释

服务热线：18090080872

四川天链机器人股份有限公司

地址：四川省绵阳高新区火炬西街南段10号

http://www.tlibot.com

E-mail:TLIBOT@163.com

本产品的额定功率、规格、外部尺寸等

如需改良而进行变更，恕不另行通告。

技术数据和插图仅作为供货参考，保留更改权利。